11. Claude Code × Next.js バイブコーディング完全ガイド

Claude Codeを使ってNext.jsアプリをバイブコーディング（自然言語で指示を出しながらAIと協調開発）するための、実践的な作業手順書です。Claude Codeの全機能（Settings、Hooks、Skills、Rules、MCP、Permissions、Plan Mode、Worktrees、Subagents、Memory、Keybindings）を網羅し、各フェーズで最適な機能を活用する方法を解説します。

この章は完全な初学者向けです。「AIエージェントってそもそも何？」というところから始めて、本格的な開発フローまで一気通貫で解説します。読みながらわからない用語が出てきたら、各セクションの冒頭の用語解説に戻って確認してください。

この章のゴールと地図

この章を読み終えると、次のことができるようになります。

- Claude Code が「指示 → 計画 → 調べる → 変更する → 確かめる → 報告する」というループで動いていることを、自分の言葉で説明できる
- あいまいな指示を「目的・場所と範囲・完成条件・確かめ方」がそろった指示に書き直せる
- 権限設定（Permissions）とプランモード（Plan Mode）を使い、AI に勝手な操作をさせずに作業を進められる
- Git のブランチとこまめなコミットを「命綱」にして、失敗してもすぐに元へ戻せる
- CLAUDE.md・Rules・Skills・Hooks・Subagents・MCP のうち「いまどれを使えばいいか」を選べる
- AI が書いたコードを鵜呑みにせず、差分・型チェック・テストで検証してから採用できる

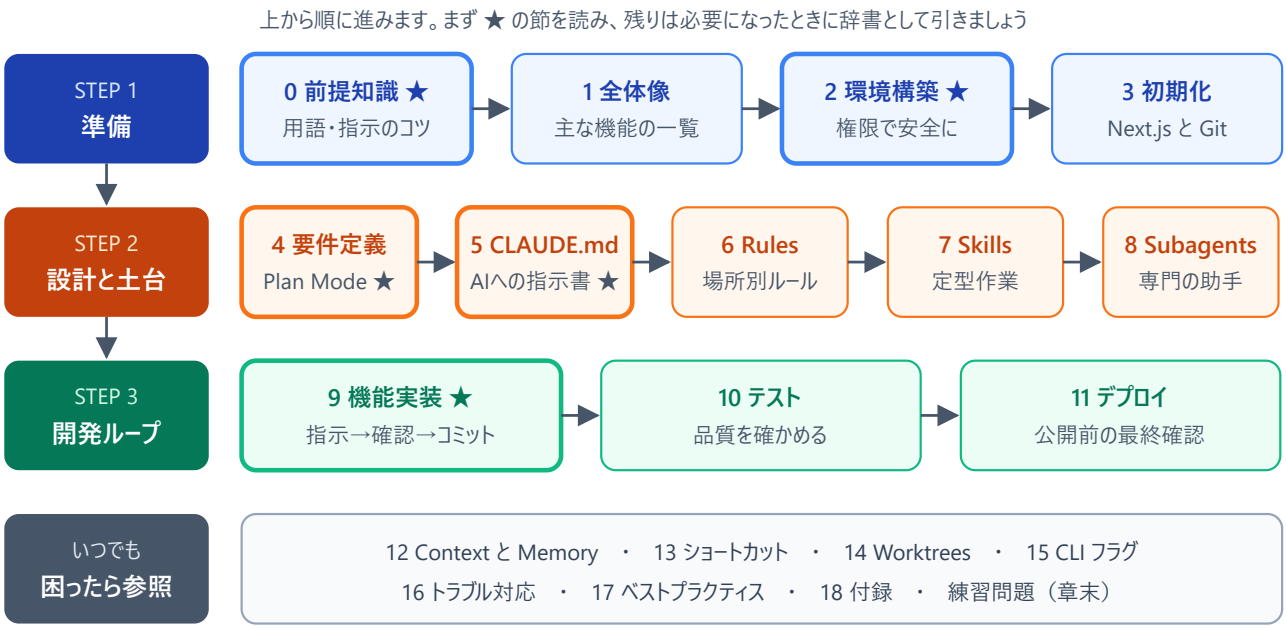


図11-1: この章の地図。★ の節（0・2・4・5・9）が「最低限ここだけは」の部分です

> **ヒント:** この章はとても長いです。Claude Code の機能を網羅した「辞書」の役割も兼ねているためです。初めて読むときは図11-1 の ★ の節を先に読み、残りは「CLAUDE.md って何だっけ？」「MCP を設定したい」と思ったときに引く使い方がおすすめです。

▼ この章の前提

- 第1〜10章で作った書籍管理アプリ（Next.js + TypeScript + Supabase）が手元にあること（なくても読めます）
- ターミナルで `cd` や `npm run dev` を実行したことがあること
- Git の `git add` / `git commit` を使ったことがあること（第1章で扱いました）

> **注意:** 2〜11節（Phase 0〜9）のプロンプト例は、書籍管理アプリより少し大きい「タスク管理アプリ TaskFlow」を題材にしています。大きなアプリでも同じ進め方が通用することを見せるためです。一方、この章で新しく追加した「NG→OK 比較」「実践ウォークスルー」などの例は、第1〜10章で作った**書籍管理アプリ**を題材にしています。TaskFlow の例は `src/` フォルダを使う構成ですが、書籍管理アプリは第6章のとおり `src/` を使わず、`app/`・`components/`・`lib/` をプロジェクト直下に置いています（例: `components/StatusBadge.tsx`、`lib/supabase/server.ts`）。パスの違いに注意してください。

---

0. 前提知識: AIアシスタントと一緒にコードを書くって何？

0.0 まず押さえておきたい言葉たち

ここから出てくる言葉を、最初にまとめて整理しておきます。雰囲気だけでもつかんでおくと、以降の説明がぐっと読みやすくなります。

- **AI（人工知能）**: コンピュータが「人間っぽい判断」をしてくれる技術の総称。
- **LLM（Large Language Model、大規模言語モデル）**: 大量の文章を学習した、自然言語を扱うAIの一種。Claude や ChatGPT の中身がこれです。
- **AIエージェント**: LLMに「ファイルを読む」「コマンドを実行する」などの行動能力（ツール）を持たせ、目的に向かって自律的に動けるようにした仕組み。Claude Code はまさにこのAIエージェントです。
- **コンテキストウィンドウ**: AIが一度に「読んで覚えていられる」文字数の上限。会話履歴・読み込んだファイル・指示文の全部がここに収まる必要があります。日本語だとざっくり数十万字ぶん。
- **トークン**: AIが文字を扱う最小単位。日本語1文字でだいたい1〜2トークン。コンテキストウィンドウの上限はトークン数で決まります。
- **プロンプト**: AIに渡す指示文のこと。「〜してください」と書くテキストそのもの。
- **プロンプトエンジニアリング**: AIから良い答えを引き出すために、プロンプトの書き方を工夫する技術。指示を具体的にする・例を見せる・ステップを区切る、などのコツがあります。

- **ハルシネーション**: AIがもっともらしいウソをつく現象。実在しない関数名を呼んだり、間違ったAPIを使ったりすることがあるため、必ず人間が検証する必要があります。
- **ツール呼び出し（tool use / function calling）**: AIエージェントが「ファイル読む」「コマンド打つ」などの行動を起こすときの仕組み。Claude Code では `Read` `Edit` `Bash` などのツールがあらかじめ用意されています。
- **コードレビュー**: 書いたコードを他の人（または別のAI）が読み、バグ・規約違反・改善点を指摘する作業。バイブコーディングでは「自分がレビュアー」になります。
- **ペアプロ（ペアプログラミング）**: 2人で1台のパソコンに向かい、1人がコードを書き、もう1人が横で見ながら助言する開発スタイル。バイブコーディングは「相手がAI」のペアプロだと考えると分かりやすいです。

これらの用語は本章で何度も出てくるので、いったん流し読みして OK です。

0.1 「バイブコーディング」とは

バイブコーディング（vibe coding） とは、「AIに自然言語で指示しながら、AIが書いたコードを確認・修正していく開発スタイル」のことです。`feel like coding`（コードを書いている雰囲気を味わう）のニュアンスで、自分はざっくりした方針を伝え、細部はAIに任せる感じです。

従来のプログラミングは「手で1行ずつコードを打つ」のが当たり前でしたが、バイブコーディングでは「日本語で要件を伝える → AIが実装する → 自分はそれをレビュー・指示」というサイクルになります。前述のペアプロのうち、相方がAIになったイメージです。

▼ 従来の開発スタイルとの違い:

観点	従来	バイブコーディング
コードを書く主体	自分	主にAI（自分はレビュー）
学習方法	ドキュメントを読む	AIに「これは何？」と聞く
エラー対処	Google検索	エラーをAIに貼って質問
実装速度	遅い〜中	非常に速い
コード理解度	高い	油断すると低くなる

> **重要な注意:** バイブコーディングはコードの「品質保証は自分の責任」です。AIは間違いも書きます（前述のハルシネーション）。**生成されたコードを必ず読んで理解する**習慣を捨てないでください。

▼ たとえ話: あなたは「現場監督」、Claude Code は「腕のいい大工さん」

家を建てる現場を想像してください。大工さんは図面を読み、材料を切り、道具を使って、驚くほど速く作業を進めてくれます。でも「どんな家を建てるか」「この柱の位置で本当にいいか」を決めて、出来上がりを確認するのは現場監督の仕事です。大工さんに「いい感じの家を建てて」とだけ言えば、腕が良くても、施主の希望と違う家ができてしまいます。

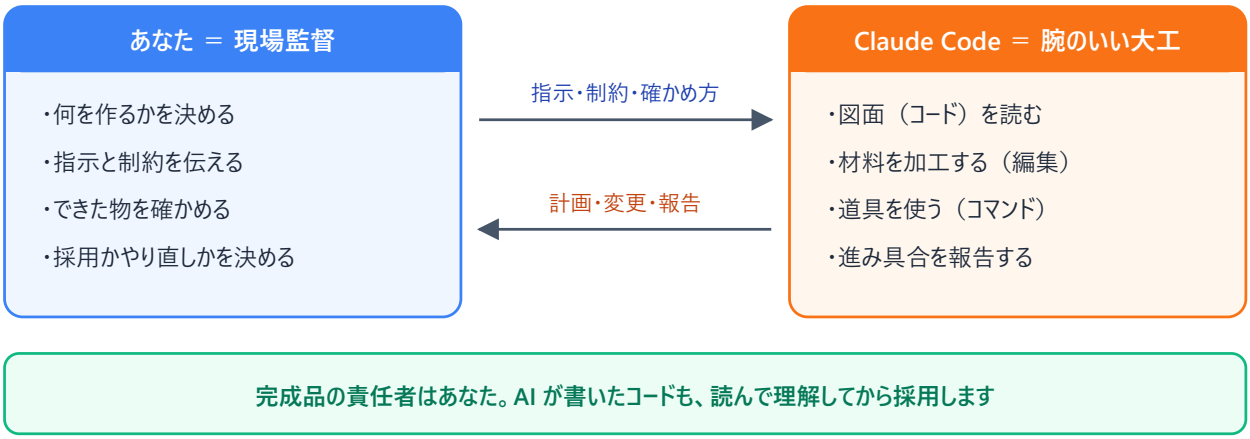


図11-2: バイブコーディングの役割分担。手を動かすのは AI、決めて確かめるのは人間

この役割分担を忘れて「全部おまかせ」にすると、動くけれど誰も中身を説明できないアプリができあがります。以降の節で紹介するテクニックは、すべて「現場監督として上手に指示し、上手に確かめる」ための道具だと考えてください。

0.1.1 なぜAIに「お願い」だけでなく「文脈」と「制約」を渡すと結果が良くなるのか

これがバイブコーディングの肝です。LLM は確率的に「次に来そうな文字」を選ぶ仕組みなので、入力に含まれる情報が多いほど、より正確で意図に近い答えを出せます。

たとえば「ボタンを作って」と言うだけだと、AIは「どんなボタン？色は？クリックしたら何が起きる？フレームワークは何？」を全部勝手に推測（つまり当てずっぽう）で埋めます。逆に「タスク一覧画面の右上に、Tailwind の bg-blue-600 でスタイリングした追加ボタン。クリックで TaskFormModal を開く。shadcn/ui の Button を使う」と書けば、推測の余地が無くなり、意図どおりのコードが返ってきます。

このため本章では「仕様」「制約」「参考ファイル」「禁止事項」を丁寧にプロンプトに書き込む方法を繰り返し紹介します。

0.1.2 指示文の NG→OK：同じお願いでも結果がこう変わる

「具体的に書く」と言われても、何を書けばよいのか迷いますよね。良い指示に入っている要素は、次の図の4つ（＋制約）にほぼ集約できます。



図11-3: 良い指示に入れる4つの要素。迷ったらこの4つが書けているかを確認します

実際に、第7章で作った `StatusBadge`（読書ステータスを色付きラベルで表示する部品）を少し変える指示で試してみましょう。

指示文の例1:「読みたい」バッジを灰色の「積読」に変える

> **▼ この指示文がやること（先に日本語で）**：StatusBadge のうち「読みたい（want_to_read）」の見た目だけを、灰色の「積読（つんどく：買ったまま読んでいない本）」ラベルに変えてもらう指示です。図11-3 の4要素（目的・場所と範囲・完成条件・確かめ方）と制約を、見出し付きで全部書いています。

```
components/StatusBadge.tsx の statusConfig だけを変更してください。

目的:
まだ読んでいない本（want_to_read）を、読書中・読了の本より目立たなくしたい。

変更内容:
- want_to_read のラベルを「読みたい」から「積読」に変える
- 色を黄色から灰色に変える（背景 bg-gray-100、文字 text-gray-700）

守ること:
- reading と completed の設定、ほかのファイルは変更しない
- 新しいライブラリは追加しない

確かめ方:
- 変更後に npm run lint を実行し、結果を報告してください
```

▼ 実行結果（Claude Code が行った変更を `git diff` で表示したもの）

```
diff --git a/components/StatusBadge.tsx b/components/StatusBadge.tsx
index 54f973a..229c542 100644
--- a/components/StatusBadge.tsx
+++ b/components/StatusBadge.tsx
@@ -19,8 +19,8 @@ const statusConfig: Record<
     textColor: "text-green-800",
   },
   want_to_read: {
-    label: "読みたい",
-    bgColor: "bg-yellow-100",
-    textColor: "text-yellow-800",
+    label: "積読",
+    bgColor: "bg-gray-100",
+    textColor: "text-gray-700",
   },
 };
```

```
 components/StatusBadge.tsx | 6 ++++---
 1 file changed, 3 insertions(+), 3 deletions(-)
```

差分の下の2行は `git diff --stat`（変更の量だけを表示するコマンド）の結果です。「1ファイル・3行追加・3行削除」と、頼んだ範囲だけが変わったことがひと目で分かります。

▼ コードを1つずつ分解して解説

解説1: 1行目で「場所と範囲」を宣言する

```
components/StatusBadge.tsx の statusConfig だけを変更してください。
```

- ファイル名と、その中のどの部分（`statusConfig`）を触るのかを最初に書いています。
- AI はプロジェクト全体を探し回らずに済み、関係ないファイルを書き換える心配も減ります。
- 「だけ」という言葉が、範囲を絞る一番かんたんな道具です。

解説2: 「目的」は AI が迷ったときの道しるべ

```
目的:
まだ読んでいない本（want_to_read）を、読書中・読了の本より目立たなくしたい。
```

- 変更内容だけでなく「なぜ変えるのか」を書くと、指示に書いていない細かい判断（どの灰色にするか等）を、目的に沿った方向で決めてくれます。
- 人間の同僚にお願いするときと同じで、理由が分かっているほうが仕事の質は上がります。

解説3:「守ること」で余計な親切を止める

```
守ること:
- reading と completed の設定、ほかのファイルは変更しない
- 新しいライブラリは追加しない
```

- AI は気を利かせて「ついでにアイコンも付けておきました」「ほかのバッジの色も統一しました」と範囲を広げがちです。
- 触ってほしくない場所を先に書いておくと、差分が小さくなり、あなたのレビューも楽になります。

解説4:「確かめ方」まで頼むと、AI が自分で検証してから報告する

```
確かめ方:
- 変更後に npm run lint を実行し、結果を報告してください
```

- `npm run lint` は、コードの書き方の誤りを機械的にチェックするコマンドです（第6章でプロジェクトを作るときに ESLint を有効にしました）。
- 確かめ方を伝えておくと、Claude Code は変更後にこのコマンドを実行し、エラーが出れば自分で直してから報告します。公式のベストプラクティスでも「Claude が自分の作業を確かめる手段（テスト・ビルド・スクリーンショットなど）を渡す」ことが最も効果の高いコツとして挙げられています。

> **用語: lint（リント）**：プログラムを実行せずに、書き方の誤りや危ないパターンを見つけてくれるチェックのこと。ESLint はその代表的な道具です。

解説5: 差分（diff）の読み方

```
   want_to_read: {
-    label: "読みたい",
+    label: "積読",
```

- 先頭が `-` の行は「消えた行」、`+` の行は「増えた行」、記号のない行は変わっていない前後の行（文脈）です。
- `@@ -19,8 +19,8 @@` は「変更前の19行目から8行ぶん、変更後の19行目から8行ぶん」という場所の目印です。
- AI の仕事の確認は、この差分を読むことから始まります。

> **つまずきポイント①:「いい感じに」は AI にとって一番むずかしい注文**
>
> 「分かりやすく」「いい感じに」「きれいに」は、人によって意味が違う言葉です。AI はその意味を自分で推測するため、アイコンを追加したり、関係ないファイルまで整えたりと、あなたの想像と違う変更を返しがちです。図11-3 の4要素に分解して書き直しましょう。

NG（あいまい）：

```
ステータスの表示をもっと分かりやすくして
```

OK（具体的）：

```
components/StatusBadge.tsx の statusConfig だけを変更してください。
want_to_read のラベルを「積読」に、色を灰色（bg-gray-100 / text-gray-700）に変えてください。
ほかのステータスとほかのファイルは変更しないでください。
```

> **つまずきポイント②: 確かめ方を伝えないと「たぶん動きます」で終わる**
>
> 確かめ方を書かないと、Claude Code はコードを書いた時点で「実装しました」と報告することがあります。実際に動くかどうかは、あなたが後で気づくことになります。最初から「何をもって完成とするか」と「それを確かめるコマンド」を渡しましょう。

NG（確かめ方がない）：

```
書籍一覧に並べ替え機能を作って
```

OK（確かめ方つき）：

```
lib/sortBooks.ts に、書籍の配列を並べ替える関数 sortBooks を作ってください。
並べ方は「タイトル順」「評価の高い順」「新しい順」の3種類です。
同じフォルダに Vitest のテスト sortBooks.test.ts も作り、
npx vitest run を実行して、全テストが通るまで修正してから報告してください。
```

> **つまずきポイント③: 範囲を絞らないと、差分が読み切れなくなる**
>
> 「全体をリファクタリングして」のような広い指示は、数十ファイルにわたる大きな差分を生みます。どこが変わったのか追いきれず、レビューが形だけになってしまいます。1回の指示で変えるのは、自分が差分を読み切れる量（目安: 数ファイル・百行程度まで）にしましょう。

NG（範囲が広すぎる）：

```
コード全体をきれいにリファクタリングして
```

OK（範囲を絞る）：

```
components/BookCard.tsx だけを対象に、読みやすさのための整理をしてください。
画面の見た目と動作は変えないでください。
変更後に npm run lint と npx tsc --noEmit を実行して結果を報告し、そこで止まってください。
```

> **用語: リファクタリング**: プログラムの動作を変えずに、中身（書き方や構造）を整理して読みやすくすること。

0.2 Claude Code でできることのイメージ

ターミナルで Claude Code を起動して、以下のような自然な指示が通ります。

▼ ユーザーの入力例:

```
書籍管理アプリの新規登録ページを実装して。
- /books/new でアクセスできる
- title, author, status をフォームに含める
- 送信時に Supabase の books テーブルに insert する
- 完了したら一覧ページにリダイレクト
```

このように、URL・必要なフィールド・処理内容・遷移先まで明示してあると、AI はほぼ迷わずに作業できます。

▼ Claude Codeの動作（イメージ）:

```
[Claude] 了解しました。以下の手順で実装します。
         1. app/books/new/page.tsx を作成
         2. components/BookForm.tsx を作成
         3. Server Action createBook を追加

         [Edit] app/books/new/page.tsx を作成中...
         [Edit] components/BookForm.tsx を作成中...
         [Edit] app/actions/books.ts を更新中...

         実装が完了しました。/books/new にアクセスして動作確認してください。
```

`[Edit]` の部分は、Claude Code が `Edit` ツール（ファイルを変更するツール）を呼んでいることを示しています。Claude Code は内部に複数のツールを持っており、状況に応じて使い分けます。主要なツールは次のとおりです。

ツール名	役割
`Read`	ファイルの中身を読む
`Edit`	既存ファイルの一部を書き換える（差分編集）
`Write`	ファイルを新規作成、または全体を上書き
`Grep`	ファイル内容を正規表現で検索（中身検索）
`Glob`	ファイル名パターン（`**/*.tsx` など）で検索
`Bash`	シェルコマンドを実行（`npm install` など）
`Agent` / `Task`	サブエージェントを起動して別タスクに委任

> **ヒント:** Windows で Git for Windows が入っていない場合、Claude Code は `Bash` の代わりに `PowerShell` ツールでコマンドを実行します。役割は同じ「コマンドを実行する道具」です。

Claude Code の作業ループ（指示 → 計画 → 調べる → 変更 → 確かめる → 報告）

Claude Code は、あなたの指示を1回で「えいっ」と仕上げているわけではありません。上の表のツールを何度も使い分けながら、次の図のようなループを回しています。

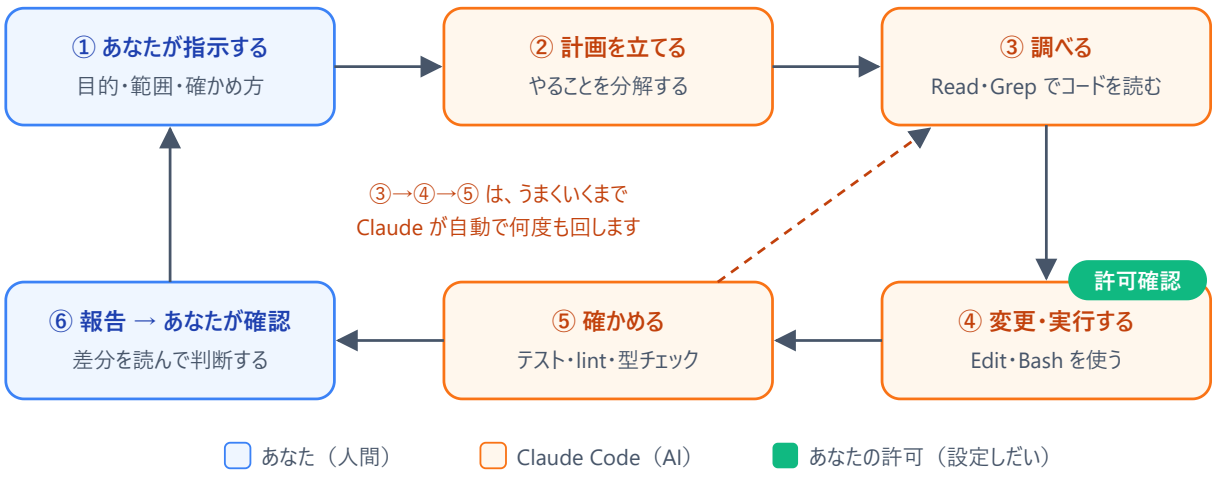


図11-4: Claude Code の作業ループ。人間が関わるのは①と⑥、そして④の許可確認です

ループの各段階で、あなたにできることを整理しておきます。

段階	Claude Code がしていること	あなたにできること
① 指示	あなたの文章を読む	目的・場所と範囲・完成条件・確かめ方を書く（図11-3）
② 計画	手順を分解し、TODO リストにする	大きな作業ではプランモードで計画だけ先に出させる（4節）
③ 調べる	`Read` / `Grep` / `Glob` で関係するコードを読む	参考ファイルを `@ファイル名` で教えて探す手間を省く
④ 変更・実行	`Edit` / `Write` / `Bash` でファイルを変えたりコマンドを打つ	許可確認の画面で中身を読んでから許可する（下の図11-5）
⑤ 確かめる	テストや lint を実行し、失敗したら③に戻って直す	「どのコマンドで確かめるか」を指示に書いておく
⑥ 報告	何をしたかをまとめて伝える	`git diff` で差分を読み、採用するか決める（0.4.1）

> **ヒント:** ループの途中でも、`Esc` キーを押せば Claude Code の作業をその場で止められます。「あ、方向が違う」と思ったら、早めに止めて指示を出し直すのがコツです（公式ドキュメントでも「軌道がずれたと気づいたらすぐに修正する」ことが推奨されています）。

画面イメージ:「このコマンドを実行していい？」と聞かれたら

④の段階で、Claude Code はファイルの編集やコマンドの実行の前に、あなたの許可を求めることがあります（どの操作で聞かれるかは、2-2 の権限設定と 2-3 のモードで決まります）。ターミナルでは、だいたい次のように表示されます。



図11-5: 画面イメージ（Bash コマンドの許可確認）。文言や並びは Claude Code のバージョンによって少し変わります

選択肢の意味は次のとおりです。

選択肢	意味	どんなときに選ぶ？
`Yes`	今回だけ許可する	中身を読んで問題ないとき（基本はこれ）
`Yes, and don't ask again for: …`	同じ種類のコマンドを今後は聞かずに許可する	`npm run lint` のように安全で何度も使うコマンド
`No`	実行させない	中身が意図と違う、危なそう、よく分からないとき

- 「don't ask again」を選ぶと、Bash コマンドの場合は許可ルールが `.claude/settings.local.json` に保存され、次回以降のセッションでも有効になります。ファイル編集の許可は、そのセッションが終わるまでだけ有効です。
- `Yes` か `No` にカーソルを合わせて `Tab` を押すと、コメント欄が開きます。`No` と一緒に「`npm run build` ではなく `npm run lint` だけにして」のように理由を書くと、Claude はそれを踏まえて作業を続けます。
- 自動モード（auto mode）が使える環境では、`Yes, and switch to auto mode`（許可して自動モードに切り替える）という選択肢が増えることがあります。慣れるまでは選ばないほうが安心です（2-3 で説明します）。

> **つまずきポイント④: 中身を読まずに Enter を連打してしまう**
>
> 許可確認が何度も出ると、つい中身を見ずに `Yes` を押したくなります。でもこれは、現場監督が図面を見ずにハンコを押すのと同じです。読まずに許可するくらいなら、安全なコマンドだけを最初から `allow` に登録して（2-2）、確認の回数そのものを減らしましょう。

NG（読まずに許可）：次の画面で、何も考えずに `Yes` のまま Enter を押してしまう。リモートの履歴が強制的に上書きされ、チームの変更が消えることがあります。

```
Bash command
  git push --force origin main
Do you want to proceed?
❯ Yes
  No
```

OK（読んでから判断）：コマンドを読み、`No` を選んで `Tab` でコメント欄を開き、「push は私が自分でやるので実行しないでください」と書いてから Enter を押す。

```
Bash command
  git push --force origin main
Do you want to proceed?
  Yes
❯ No
```

2-2 の設定で `git push --force` を `deny`（禁止）に入れておけば、この確認画面すら出ずにブロックされます。「危ない操作は設定で止める・それ以外は読んでから許可する」の二段構えが安全です。

0.3 まず触ってみる: 最小コマンド集

やりたいこと	入力するコマンド/操作
Claude Code を起動	プロジェクトフォルダで `claude`
新しい会話を始める	`/clear`
プランモード切替	`Shift+Tab`（読み取り専用で計画だけ立てる）
設定画面を開く	`/config`
ヘルプ	`/help`
終了	`/exit` または `Ctrl+D` を2回

`/help` や `/clear` のように「スラッシュ `/` で始まるコマンド」を **スラッシュコマンド** と呼びます。チャット欄にメッセージを送るのではなく、Claude Code 自体に対する操作命令を表します。スラッシュコマンドは Tab キーで補完できます。

▼ 慣れてきたら覚えたい操作（2026年10月時点）

やりたいこと	入力するコマンド/操作
作業をその場で止める	`Esc`
直前の状態に巻き戻す	`Esc` を2回（入力欄が空のとき）または `/rewind`
プロジェクト用の CLAUDE.md の下書きを作らせる	`/init`
コンテキスト（AI の作業机）の使用量を見る	`/context`
許可ルール（allow / deny）を確認・編集する	`/permissions`
使うモデルを変える	`/model`
計画だけ立ててもらう（1回だけプランモード）	`/plan 計画してほしい内容`
使用量・料金の目安を見る	`/usage`（`/cost` も同じ）

入力欄で使う3つの記号: `/` ・ `@` ・ `!`

入力欄の先頭や途中に記号を書くと、ふつうの文章とは違う意味になります。

記号	意味	例
`/`	スラッシュコマンド（Claude Code への操作命令）やスキルを呼ぶ	`/clear`
`@`	ファイルを指定して、その中身を読ませる	`@components/BookCard.tsx を読んで説明して`
`!`	行頭に書くと、AI を通さずにそのままシェルコマンドを実行する	`!git status`

入力例: `@` と `!` を使って質問する

> **▼ この入力例がやること（先に日本語で）**：1行目は `!` を使い、AI を通さずに自分で `git status` を実行して状態を確認します。2行目は `@` でファイルを指定して、その部品の役割を説明してもらいます。どちらも「何を見てほしいか」をはっきりさせる工夫です。

```
!git status
```

```
@components/StatusBadge.tsx の statusConfig が何をしているか、初心者向けに3行で説明して
```

▼ 実行結果（1行目の `!git status` の出力例。変更が無いとき）

```
On branch main
nothing to commit, working tree clean
```

▼ コードを1つずつ分解して解説

解説1: `!` はシェルの直通窓口

```
!git status
```

- `!` で始めた行は Claude に送られず、その場でコマンドとして実行されます。結果は会話にも残るので、続けて「この状態で作業を始めて」と頼めます。
- 自分で状態を確認したいだけのときに、ターミナルを切り替えずに済みます。

解説2: `@` は「このファイルを見て」の指さし

```
@components/StatusBadge.tsx の statusConfig が…
```

- `@` に続けてファイルのパスを書くと、そのファイルの中身が会話に読み込まれます。`@` を入力するとファイル名の候補が出るので、Tab で補完できます。
- 「どのファイルの話か」を AI に探させずに済むので、速く、正確になります。

0.4 安全に使うための基本ルール

1. **重要な変更は Git コミット直後に依頼する** — いつでも `git reset` で戻せる状態にしておく。AIに任せる前に必ず `git status` でクリーンな状態かを確認しましょう。
2. **生成コードを必ず読む** — 「動いたからOK」ではなく「理解したからOK」を目標に。読まずにマージしてしまうと、自分のリポジトリなのに何が起きているか分からなくなります。
3. **`rm -rf` や `git push --force` などの破壊的コマンドはユーザー確認を要求する設定にする** — `.claude/settings.json` で制御可能（後の章で説明）。
4. **API キーや秘密情報を AI に直接見せない** — `.env.local` や `git-ignore` を活用。AIに `.env` の中身を「読んで」と頼んでも答えてくれますが、その内容が会話履歴に残るため漏えいリスクが上がります。
5. **`git diff` や `pwd` で自分の状態を確認する** — 「いま自分はどのブランチにいて」「どのファイルが変更中で」「どこのディレクトリにいるか」を把握しないまま AI に指示を出すと、思わぬところに変更が入ってしまいます。
6. **AIに自動でコミット・プッシュさせない** — コミット内容は人間が `git diff` で確認してから push しましょう。AI に直接 `git push` を実行させるのは原則禁止です。

0.4.1 Git を命綱にする: ブランチ → 差分を読む → 小さくコミット → だめなら戻す

上のルール1・5・6を、実際の手順にまとめたのが次の図です。登山にたとえると、Git のコミットは「岩に打ち込む杭」です。杭をこまめに打っておけば、足を滑らせても（AI が変な変更をしても）、直前の杭まで落ちるだけで済みます。

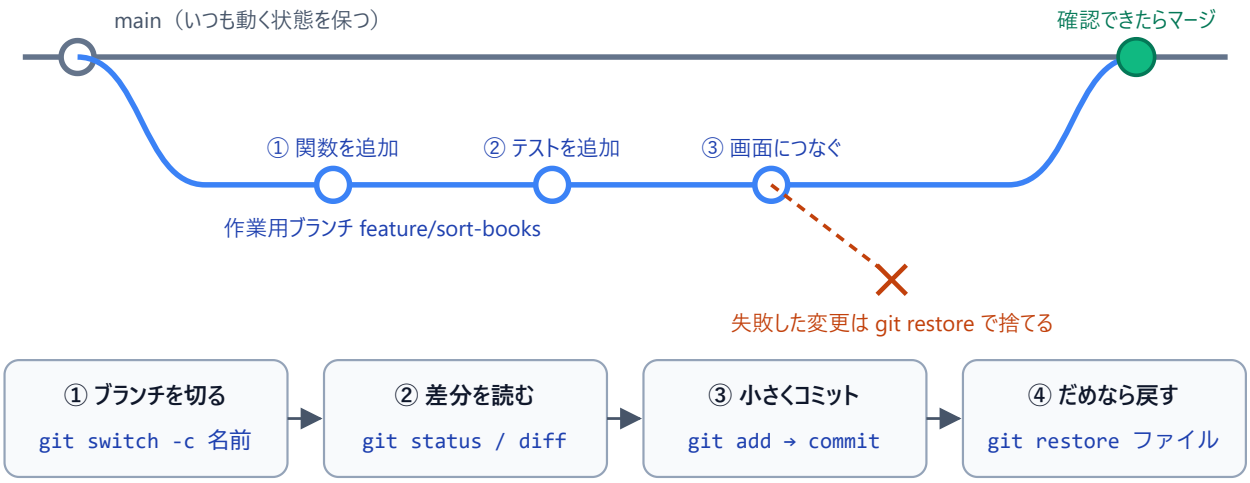


図11-6: Git を命綱にした進め方。main は触らず、作業用ブランチで小さな杭（コミット）を打ちながら進む

- **ブランチを切る理由**: `main` ブランチを「いつでも動く状態」に保つためです。AI の作業はすべて作業用ブランチで行い、確認が済んでから `main` に取り込みます（マージ）。

- **差分を読む理由**: AI は頼んでいないファイルまで触ることがあります。`git status` で「どのファイルが変わったか」、`git diff` で「どう変わったか」を毎回確かめます。
- **小さくコミットする理由**: 1つの機能（または1つの小さな変更）ごとにコミットしておくと、失敗したときに「その1つだけ」を戻せます。

Git で安全に進める一連のコマンド

> **▼ このコードがやること（先に日本語で）**：書籍アプリに並べ替え関数を追加する作業を、図11-6 の①〜④の順で進めるコマンドです。作業用ブランチを作り、Claude Code が作業した後に「何が変わったか」を確かめ、頼んでいない変更を元に戻し、頼んだ変更だけをコミットします。

```
# 作業前: 未コミットの変更が無いことを確かめる（何も表示されなければOK）
git status --short
# ① 作業用ブランチを作って、そこへ移動する
git switch -c feature/sort-books

# （ここで Claude Code に「sortBooks 関数とテストを作って」と頼む）

# ② どのファイルが変わったかを一覧で見る
git status --short
# ② 既存ファイルの変更量と中身を見る
git diff --stat
git diff
# ④ 頼んでいない変更（BookCard.tsx）を元に戻す
git restore components/BookCard.tsx
# 戻ったことを確かめる
git status --short
# ③ 頼んだ2ファイルだけをステージしてコミットする
git add lib/sortBooks.ts lib/sortBooks.test.ts
git commit -m "feat: 書籍の並べ替え関数 sortBooks を追加"
# 履歴を確かめる
git log --oneline
```

▼ 実行結果

```
$ git status --short
$ git switch -c feature/sort-books
Switched to a new branch 'feature/sort-books'
$ git status --short
 M components/BookCard.tsx
?? lib/sortBooks.test.ts
?? lib/sortBooks.ts
$ git diff --stat
 components/BookCard.tsx | 1 +
 1 file changed, 1 insertion(+)
$ git diff
diff --git a/components/BookCard.tsx b/components/BookCard.tsx
index 784951d..80ea6ae 100644
--- a/components/BookCard.tsx
+++ b/components/BookCard.tsx
@@ -1,3 +1,4 @@
 export default function BookCard() {
+  // TODO: 並べ替えボタンを置く
   return null;
 }
$ git restore components/BookCard.tsx
$ git status --short
?? lib/sortBooks.test.ts
?? lib/sortBooks.ts
$ git add lib/sortBooks.ts lib/sortBooks.test.ts
$ git commit -m "feat: 書籍の並べ替え関数 sortBooks を追加"
[feature/sort-books f9d7434] feat: 書籍の並べ替え関数 sortBooks を追加
 2 files changed, 63 insertions(+)
 create mode 100644 lib/sortBooks.test.ts
 create mode 100644 lib/sortBooks.ts
$ git log --oneline
f9d7434 feat: 書籍の並べ替え関数 sortBooks を追加
53d9d84 feat: 書籍カードを作成
```

（説明のため、BookCard.tsx を最小限の中身にした練習用リポジトリで実行した結果です。コミットの ID（`f9d7434` など）は実行するたびに変わります。Windows では `LF will be replaced by CRLF` という改行コードの警告が出ることがありますが、エラーではありません。）

▼ コードを1つずつ分解して解説

解説1: `git status --short` の記号を読む

```
 M components/BookCard.tsx
?? lib/sortBooks.test.ts
```

- `M` は Modified（既存ファイルが変更された）、`??` は Untracked（Git がまだ知らない新しいファイル）という意味です。
- 頼んだのは「sortBooks の新規作成」だけなのに `M components/BookCard.tsx` が出ています。これが「頼んでいない変更」を見つける一番かんたんな方法です。

解説2: `git diff` は新しいファイルを表示しない

```
$ git diff --stat
 components/BookCard.tsx | 1 +
```

- `git diff` が表示するのは、Git がすでに知っているファイルの変更だけです。新しく作られた `sortBooks.ts` は表示されません。
- そのため、まず `git status` で全体を見てから、`git diff` で中身を見る、という順番が大切です。新しいファイルの中身は、エディタで開くか `git add` した後に `git diff --staged` で確認できます。

解説3: `git restore` で1ファイルだけ元に戻す

```
git restore components/BookCard.tsx
```

- 指定したファイルを、最後のコミットの状態に戻します。ほかのファイル（新しく作った sortBooks.ts など）には影響しません。
- **注意:** 戻した変更は取り消せません。戻す前に `git diff` で「本当に要らない変更か」を確かめましょう。

解説4: `git add` はファイルを指定する

```
git add lib/sortBooks.ts lib/sortBooks.test.ts
git commit -m "feat: 書籍の並べ替え関数 sortBooks を追加"
```

- `git add .`（全部まとめて）ではなく、コミットしたいファイルを名指しすると、紛れ込んだ変更をコミットしてしまう事故を防げます。
- コミットメッセージの `feat:` は「機能追加」を表す接頭辞です（5-4 の CLAUDE.md の Git ルールも参照）。

> **つまずきポイント⑤: AI の変更を溜めこんで、まとめて1つの巨大コミットにする**
>
> 何時間も AI に作業させてから `git add .` で全部まとめてコミットすると、後で「この不具合はどの変更のせい？」と調べるのが非常に大変になります。戻したいときも、良い変更まで一緒に戻すしかなくなります。1つの指示が終わって確認できたら、その場でコミットしましょう。

NG（溜めこんで全部まとめる）：

```
# 一日の作業を最後にまとめてコミットしてしまう
git add .
git commit -m "いろいろ修正"
```

OK（1つ確認できたら1コミット）：

```
# 並べ替え関数とテストが通ったので、その2ファイルだけコミット
git add lib/sortBooks.ts lib/sortBooks.test.ts
git commit -m "feat: 書籍の並べ替え関数 sortBooks を追加"
# 次に画面へつなぐ変更を確認できたら、それだけをコミット
git add components/BookList.tsx
git commit -m "feat: 書籍一覧に並べ替えボタンを追加"
```

0.4.2 巻き戻しの3段階: Esc・/rewind・Git

「あ、まずい」と思ったときに使える道具は3つあります。軽いものから順に並べると次のとおりです。

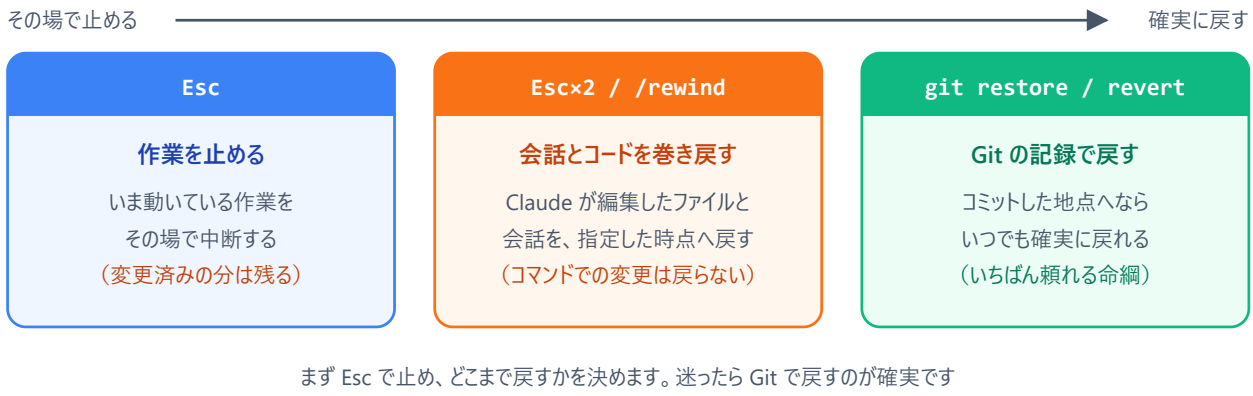


図11-7: 巻き戻しの3段階。右へ行くほど「確実に・大きく」戻せます

- `Esc`: Claude Code の作業をすぐに止めます。止めた時点までに書き換えられたファイルはそのまま残ります。
- `Esc` を2回（または `/rewind`）: Claude Code は、あなたが指示を送るたびに「チェックポイント（その時点の控え）」を自動で取っています。巻き戻しメニューで戻したい時点を選び、「コードと会話を戻す（Restore code and conversation）」「会話だけ戻す（Restore conversation）」「コードだけ戻す（Restore code）」などを選べます。
- **Git**: コミットした地点へは、いつでも確実に戻れます。未コミットの変更を捨てるなら `git restore ファイル名`、コミット済みの変更を打ち消すなら `git revert コミットID` です。

> **注意**: チェックポイント（`/rewind`）は Git の代わりにはなりません。`rm` や `mv` などの **Bash コマンドで行われた変更**や、Claude Code の外（エディタなど）で行った変更は記録されないため、巻き戻しても元に戻りません。公式ドキュメントでも「Git の代わりではない」と明記されています。大事な地点では必ずコミットしておきましょう。

目次

1. Claude Code の全体像と主要機能
2. Phase 0：環境構築と安全設定
3. Phase 1：プロジェクト初期化
4. Phase 2：要件定義（Plan Mode活用）
5. Phase 3：CLAUDE.md の作成と配置戦略
6. Phase 4：Rules の構成
7. Phase 5：Skills の作成と活用
8. Phase 6：Subagents（カスタムエージェント）の構成
9. Phase 7：開発フェーズ — 機能実装の進め方
10. Phase 8：テスト・品質管理
11. Phase 9：デプロイと最終調整
12. Context管理とMemoryシステム
13. キーボードショートカットのカスタマイズ
14. Worktrees（並列開発）
15. CLIフラグとオプション一覧
16. トラブルシューティング
17. ベストプラクティス総まとめ
18. 付録：設定ファイルパス早見表

章末: 付録：開発開始から完了までのチェックリスト　／　練習問題（解答・解説つき）（8問）

---

1. Claude Code の全体像と主要機能

1-1. Claude Code とは

Claude CodeはAnthropicが提供する公式CLIツールで、ターミナル上でClaudeと対話しながらソフトウェア開発を行えます。CLI、デスクトップアプリ（Mac/Windows。Linux はベータ版）、Webアプリ（claude.ai/code）、IDE拡張（VS Code、JetBrains）などで利用可能です（2026年10月時点）。

「CLI」とは Command Line Interface の略で、ターミナル（黒い画面）から文字入力で操作するタイプのアプリのことです。GUI（マウス操作のアプリ）と比べてキーボード中心で素早く操作でき、スクリプトとも連携しやすいのが特徴です。

1-2. 設定ファイルの階層構造

Claude Codeは4層のスコープで設定を管理します。**最も具体的なスコープが優先**されます。「スコープ」とは設定が効く範囲のこと。「自分のマシン全体」よりも「いまのプロジェクトだけ」のほうがより具体的、という意味です。

優先度: **高** ←――――――→ **低**（より具体的なスコープが優先）

Managed（企業管理者向け）

場所: `C:\Program Files\ClaudeCode\settings.json`

対象: マシン全ユーザー、IT 管理者が制御

User（個人設定）

場所: `~/.claude/settings.json`

対象: 自分の全プロジェクト共通

Project（プロジェクト設定）

場所: `.claude/settings.json`

対象: このリポジトリの全メンバー（Git 管理可能）

Local（ローカル設定）

場所: `.claude/settings.local.json`

対象: 自分のこのプロジェクトのみ（gitignore 推奨）

`~` (チルダ) はホームディレクトリの略記号で、Windows では `C:\Users\<ユーザー名>` を指します。`.claude` のように先頭にドットが付くフォルダは「隠しフォルダ」で、エクスプローラの設定によっては見えません。

1-3. 主要機能一覧

機能	説明	設定場所
Settings	権限・モデル・挙動の制御	`settings.json`
CLAUDE.md	プロジェクト固有の指示書	プロジェクトルート等
Rules	パス別の自動適用ルール	`.claude/rules/*.md`
Skills	再利用可能なプロンプトテンプレート	`.claude/skills/*/SKILL.md`
Subagents	カスタムAIエージェント定義	`.claude/agents/*.md`
Hooks	イベント駆動の自動化	`settings.json` 内
MCP	外部サービス連携	`.mcp.json`
Permissions	コマンド実行の許可/拒否	`settings.json` 内
Memory	会話を跨ぐ記憶	`~/.claude/projects/*/memory/`
Keybindings	キーボードショートカット	`~/.claude/keybindings.json`
Worktrees	Git worktreeで並列開発	CLI引数
Plan Mode	読み取り専用の設計モード	`Shift+Tab` で切り替え

それぞれの詳細は後続の Phase で順番に登場します。「いま全部理解しなくてOK」、表は索引代わりに使ってください。

1-4. どの機能を使えばいい？（使い分けの早見図）

機能がたくさんあって迷ったら、「何をしたいか」から逆引きしてください。

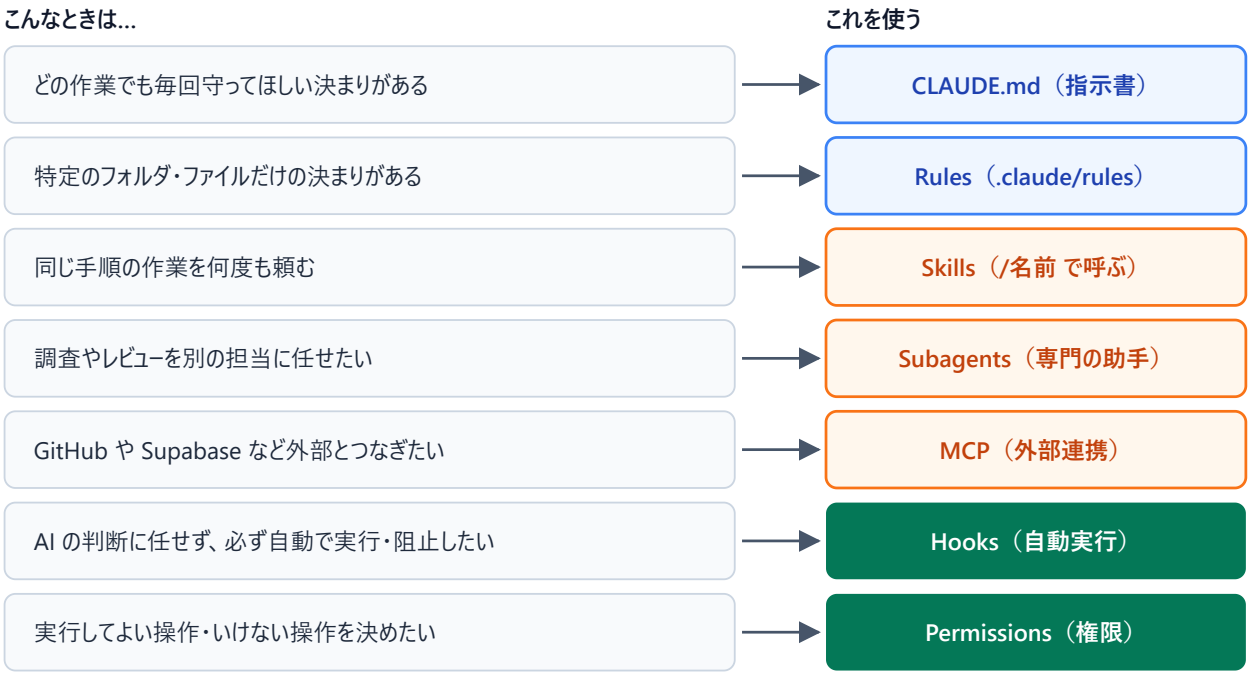


図11-8: 機能の使い分け早見図。青は「AI が読むお願い」、橙は「AI が使う道具」、緑は「AI の判断に関係なく必ず効く仕組み」

いちばん大事な区別は、**「お願い」か「強制」か**です。

- **CLAUDE.md・Rules は「お願い」**: Claude はこれを読んで従おうとしますが、公式ドキュメントにも「強制される設定ではなく、文脈（コンテキスト）として扱われる」と書かれています。うっかり守られないこともあります。
- **Permissions・Hooks は「強制」**: AI がどう判断しようと、設定どおりに止めたり実行したりします。「`.env` を絶対に読ませない」「編集のたびに必ず整形する」のように、例外があっては困ることはこちらに書きます。

> **つまずきポイント⑥: 「絶対にしてほしくないこと」を CLAUDE.md にしか書かない**
>
> CLAUDE.md に「.env は読まないこと」と書いても、それはお願いです。長い作業で文脈が薄れたときなどに守られない可能性がゼロではありません。秘密情報や破壊的な操作は、Permissions の `deny` で止めましょう。

NG（お願いだけ）:

```
## 禁止事項
- .env.local は絶対に読まないこと
```

OK（設定で強制する。`.claude/settings.json`）:

```
{
  "permissions": {
    "deny": ["Read(.env.local)", "Edit(.env.local)"]
  }
}
```

2. Phase 0：環境構築と安全設定

2-1. Claude Code のインストール

```
# 【おすすめ】公式インストーラーを使う方法（Node.js は不要・自動で更新される）
# Windows の PowerShell で実行する場合
irm https://claude.ai/install.ps1 | iex
# macOS / Linux / WSL のターミナルで実行する場合
curl -fsSL https://claude.ai/install.sh | bash

# 【別の方法】npm でインストールする（Node.js 22 以上が必要）
# Claude Code 本体を npm のグローバル領域にインストールする
# -g は global の意味で、どのフォルダからでも claude コマンドが使えるようになる
# @anthropic-ai/claude-code は npm 上のパッケージ名（スコープ名@パッケージ名）
npm install -g @anthropic-ai/claude-code

# インストール確認: バージョンが出れば成功
# --version は「現在のバージョン情報だけ出して終了」を意味する一般的なオプション
claude --version

# ログイン（初回のみ）
# 引数なしで claude を実行すると対話モードが起動する
# 起動時に Claude のアカウント（または Anthropic の API キー）での認証を求められる
claude
```

インストール方法は、公式インストーラー（上の2行のどちらか）がおすすめです。Node.js が無くても動き、新しいバージョンが出ると自動で更新されます。Windows では `winget install Anthropic.ClaudeCode` でも入れられます。

`Node.js` はJavaScriptをパソコン上で動かすための実行環境です。npm でインストールする場合はバージョン 22 以上が必要で、なければ https://nodejs.org からインストールしてください（Next.js の開発自体にも Node.js は必要です）。

> **ヒント:** Windows では、Git for Windows（第1章でインストールしたもの）が入っていると、Claude Code が `Bash` ツールでコマンドを実行できます。入っていなくても `PowerShell` ツールで動きますが、本章のコマンド例は Bash 形式なので、入れておくと読み替えが不要です。

2-2. 危険コマンドの禁止設定（Permissions）

これが最も重要な初期設定です。必ず開発を始める前に設定してください。AIエージェントは賢いとはいえ、勘違いやハルシネーションで「`rm -rf .`」のような危険コマンドを提案することがあり得ます。事前に「これは絶対に実行しない」と Claude Code 側でブロックしておくのが安全策です。

Claude Codeへのプロンプト：

```
# /update-config は設定ファイルを編集するための組み込みスキル
# このスキルを呼び出すと Claude が settings.json の編集をサポートしてくれる
/update-config

Claude Codeの ~/.claude/settings.json を設定してください。
以下のbashコマンドをdenyリストに追加してください：

1. ファイル削除系:
   - rm -rf
   - rm -r
   - del /s（Windows）
   - rmdir /s（Windows）

2. Git破壊系:
   - git push --force
   - git reset --hard
   - git clean -fd
   - git checkout .（全ファイル復元）
   - git branch -D（強制ブランチ削除）

3. データベース破壊系:
   - DROP TABLE
   - DROP DATABASE
   - TRUNCATE

4. システム系:
   - sudo
   - chmod 777

また以下のallowリストも設定してください：
- Read（全ファイル読み取り）
- Bash(npm run *)
- Bash(npx *)
- Bash(git status*)
- Bash(git add *)
- Bash(git commit *)
- Bash(git log*)
- Bash(git diff*)
- Bash(git branch*)
```

手動で設定する場合の完全な `~/.claude/settings.json` ：

```
{
  "permissions": {
    "defaultMode": "default",
    "allow": [
      "Read",
      "Glob",
      "Grep",
      "Bash(npm run *)",
      "Bash(npm install *)",
      "Bash(npm uninstall *)",
      "Bash(npx *)",
      "Bash(node *)",
      "Bash(git status*)",
      "Bash(git add *)",
      "Bash(git commit *)",
      "Bash(git log*)",
      "Bash(git diff*)",
      "Bash(git branch*)",
      "Bash(git checkout -b *)",
      "Bash(git switch *)",
      "Bash(git stash*)",
      "Bash(git merge *)",
      "Bash(git pull*)",
      "Bash(git fetch*)",
      "Bash(ls *)",
      "Bash(cat *)",
      "Bash(mkdir *)",
      "Bash(cp *)",
      "Bash(mv *)",
      "Bash(echo *)",
      "Bash(cd *)",
      "Bash(pwd)",
      "Bash(which *)",
      "Bash(curl *)"
    ],
    "deny": [
      "Bash(rm -rf *)",
      "Bash(rm -r *)",
      "Bash(rm *)",
      "Bash(del /s *)",
      "Bash(rmdir /s *)",
      "Bash(git push --force*)",
      "Bash(git push -f *)",
      "Bash(git reset --hard*)",
      "Bash(git clean -fd*)",
      "Bash(git clean -f*)",
      "Bash(git checkout -- .)",
      "Bash(git branch -D *)",
      "Bash(*DROP TABLE*)",
```

```
      "Bash(*DROP DATABASE*)",
      "Bash(*TRUNCATE*)",
      "Bash(sudo *)",
      "Bash(chmod 777 *)",
      "Bash(shutdown*)",
      "Bash(reboot*)",
      "Edit(.git/**)",
      "Edit(.env)",
      "Edit(.env.local)",
      "Read(.env)",
      "Read(.env.local)"
    ]
  }
}
```

各キーの意味は次のとおりです。

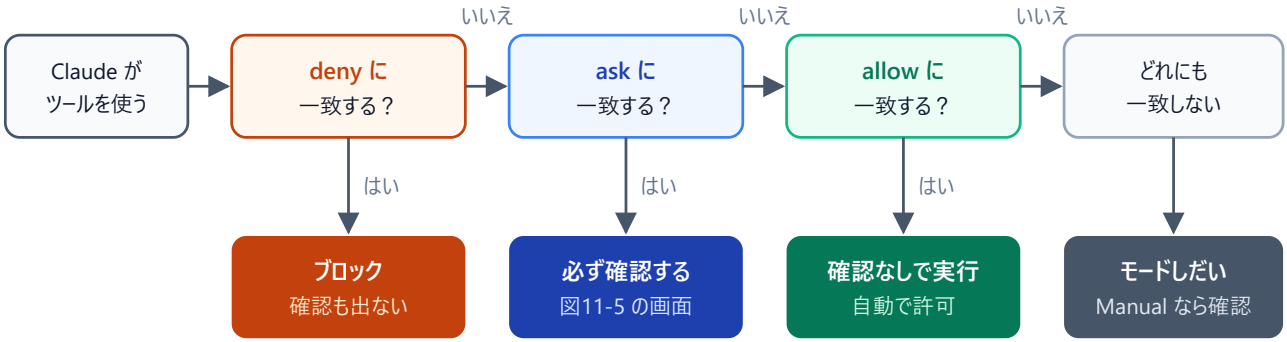
- `permissions`: 権限制御のルートキー。許可・拒否設定はすべてこの下にぶら下がる。
- `defaultMode`: 普段使うパーミッションモード。`default` は「毎回確認」モード。
- `allow`: 自動的に許可するツール・コマンドのリスト。マッチしたら確認なしで実行される。
- `deny`: 絶対に実行させないツール・コマンドのリスト。マッチしたら確認すら出ずブロック。
- `Bash(...)`: Bash ツールの引数パターン。`*` はワイルドカードで任意の文字列にマッチ。
- `Edit(.env)`: Edit ツール（Write などファイルを書き換える組み込みツールも含む）で `.env` を編集しようとするのをブロック。先頭に `/` を付けないファイル名だけの書き方は、作業フォルダ配下のどの階層の `.env` にもマッチします。
- `Read(.env)`: `.env` の中身を Claude が読むのをブロック。0.4 のルール4「秘密情報を AI に直接見せない」を設定で強制できます。

> **注意（以前の版からの修正）**: この教材の以前の版では `Edit(/.env)` のように先頭に `/` を付けていました。`/` で始まるパスは「その設定ファイルの場所を基準にした相対パス」という意味で、`~/.claude/settings.json`（ユーザー設定）に書くと `~/.claude/.env` を指してしまい、プロジェクトの `.env` を守れません。ユーザー設定ですべてのプロジェクトに効かせたいときは、上のようにファイル名だけで書きます（`//` で始めるとディスクのルートからの絶対パス、`~/` で始めるとホームフォルダからのパスになります）。

> **Permission ルールの優先順位**: Deny > Ask > Allow（最初にマッチしたルールが適用）。つまり「`deny` に書いた条件は、`allow` に同じものがあっても deny が勝つ」。
>
> **ワイルドカード `*`**: パターンマッチで広範囲にブロック可能。`Bash(rm -rf *)` は `rm -rf /` も `rm -rf .` もブロックします。

権限ルールはどの順番で判定される？

Claude Code がツールを使おうとするたびに、次の図の順番でルールが調べられます。お店の入口の警備員が「出入り禁止リスト → 要確認リスト → 顔パスリスト」の順に名簿を見るイメージです。



deny → ask → allow の順に調べ、最初に一致したもので決まります。deny は allow より必ず優先されます

図11-9: 権限ルールの判定順。どのスコープ（ユーザー・プロジェクトなど）に書いた deny でも、allow より先に効きます

- 「どれにも一致しない」場合の動きはモードで変わります（2-3）。たとえば Manual（`default`）モードでは確認画面が出ます。
- `git status` や `ls` のような**読み取り専用のコマンド**は、ルールに書かなくても最初から確認なしで実行されます。
- 許可ルールは、Claude Code の中で `/permissions` と入力すると一覧で確認・追加・削除できます。

書籍アプリ用の小さな `.claude/settings.json`

上の `~/.claude/settings.json` は「自分のパソコン全体」に効く大きな設定でした。ここでは、書籍管理アプリのリポジトリに置いて**チームで共有する**、最小限のプロジェクト設定を作ってみます。

> **▼ このコードがやること（先に日本語で）**：書籍アプリでよく使う「確かめるためのコマンド」（lint・ビルド・型チェック・テスト）は確認なしで実行させ、コミットと新しいパッケージの追加は必ず確認させ、秘密情報（ `.env` ）の読み書きと `git push` ・ `rm -rf` は禁止します。

```
{
  "$schema": "https://json.schemastore.org/claude-code-settings.json",
  "permissions": {
    "allow": [
      "Bash(npm run lint)",
      "Bash(npm run build)",
      "Bash(npx tsc --noEmit)",
      "Bash(npx vitest run *)"
    ],
    "ask": [
      "Bash(git commit *)",
      "Bash(npm install *)"
    ],
    "deny": [
      "Read(.env)",
      "Read(.env.local)",
      "Edit(.env)",
      "Edit(.env.local)",
      "Bash(git push *)",
      "Bash(rm -rf *)"
    ]
  }
}
```

▼ 確かめ方（この設定を保存して Claude Code を起動し、`/permissions` を開くと、ルールが次の3つに分かれて並びます。表示を簡略化したイメージです）

```
Allow:  npm run lint / npm run build / npx tsc --noEmit / npx vitest run *
Ask:    git commit * / npm install *
Deny:   Read(.env) / Read(.env.local) / Edit(.env) / Edit(.env.local) / git push * / rm
-rf *
（それぞれ .claude/settings.json から読み込まれたルールとして表示される）
```

▼ コードを1つずつ分解して解説

解説1: `$schema` でエディタに補完してもらう

```
"$schema": "https://json.schemastore.org/claude-code-settings.json",
```

- JSON の「設計図（スキーマ）」の場所を書いておくと、VS Code などのエディタがキー名の補完や入力ミスの指摘をしてくれます。
- Claude Code の動作には影響しません。書いておくと得をする「おまけ」です。

> **用語: JSON:** `{ "キー": 値 }` の形でデータを書く書式。コメント（ `//` ）が書けないので、説明は別の場所に書きます。

解説2: `allow` には「確かめるためのコマンド」だけを入れる

```
"allow": [
  "Bash(npm run lint)",
  "Bash(npx vitest run *)"
]
```

- lint・ビルド・型チェック・テストは、何度実行してもファイルを壊さない「確かめる」ためのコマンドです。確認なしにしておくと、Claude Code が作業ループの⑤（確かめる）を止まらずに回せます。
- `Bash(npx vitest run *)` の末尾の `*`（スペース＋アスタリスク）は「後ろに何が続いてもよい」という意味で、`npx vitest run` 単体にも、`npx vitest run lib` にも一致します。

解説3: `ask` は「結果が残る操作」

```
"ask": [
  "Bash(git commit *)",
  "Bash(npm install *)"
]
```

- コミットやパッケージの追加は、プロジェクトに記録が残る操作です。毎回あなたが内容を見てから許可するように、`ask`（必ず確認）にしています。
- `ask` に書いたコマンドは、`allow` に同じものがあっても毎回確認が出ます。判定の順番が deny → ask → allow だからです（図11-9）。

解説4: `deny` で「絶対にさせないこと」を決める

```
"deny": [
  "Read(.env)",
  "Bash(git push *)"
]
```

- `Read(.env)` で、Supabase のキーなどが入った `.env` を Claude が読めなくなります。
- `git push` は、0.4 のルール6のとおり人間が自分で行う操作なので、禁止しています。

> **つまずきポイント⑦: 確認が面倒で `allow` を広げすぎる**
>
> 確認画面にうんざりして `"Bash"`（すべてのコマンド）や `"Bash(*)"` を `allow` に入れると、`rm -rf` も `git push --force` も確認なしで実行できる状態になります（ `deny` に書いたものだけは止まりますが、思いつかなかった危険なコマンドは素通りです）。`allow` には、何度実行しても安全なコマンドを1つずつ名指しで入れましょう。

NG（何でも許可）：

```
{
  "permissions": {
    "allow": ["Bash"]
  }
}
```

OK（安全なコマンドだけを名指しで許可）：

```
{
  "permissions": {
    "allow": [
      "Bash(npm run lint)",
      "Bash(npx tsc --noEmit)",
      "Bash(npx vitest run *)"
    ]
  }
}
```

2-3. Permission Mode の種類と使い分け

Claude Codeには6つのパーミッションモードがあります。**セッション中に `Shift+Tab` で切り替え可能**です（2026年10月時点の内容です）。

モード	コマンド	説明	使う場面
default（画面の表示名は Manual）	`claude --permission-mode default`	読み取り以外は、各操作ごとに確認を求める	初心者・大事な作業
acceptEdits	`claude --permission-mode acceptEdits`	ファイル編集と `mkdir`・`mv` などの基本操作は自動承認	差分を後でまとめて確認できるとき
plan	`claude --permission-mode plan`	読み取りと調査だけ。ソースの編集は計画を承認するまでできない	設計・調査フェーズ
auto	`claude --permission-mode auto`	別の AI（分類器）が裏で安全性をチェックし、危険な操作だけ止めて自動で進む	長い作業・確認疲れを減らしたいとき
dontAsk	`claude --permission-mode dontAsk`	事前に許可したツールだけ実行し、それ以外は自動で拒否	CI などの無人の自動化
bypassPermissions	`claude --dangerously-skip-permissions`	チェックなし（危険）	**隔離したコンテナや仮想マシン以外では使用禁止**

> **注意（以前の版からの修正）**：以前の版では「`claude` とだけ打つと default（毎回確認）モードで起動する」と説明していました。Claude Code v2.1.283 以降は、ターミナルで `claude` と打つと **auto モードで起動する**のが標準になっています（auto モードが使えない環境では Manual で起動します）。auto モードは便利ですが、公式ドキュメントにも「安全を保証するものではない」と書かれています。慣れるまでは、下の「デフォルトモードを設定する場合」の設定で、毎回 Manual（`default`）モードから始めるのがおすすめです。

`Shift+Tab` を押すたびに、モードは次のように切り替わります。いまのモードは入力欄の下（ステータスバー）に `⏸ manual mode on` や `⏸ plan mode on` のように表示されます。

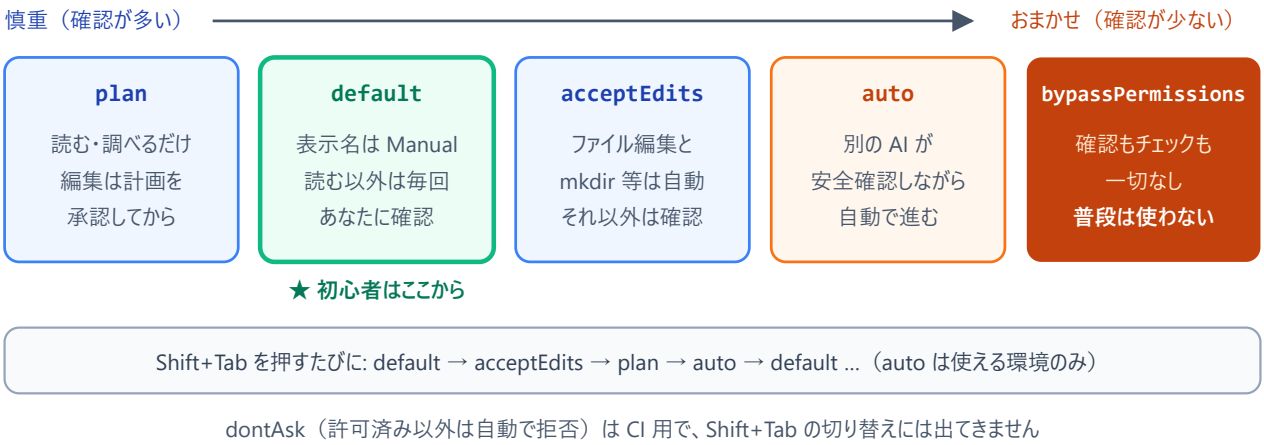


図11-10: パーミッションモードの違い。左ほど慎重、右ほどおまかせ。bypassPermissions は起動時に特別なオプションを付けたときだけ切り替えに加わります

`--permission-mode` は「パーミッションモードを指定して起動する」フラグです。`--` で始まるオプションは「ロングオプション」と呼ばれ、後ろに値を渡せます。

デフォルトモードを設定する場合：

```
{
  "permissions": {
    "defaultMode": "default"
  }
}
```

- `permissions`：設定のグループ名。
- `defaultMode`：何も指定せず `claude` と打ったときに使うモード。`"default"` にすると、毎回 Manual（確認あり）モードで始まります。
- この設定は `~/.claude/settings.json`（ユーザー設定）に書くのが確実です。なお、`"auto"` と `"bypassPermissions"` はプロジェクトの `.claude/settings.json` や `.claude/settings.local.json` に書いても効きません（共有の設定ファイルで勝手に危ないモードにされないための安全策です）。

2-4. MCP サーバーの設定

MCP（Model Context Protocol）により、Claude Codeに外部サービスとの連携能力を追加できます。MCP は「AIに外部ツールを足すための共通プロトコル」で、これを使うと GitHub の Issue を読んだり Supabase のテーブルを操作したり、といった外部連携が可能になります。

MCP設定ファイルの階層

```
スコープ（効く範囲）と保存場所

local    （既定）：~/.claude.json の中の「このプロジェクト用」の欄 ← 自分だけ・このプロジェクトだけ
project          ：プロジェクト直下の .mcp.json                  ← チームで共有（Git管理）
user             ：~/.claude.json                                ← 自分の全プロジェクト共通
```

> **注意（以前の版からの修正）**：以前の版では `.claude/.mcp.json`・`.claude/.mcp.local.json`・`~/.claude/.mcp.json` という場所を紹介していましたが、これらは Claude Code が読み込む場所ではありません。プロジェクトで共有する MCP 設定は**プロジェクト直下（`package.json` と同じ場所）の `.mcp.json`**、個人用は `~/.claude.json` に保存されます（`~/.claude.json` は `claude mcp add` コマンドが自動で書き換えるので、手で編集する必要はほとんどありません）。

コマンドで追加するのがいちばん簡単です。

```
# リモート（HTTP）の MCP サーバーをプロジェクト共有で追加する（.mcp.json に書き込まれる）
claude mcp add --scope project --transport http supabase "https://mcp.supabase.com/mcp"
# 自分のパソコンで動かす（stdio）MCP サーバーを、自分の全プロジェクト用に追加する
claude mcp add --scope user --transport stdio playwright -- npx -y
@playwright/mcp@latest
# 登録済みのサーバーと接続状態を一覧する
claude mcp list
```

- `--scope`：効く範囲。`local`（既定）・`project`・`user` から選びます。
- `--transport`：接続方式。リモートのサーバーは `http`、自分のパソコンで動かすサーバーは `stdio` です。
- `--` より後ろは「MCP サーバーを起動するコマンド」です。Claude Code 自身のオプションと混ざらないよう `--` で区切ります。
- ログイン（OAuth 認証）が必要なサーバーは、Claude Code の中で `/mcp` と入力し、一覧からサーバーを選んで認証します。

Claude Codeへのプロンプト：

```
以下のMCPサーバーを設定してください。

1. GitHub MCP
   - GitHubのIssue/PR操作に使用
   - 個人アクセストークンが必要

2. Supabase MCP
   - データベース操作に使用
   - Supabaseアクセストークンが必要

3. Playwright MCP（オプション）
   - ブラウザテスト自動化に使用

プロジェクトスコープ（プロジェクト直下の .mcp.json）に設定してください。
トークンは環境変数から読み込む形式にしてください。
```

手動設定の場合 — プロジェクト直下の `.mcp.json`：

```
{
  "mcpServers": {
    "github": {
      "type": "http",
      "url": "https://api.githubcopilot.com/mcp/",
      "headers": {
        "Authorization": "Bearer ${GITHUB_TOKEN}"
      }
    },
    "supabase": {
      "type": "http",
      "url": "https://mcp.supabase.com/mcp?read_only=true"
    },
    "playwright": {
      "type": "stdio",
      "command": "npx",
      "args": ["-y", "@playwright/mcp@latest"]
    }
  }
}
```

各キーの意味は以下のとおりです。

- `mcpServers`: MCP サーバー定義をまとめるルートキー。
- `"github"` などのキー: サーバーの識別名。Claude 側で参照するためのラベル。
- `type`: 接続方式。`http`（リモートのサーバー。推奨）/ `stdio`（自分のパソコンで起動するサーバー）から選ぶ。`sse` は古い方式で非推奨。

- `url`：リモートのサーバーの接続先。`?read_only=true` は Supabase の MCP サーバーを「読み取り専用」で使う指定で、AI がデータベースを書き換える事故を防げます。
- `headers`：接続時に送る HTTP ヘッダー。ここでは GitHub の個人アクセストークンを送っています。
- `${GITHUB_TOKEN}`：環境変数 `GITHUB_TOKEN` の値に置き換わる書き方。トークンそのものをファイルに書かずに済むので、`.mcp.json` を Git で共有しても安全です。
- `command`：サーバーを起動するコマンド。`npx` は npm パッケージを直接実行するツール。
- `args`：上記コマンドに渡す引数の配列。`-y` は npx の「確認なしで進める」フラグ。
- Supabase のサーバーはトークンを書く代わりに、Claude Code の中で `/mcp` を開いて「Authenticate」を選び、ブラウザでログインします。

MCP接続タイプの違い

タイプ	形式	用途
stdio	ローカルプロセス	npm パッケージ型のMCPサーバー
http	HTTP接続	リモートAPIサービス（推奨）
sse	Server-Sent Events	古い方式（**非推奨**。使えるなら http を選ぶ）

HTTP型の例（Sentry連携）：

```
{
  "mcpServers": {
    "sentry": {
      "type": "http",
      "url": "https://mcp.sentry.dev/mcp"
    }
  }
}
```

- `type: "http"`：インターネット上の MCP サーバーに HTTP で接続するタイプ。
- `url`：接続先 URL。コマンドなら `claude mcp add --transport http sentry https://mcp.sentry.dev/mcp` と同じ意味です。追加後に `/mcp` でログインします。

OAuth認証付きの例：

多くのサーバーは、上の Sentry の例のように URL を登録して `/mcp` からログインするだけで使えます。サーバー側で事前に登録した OAuth アプリ（クライアント ID）を使う必要がある場合だけ、次のように `oauth` を書きます。

```
{
  "mcpServers": {
    "my-server": {
      "type": "http",
      "url": "https://mcp.example.com/mcp",
      "oauth": {
        "clientId": "your-client-id",
        "callbackPort": 8080
      }
    }
  }
}
```

- `oauth`: OAuth 2.0 認証の設定をまとめるキー。
- `clientId`: 事前に登録した OAuth アプリケーションの ID。
- `callbackPort`: 認証完了後にブラウザから戻ってくる通信を受け取る、自分のパソコンのポート番号。
- クライアントシークレット（秘密鍵）は設定ファイルには書きません。`claude mcp add-json` コマンドに `--client-secret` を付けて登録すると、OS のキーチェーンなど安全な場所に保存されます。

MCP内で使える環境変数

```
{
  "mcpServers": {
    "local-tools": {
      "type": "stdio",
      "command": "node",
      "args": ["${CLAUDE_PROJECT_DIR:-.}/scripts/server.js"],
      "env": {
        "API_KEY": "${MY_API_KEY}",
        "LOG_LEVEL": "${LOG_LEVEL:-info}"
      }
    }
  }
}
```

`${VAR}` の形式は、`.mcp.json` を読み込むときに環境変数の値へ置き換える展開構文です。`command`・`args`・`env`・`url`・`headers` の中で使えます。

書き方	説明
`${VAR}`	環境変数 `VAR` の値に置き換わる
`${VAR:-既定値}`	`VAR` が設定されていればその値、なければ `:-` の後ろの既定値
`${CLAUDE_PROJECT_DIR:-.}`	プロジェクトのルートフォルダ。この変数は MCP サーバーの中でだけ設定されるため、`.mcp.json` で使うときは `:-.`（無ければカレントフォルダ）を付ける

> **注意（以前の版からの修正）**：以前の版で紹介した `allowedEnvVars` というキーと `${CLAUDE_TOOLS_DIR}` という変数は、Claude Code には存在しません。また `auth` ではなく `oauth` が正しいキー名です。

2-5. Hooks の設定（イベント駆動自動化）

Hooksは特定のイベントが発生した時に自動でコマンドを実行する仕組みです。コード品質の自動維持に不可欠です。たとえば「ファイル編集のたびに自動で Prettier をかける」「コミット前に lint を走らせる」など、機械的な作業を肩代わりさせられます。

全Hookイベント一覧

イベント	発火タイミング	主な用途
`SessionStart`	セッション開始/再開時	コンテキスト注入、環境設定
`UserPromptSubmit`	ユーザーがプロンプト送信時	入力バリデーション
`PreToolUse`	ツール実行前	危険操作のブロック、入力の書き換え
`PostToolUse`	ツール実行成功後	コードフォーマット、リント
`PostToolUseFailure`	ツール実行失敗後	エラー処理
`Notification`	通知発火時	デスクトップ通知連携
`Stop`	Claudeの応答完了時	最終チェック
`PreCompact`	コンテキスト圧縮前	重要情報の退避
`PostCompact`	コンテキスト圧縮後	ログ記録などの後処理
`SessionEnd`	セッション終了時	クリーンアップ

> **補足:** 上の表は主なイベントだけです。2026年10月時点では、許可確認の画面が出るときの `PermissionRequest`、サブエージェントが終わったときの `SubagentStop` など、全部で30種類以上のイベントがあります。最新の一覧は公式ドキュメントの Hooks リファレンスで確認してください。

Hook設定の構造

```
{
  "hooks": {
    "イベント名": [
      {
        "matcher": "マッチパターン（ツール名やイベント種別）",
        "hooks": [
          {
            "type": "command",
            "command": "実行するシェルコマンド",
            "timeout": 600
          }
        ]
      }
    ]
  }
}
```

- `hooks`: フック定義をまとめるルートキー。
- `"イベント名"`: 上の表のイベント名（例 `PostToolUse`）。
- `matcher`: そのイベント内でさらに絞り込むためのパターン。たとえば `Edit|Write` で Edit と Write ツールだけにマッチ。
- 内側の `hooks` 配列: 実行する処理のリスト。
- `type: "command"`: シェルコマンドを実行する種別。
- `command`: 実行する具体的なコマンド文字列。
- `timeout`: 何秒で打ち切るか（秒単位）。重い処理を放置しないための保険。

Hookの終了コードの意味

終了コード	意味	動作
`0`	成功	JSON出力があればパース
`2`	ブロッキングエラー	stderrがClaudeへフィードバックされる
その他	非ブロッキングエラー	トランスクリプトに記録のみ

「終了コード」とはコマンドの結果を表す整数で、`0` が成功、それ以外は何らかのエラー、というのが UNIX 系の慣習です。

Claude Codeへのプロンプト：

```
/update-config

Claude Codeのhooksを以下のように設定してください。
settings.jsonに追記してください。

1. PostToolUse: ファイル編集(Edit/Write)後にPrettierで自動フォーマット
2. PostToolUse: ファイル編集後にESLintで自動修正
3. PreToolUse: .envファイルへの書き込みをブロック
4. SessionStart: セッション開始時にリマインダーを表示
   「テストを書いてからコミットすること」
5. Notification: デスクトップ通知（Windows用PowerShell）
6. SessionStart（compact）: コンテキスト圧縮後にCLAUDE.mdの重要ルールを再注入
```

手動設定の完全例（`~/.claude/settings.json` に追記）：

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "FILE=$(jq -r '.tool_input.file_path // empty') && [ -n
\"$FILE\" ] && npx prettier --write \"$FILE\" 2>/dev/null || true",
            "timeout": 30
          }
        ]
      },
      {
        "matcher": "Edit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "FILE=$(jq -r '.tool_input.file_path // empty') && [ -n
\"$FILE\" ] && [[ \"$FILE\" == *.ts || \"$FILE\" == *.tsx ]] && npx eslint --fix
\"$FILE\" 2>/dev/null || true",
            "timeout": 30
          }
        ]
      }
    ],
    "PreToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "INPUT=$(cat) && FILE_PATH=$(echo \"$INPUT\" | jq -r
'.tool_input.file_path // empty') && if [[ \"$FILE_PATH\" == *.env* ]]; then echo
'BLOCKED: .envファイルの編集は禁止されています' >&2; exit 2; fi; exit 0"
          }
        ]
      }
    ],
    "SessionStart": [
      {
        "matcher": "startup",
        "hooks": [
          {
            "type": "command",
            "command": "echo '[リマインダー] テストを書いてからコミットしてください。CLAUDE.md
の規約を遵守してください。'"
          }
```

```
        ]
      },
      {
        "matcher": "compact",
        "hooks": [
          {
            "type": "command",
            "command": "echo '[コンテキスト再注入] 重要: any型禁止、Server Componentをデフォルト使用、コミットは日本語プレフィックス付き'"
          }
        ]
      }
    ],
    "Notification": [
      {
        "matcher": "",
        "hooks": [
          {
            "type": "command",
            "command": "powershell.exe -Command \"[System.Reflection.Assembly]::LoadWithPartialName('System.Windows.Forms'); [System.Windows.Forms.MessageBox]::Show('Claude Codeの確認が必要です', 'Claude Code')\""
          }
        ]
      }
    ]
  }
}
```

長いのでポイントだけ解説します。

- `"matcher": "Edit|Write"`: 正規表現で「Edit または Write」を意味し、その両ツールの実行後に発火。
- `FILE=$(jq -r '.tool_input.file_path // empty')`: Claude Code が**標準入力**に JSON で渡してくる情報から、編集したファイルのパス（`tool_input.file_path`）を取り出す。`jq` は JSON 操作の定番ツール。
- `"matcher": "compact"`（SessionStart）: コンテキストの圧縮（compact）が終わってセッションが続くときに発火。SessionStart の標準出力は Claude のコンテキストに追加されるので、圧縮で薄れた重要ルールを思い出させる用途に使えます。

> **注意（以前の版からの修正）**：以前の版では `$CLAUDE_TOOL_INPUT` という環境変数からツールの入力を読んでいましたが、そのような環境変数はありません。フックへの入力は標準入力に JSON で届きます。また、`PostCompact` フックの標準出力は Claude には届かない（ログに残るだけ）ため、圧縮後のルール再注入は `SessionStart` の `compact` マッチャーで行うように修正しました。- `[ -n "$FILE" ] && ...`：`$FILE` が空でないときだけ次のコマンドを実行（&& は「左が成功したら右を実行」）。- `npx prettier --write "$FILE"`：Prettier で対象ファイルをフォーマット。`--write` は実ファイルへの書き戻し。- `2>/dev/null || true`：エラーを捨てて常に成功扱い。Hook 全体は壊さない設計。- `exit 2`：Claude にブロックを伝える終了コード。`echo ... >&2` で標準エラー出力にメッセージを出す。- `powershell.exe -Command ...`：Windows の PowerShell でメッセージボックスを表示。

高度なHook例：保護ファイルのブロックスクリプト

プロジェクト内にスクリプトを配置して、より複雑なロジックを実行できます。

`.claude/hooks/protect-files.sh`：

```
#!/bin/bash
# 上の行は shebang（シバン）。このスクリプトを bash で実行するよう指示
# 標準入力（Claude が JSON で渡してくる情報）を変数 INPUT に読み込む
INPUT=$(cat)
# jq でツール入力の file_path フィールドを取り出す。値がなければ空文字列
FILE_PATH=$(echo "$INPUT" | jq -r '.tool_input.file_path // empty')

# 保護対象ファイルリスト（配列）
PROTECTED=(
  ".env"
  ".env.local"
  ".env.production"
  "package-lock.json"
  ".git/"
  "next.config.js"
)

# 配列の各パターンに対してループ
for pattern in "${PROTECTED[@]}"; do
  # FILE_PATH に保護対象パターンが含まれていればブロック
  if [[ "$FILE_PATH" == *"$pattern"* ]]; then
    # 標準エラー出力にメッセージを書き、終了コード 2 でブロック
    echo "BLOCKED: $FILE_PATH は保護されたファイルです。手動で編集してください。" >&2
    exit 2
  fi
done

# 何にもマッチしなければ正常終了
exit 0
```

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "bash \"$CLAUDE_PROJECT_DIR/.claude/hooks/protect-files.sh\""
          }
        ]
      }
    ]
  }
}
```

- `"$CLAUDE_PROJECT_DIR/.claude/hooks/protect-files.sh"`：プロジェクトルート配下のスクリプトを実行。複雑なロジックはスクリプトに切り出すと管理しやすい。

---

3. Phase 1：プロジェクト初期化

3-1. Next.js プロジェクトの作成

Claude Codeへのプロンプト：

```
Next.jsの新規プロジェクトを作成してください。以下の構成でお願いします：

- プロジェクト名: my-app
- TypeScript: 使用する
- App Router: 使用する
- Tailwind CSS v4: 使用する
- ESLint: 使用する
- src/ ディレクトリ: 使用する
- import alias: @/*
- Cache Components: 使用しない（No）
- パッケージマネージャ: npm

create-next-app 実行後、追加で以下もインストール:
- shadcn/ui 初期化（New Yorkスタイル）
- zod（バリデーション）
- react-hook-form + @hookform/resolvers
- zustand（状態管理）
- date-fns（日付処理）
- lucide-react（アイコン）

インストールが完了したら、動作確認のため npm run dev を実行して
ビルドエラーがないことを確認してください。

まだ機能のコードは書かないでください。
```

「まだ機能のコードは書かないでください」と書くことで、AIが暴走して未承認のページや機能を追加するのを防いでいます。

> **注意:** 2026年10月時点の最新安定版 Next.js 16 の `create-next-app` では、推奨設定をそのまま使わず「カスタマイズ」を選び、**Cache Components は No** にしてください（第6章と同じ方針です）。Yes にすると、データベースから毎回データを読むページが `npm run build` で失敗することがあり、初心者には原因が分かりにくいためです。AI に作業させる場合も、このように**選択肢の答えまで指示に書いておく**と、AI が勝手に既定値を選ぶのを防げます。

3-2. Git リポジトリの初期化

Claude Codeへのプロンプト：

```
gitリポジトリを初期化して、以下を設定してください：

1. .gitignore に以下を追加
   - .env*（.env.exampleは除外）
   - .claude/settings.local.json
   - CLAUDE.local.md
   - node_modules/
   - .next/
   - *.log

2. .env.example を作成（値は空で、変数名のみ）
   - NEXT_PUBLIC_SUPABASE_URL=
   - NEXT_PUBLIC_SUPABASE_ANON_KEY=
   - SUPABASE_SERVICE_ROLE_KEY=

3. 初回コミット：「feat: Next.jsプロジェクト初期構築」

.envファイルが含まれていないことを確認してからコミットしてください。
```

`.gitignore` は「Git の追跡対象から除外するファイル」を指定するファイルです。`.env`（秘密情報）や `node_modules`（巨大）は必ず除外しないと事故ります。

3-3. Claude Code 用ディレクトリ構造の作成

Claude Codeへのプロンプト：

```
プロジェクト内に以下のClaude Code用ディレクトリ構造を作成してください。
まだ中身のファイルは作らなくて結構です。

.claude/
├── settings.json          # プロジェクト共有設定
├── settings.local.json    # 個人設定（gitignore）
├── rules/                 # パス別自動適用ルール
│   ├── components.md
│   ├── api-routes.md
│   ├── database.md
│   └── testing.md
├── skills/                # カスタムSkills
│   ├── review/
│   │   └── SKILL.md
│   ├── create-component/
│   │   └── SKILL.md
│   └── create-api/
│       └── SKILL.md
├── agents/                # カスタムSubagents
│   ├── code-reviewer.md
│   └── test-writer.md
└── hooks/                 # Hookスクリプト
    └── protect-files.sh
```

ここで作るディレクトリは後の Phase で1つずつ埋めていきます。先に「箱」だけ作っておくと、後の設定作業が見通しよく進みます。

> **注意（以前の版からの修正）**：以前の版では、この図の `.claude/` の中に `.mcp.json` と `.mcp.local.json` を置いていました。MCP の共有設定 `.mcp.json` は `.claude/` の中ではなく**プロジェクト直下**に置き、個人用の MCP 設定は `~/.claude.json` に保存されます（2-4 参照）。

---

4. Phase 2：要件定義（Plan Mode活用）

4-1. Plan Mode とは

Plan Modeは**ソースコードを書き換えない**モードで、Claudeはコードを読んだり調査用のコマンドを実行したりして計画を立てますが、ファイルの作成・編集は、あなたが計画を承認するまでできません。**設計フェーズに最適**です。

要件が定まる前に AI に勝手にコードを書かれてしまうと、後戻りが大変です。「最初は計画だけ立てる」というのを Plan Mode で強制できます。

Plan Mode の起動方法

```
# 起動時にPlan Modeで開始
# --permission-mode plan で読み取り専用モードに固定
claude --permission-mode plan

# セッション中に切り替え
# Shift+Tab を何回か押し、入力欄の下に「⏸ plan mode on」と出たら Plan Mode
# 1回の指示だけ計画させたいときは、指示の先頭に /plan を付けて送る
```

Plan Mode の活用フロー

```
Plan Mode（設計） ⟶ 方針合意 ⟶ Default Mode（実装）
      │                              │
      │ ・要件整理                    │ ・コード生成
      │ ・画面設計                    │ ・ファイル作成
      │ ・DB設計                      │ ・テスト実行
      │ ・API設計                     │
      │ ・実装方針の提案               │
      ↓                              ↓
  承認オプション（計画ができると表示される）：
  ├─ Yes, and use auto mode（承認して auto モードで実装）
  ├─ Yes, manually approve edits（承認して、編集を1つずつ確認しながら実装）
  └─ No, keep planning（承認せず、計画の修正を続ける）
```

実践: 書籍アプリの「並べ替え機能」をプランモードで計画する

> **▼ この指示文がやること（先に日本語で）**：書籍一覧に並べ替え機能を足す前に、`/plan` を付けて「計画だけ」を出してもらいます。どのファイルを新しく作り、どのファイルを変えるのかを先に見て、あなたが納得してから実装に進むための指示です。

```
/plan 書籍一覧ページ（app/page.tsx）に並べ替え機能を追加したい。
並べ方は「タイトル順」「評価の高い順」「新しい順」の3つ。
並べ替えの処理は lib/sortBooks.ts に関数として切り出し、Vitest でテストする。
URL のクエリ（?sort=title など）で並べ方を選べるようにしたい。
まだコードは書かず、変更するファイルと手順を計画してください。
```

▼ 実行結果（Claude Code が出した計画の例）

```
## 計画: 書籍一覧の並べ替え

### 変更するファイル
1. lib/sortBooks.ts（新規）: sortBooks(books, key) を作る
2. lib/sortBooks.test.ts（新規）: 3種類の並べ方と「元の配列を変えない」ことをテスト
3. components/SortSelect.tsx（新規）: 並べ方を選ぶセレクトボックス（Client Component）
4. app/page.tsx（変更）: URL の sort を読み、sortBooks で並べてから BookList に渡す

### 手順
1. sortBooks とテストを作り、npx vitest run で確認する
2. SortSelect を作り、選んだ値を URL の ?sort= に反映する
3. page.tsx を変更し、npm run build で確認する

### 確認したいこと
- ?sort=abc のような想定外の値のときは「新しい順」にしてよいですか？
```

計画ができると、ターミナルには次のような選択肢が表示されます。



図11-11: 画面イメージ（プランモードの承認画面）。文言はバージョンや設定によって少し変わります

▼ コードを1つずつ分解して解説

解説1: `/plan` を先頭に付けると、その指示だけプランモードになる

```
/plan 書籍一覧ページ（app/page.tsx）に並べ替え機能を追加したい。
```

- `Shift+Tab` でモードを切り替えなくても、指示の先頭に `/plan` を付けるだけで「計画だけ立てて」と伝えられます。
- 計画を立てる間、Claude Code はコードを読んだり調べたりしますが、ソースファイルは書き換えません。

解説2: 計画の「変更するファイル」を真っ先に読む

```
### 変更するファイル
1. lib/sortBooks.ts（新規）: sortBooks(books, key) を作る
```

- 計画の中で一番大事なのは「どのファイルを新しく作り、どのファイルを変えるか」です。頼んでいないファイル（例: 認証まわり）が混ざっていたら、ここで止めます。
- 「新規」と「変更」の区別も確認しましょう。既存ファイルの変更は、壊してしまう可能性があるぶん注意が必要です。

解説3: 「確認したいこと」には必ず答える

```
### 確認したいこと
- ?sort=abc のような想定外の値のときは「新しい順」にしてよいですか？
```

- AI が迷った点を質問してくれています。答えずに承認すると、AI が推測で決めた仕様のまま実装されます。
- 「No, keep planning」を選んで「はい、新しい順にしてください」と答えると、計画が更新されます。

解説4: 承認の選択肢の選び方

選択肢	その後の動き	おすすめの場面
`Yes, and use auto mode`	承認し、auto モードで確認なしに実装を進める	計画が小さく、内容に完全に納得しているとき
`Yes, manually approve edits`	承認し、ファイル編集のたびに確認しながら実装する	**初心者はこれ。**差分を見ながら進められる
`No, keep planning`	承認せず、プランモードのまま計画を直す	計画に直してほしい点があるとき

- auto モードが使えない環境では、1つ目の選択肢が `Yes, auto-accept edits`（ファイル編集を自動で承認）と表示されます。
- `Ctrl+G` を押すと、計画をテキストエディタで開いて直接書き直せます。

> **つまずきポイント⑧: 計画を読まずに「Yes」を押す**
>
> プランモードの価値は「実装の前に、あなたが計画をチェックできる」ことです。計画を読まずに承認すると、プランモードを使わなかったのと同じになります。特に「変更するファイル」と「確認したいこと」は必ず読みましょう。

NG（読まずに承認する）：計画が表示されたら、すぐに `Yes, and use auto mode` を選ぶ。頼んでいない `SortSelect.tsx` の作成まで確認なしで進み、質問への答えも AI の推測で決まってしまう。

OK（読んで、直してから承認する）：`No, keep planning` を選び、次のように伝えてから、もう一度計画を確認する。

```
計画を次のように直してください。
- 今回は手順1（sortBooks とテスト）だけにして、SortSelect と page.tsx の変更は次回にします
- 想定外の ?sort= の値は「新しい順」で OK です
```

4-2. アプリ全体像の伝え方

Plan Modeを起動してから以下を入力：

```
以下のWebアプリケーションを開発します。
まず要件を整理してドキュメントとして出力してください。

## アプリ概要
タスク管理Webアプリ「TaskFlow」

## ターゲットユーザー
- 小規模チーム（5-10人）のプロジェクトマネージャー
- 日常的にタスクの割り振りと進捗管理を行う

## 主要機能（優先度順）
### P0（必須）
1. ユーザー認証（メール/パスワード、Google OAuth）
2. プロジェクト作成・管理
3. タスクのCRUD（作成・表示・更新・削除）
4. タスクのステータス管理（未着手/進行中/レビュー/完了）
5. カンバンボード表示

### P1（重要）
6. チームメンバーへのタスク割り当て
7. 期限設定とリマインダー通知
8. ダッシュボード（進捗の可視化）
9. タスクのフィルタリング・検索

### P2（あれば嬉しい）
10. タスクへのコメント機能
11. ファイル添付
12. アクティビティログ

## 技術スタック
- フロントエンド: Next.js 16 (App Router), TypeScript, Tailwind CSS v4, shadcn/ui
- バックエンド: Next.js Route Handlers
- データベース: Supabase (PostgreSQL)
- 認証: Supabase Auth
- 状態管理: Zustand
- バリデーション: Zod
- デプロイ: Vercel

## 非機能要件
- レスポンシブ対応（モバイルファースト）
- ダークモード対応
- ページ読み込み3秒以内（LCP）
- アクセシビリティ（WCAG 2.1 AA準拠）
- 日本語UI

## 出力してほしいもの
1. 画面一覧（画面名、URL、概要）
2. 全テーブルのER図（テキスト形式）
3. 全テーブルのカラム定義
```

```
4. API一覧（メソッド、パス、リクエスト/レスポンス）
5. 実装の推奨順序

コードは絶対に書かないでください。ドキュメントのみ出力してください。
```

> **なぜPlan Modeを使うのか？**
>
> - 計画を承認するまでソースを書き換えないため、設計に集中できる
> - 誤ってファイルが生成されるリスクがない
> - 要件の漏れを防げる

4-3. 画面設計の詳細化

Plan Mode で継続：

```
先ほどの画面一覧を元に、各画面の詳細を設計してください。

各画面について以下を定義：
1. 画面名とURL（ルーティングパス）
2. ページの種類（Server Component / Client Component）
3. 画面レイアウト（ヘッダー、サイドバー、メインエリアの配置）
4. 表示する要素の一覧（UI要素単位で）
5. ユーザーが行える操作（ボタン、フォーム、ドラッグ等）
6. 各操作に対応するAPI呼び出し
7. ローディング状態の表示方法
8. エラー状態の表示方法
9. 空状態（データなし）の表示方法
10. レスポンシブ時の変更点（モバイル/タブレット/デスクトップ）

対象画面：
- ログイン画面（/login）
- 新規登録画面（/register）
- ダッシュボード（/dashboard）
- プロジェクト一覧（/projects）
- プロジェクト詳細（/projects/[id]）
- カンバンボード（/projects/[id]/board）
- タスク詳細（モーダル or /projects/[id]/tasks/[taskId]）
- メンバー管理（/projects/[id]/members）
- 設定画面（/settings）
- プロフィール画面（/settings/profile）
```

4-4. データベース設計

Plan Mode で継続：

```
データベースのテーブル設計を詳細に定義してください。

各テーブルについて以下を含めてください：
1. テーブル名（スネークケース、複数形）
2. 全カラム定義
   - カラム名
   - データ型（PostgreSQL）
   - NOT NULL制約
   - デフォルト値
   - UNIQUE制約
   - CHECK制約
3. プライマリキー（UUID使用）
4. 外部キー（リレーション）
5. インデックス（頻繁にWHEREで使うカラム）
6. RLS（Row Level Security）ポリシー
   - SELECT: 誰が読めるか
   - INSERT: 誰が作成できるか
   - UPDATE: 誰が更新できるか
   - DELETE: 誰が削除できるか
7. トリガー（updated_at の自動更新など）

共通ルール：
- 全テーブルに id (UUID), created_at, updated_at を含める
- ソフトデリート（deleted_at）を採用
- タイムスタンプは全て timestamptz（タイムゾーン付き）

テーブル一覧：
- profiles（ユーザープロフィール）
- projects（プロジェクト）
- project_members（プロジェクトメンバー、多対多の中間テーブル）
- tasks（タスク）
- task_comments（コメント）
- task_labels（ラベル、多対多の中間テーブル）
- labels（ラベルマスタ）

Supabaseのマイグレーション SQL形式で出力してください。
```

4-5. 設計ドキュメントの保存

Plan Mode からDefault Modeに切り替えてから：

```
（Shift+Tab でDefault Modeに切り替え）

先ほどの設計内容をドキュメントとして保存してください。
以下のファイルに分けて作成：

1. docs/requirements.md - 要件定義書
2. docs/screens.md - 画面設計書
3. docs/database.md - データベース設計書
4. docs/api.md - API設計書
5. supabase/migrations/001_initial_schema.sql - 初回マイグレーション

これらのドキュメントは今後の開発中に参照するため、
具体的かつ正確に記述してください。
```

ここで作るドキュメントは、後の Phase で `@docs/requirements.md` のように `@` 参照で読み込ませて、AI に常に同じ前提を共有させるための「正典」になります。

---

5. Phase 3：CLAUDE.md の作成と配置戦略

5-1. CLAUDE.md とは

CLAUDE.mdはClaude Codeへの「永続的な指示書」です。セッション開始時に自動的に読み込まれ、Claudeの振る舞いを制御します。

人間で言うなら「新人さんへの引き継ぎマニュアル」に近いです。プロジェクトの命名規約・禁止事項・コミットルールなどを書いておくと、Claude が毎回それを踏まえて作業してくれます。

5-2. 配置場所と優先順位

```
優先度（全て結合される。上書きではなく追加）：

C:\Program Files\ClaudeCode\CLAUDE.md   ← 企業ポリシー（IT管理者）
~/.claude/CLAUDE.md                      ← 全プロジェクト共通の個人ルール
./CLAUDE.md                              ← プロジェクトルート（Git共有）
./.claude/CLAUDE.md                      ← .claude内（Git共有）
./CLAUDE.local.md                        ← 個人用上書き（gitignore）
./src/CLAUDE.md                          ← サブディレクトリ（そのDir以下で有効）
```

> **重要**: これらは全て **結合（concatenate）** されます。上位が下位を上書きするのではなく、全てが読み込まれます。

`AGENTS.md`（ほかの AI コーディングツールと共通で使われる指示書の名前）しか無いリポジトリでは、Claude Code は `AGENTS.md` を `CLAUDE.md` の代わりに読みます。ただし、作業フォルダかその上のフォルダに `CLAUDE.md`（または `.claude/CLAUDE.md`・`CLAUDE.local.md`）があるときは、既定では **`CLAUDE.md` 側だけが読まれ、`AGENTS.md` は読まれません**。両方を使いたい場合は、`CLAUDE.md` の中に `@AGENTS.md` と1行書いて取り込みます。

> **注意（以前の版からの修正）**：以前の版では「両方ある場合は両方読まれる」と説明していましたが、2026年10月時点の既定の動きは上のとおりです（`/config` の設定で変更できます）。

5-3. 作成タイミング

要件定義完了後、最初のコードを書く直前に作成します。要件が固まる前に書くと「とりあえず書いただけのルール」になりがち。設計を踏まえた具体的なルールを書きましょう。

5-4. CLAUDE.md 作成のプロンプト

Claude Codeへのプロンプト：

```
プロジェクトルートにCLAUDE.mdを作成してください。
先ほど合意した要件定義・設計ドキュメントの内容を反映し、
以下の全セクションを含めてください。

---

# プロジェクト概要
- アプリ名、目的、ターゲットユーザーの1-2行の要約

# 技術スタック
- 使用する全技術とバージョン（Next.js 16, TypeScript, etc.）

# ディレクトリ構成
src/
├── app/                    # App Router ページ
│   ├── (auth)/             # 認証グループ（layout共有）
│   │   ├── login/
│   │   └── register/
│   ├── (main)/             # メインアプリグループ
│   │   ├── dashboard/
│   │   ├── projects/
│   │   └── settings/
│   └── api/                # Route Handlers
├── components/
│   ├── ui/                 # shadcn/ui コンポーネント
│   ├── features/           # 機能別コンポーネント
│   │   ├── auth/
│   │   ├── project/
│   │   ├── task/
│   │   └── dashboard/
│   └── layouts/            # レイアウトコンポーネント
├── lib/
│   ├── supabase/           # Supabaseクライアント
│   ├── validations/        # Zodスキーマ
│   └── utils/              # ユーティリティ関数
├── hooks/                  # カスタムHooks
├── stores/                 # Zustandストア
└── types/                  # 型定義

# コーディング規約
## コンポーネント
- 関数コンポーネントのみ使用（classコンポーネント禁止）
- named export を基本（default export は page.tsx/layout.tsx のみ）
- Server Component をデフォルト、'use client' は必要な場合のみ
- Props型は ComponentName + Props で命名（例: TaskCardProps）
- 1ファイル1コンポーネント
- コンポーネントファイルは PascalCase（例: TaskCard.tsx）

## 型定義
```

```
- interface を優先（type は共用体型の場合のみ）
- any 型は絶対に使わない（unknown を使う）
- 型アサーション（as）は最小限に

## インポート
- パスエイリアス @/ を必ず使用（相対パス禁止）
- インポート順: React → 外部ライブラリ → 内部モジュール → 型
- 未使用インポートは残さない

## スタイリング
- Tailwind CSS ユーティリティクラスのみ（カスタムCSS禁止）
- インラインスタイル禁止
- cn() ヘルパーでクラスを結合

## 状態管理
- サーバーの状態: Server Component + Supabase直接クエリ
- クライアントの状態: Zustand
- フォームの状態: react-hook-form
- URL状態: searchParams

## エラーハンドリング
- error.tsx で画面単位のエラーをキャッチ
- not-found.tsx で404をハンドル
- loading.tsx でローディング状態を表示
- API Route ではtry-catch + 適切なHTTPステータスコードを返す
- ユーザー向けエラーはトースト（sonner）で表示

# 命名規約
| 種別 | 規則 | 例 |
|------|------|-----|
| コンポーネント | PascalCase | TaskCard.tsx |
| ユーティリティ | camelCase | formatDate.ts |
| 定数 | UPPER_SNAKE_CASE | MAX_FILE_SIZE |
| 型/Interface | PascalCase | TaskStatus |
| Hooks | camelCase + use prefix | useTaskList.ts |
| Store | camelCase + Store suffix | taskStore.ts |
| API Route | route.ts（Next.js規約） | app/api/tasks/route.ts |

# Git ルール
- コミットメッセージは日本語で記述
- プレフィックス: feat:, fix:, refactor:, docs:, style:, test:, chore:
- 例: 「feat: タスク作成機能を追加」
- 1機能1コミットを原則
- .env ファイルは絶対にコミットしない

# 開発コマンド
- 開発サーバー: npm run dev
- ビルド: npm run build
- Lint: npm run lint
- 型チェック: npx tsc --noEmit
```

```
- テスト: npm test
- E2Eテスト: npm run test:e2e

# やってはいけないこと（絶対厳守）
- rm -rf や destructive なコマンドを実行しない
- .env ファイルを編集・作成しない
- any 型を使わない
- console.log をプロダクションコードに残さない
- インラインスタイルを使わない
- node_modules/ を Git にコミットしない
- 1ファイルが300行を超えないようにする（超える場合は分割）
- テストなしでコミットしない

# 設計ドキュメント参照
@docs/requirements.md
@docs/screens.md
@docs/database.md
@docs/api.md
```

最後の `@docs/requirements.md` のような行は、CLAUDE.md 内の **ファイル参照記法** です。`@` で始めるとそのファイルの中身がコンテキストに自動で取り込まれ、Claude がドキュメントを踏まえて作業できるようになります。

5-4-1. 書籍アプリ用の小さな CLAUDE.md（実例）

上のテンプレートは大きなチーム開発向けの「全部入り」です。第1〜10章の書籍管理アプリくらいの規模なら、もっと短い CLAUDE.md で十分です。公式ドキュメントでも「1つの CLAUDE.md は200行以内を目安に。長いほどコンテキストを消費し、指示が守られにくくなる」とされています。

> **ヒント:** 何もない状態から書くのが大変なら、Claude Code の中で `/init` と入力してください。プロジェクトのファイルを調べて CLAUDE.md の下書きを作ってくれます。下書きを読み、間違いを直し、不要な行を削ってから使いましょう。

書籍管理アプリの CLAUDE.md

> **▼ このコードがやること（先に日本語で）：** 書籍管理アプリのリポジトリの一番上に置く「AI への引き継ぎメモ」です。何のアプリか、よく使うコマンド、フォルダの役割、守ってほしい決まり、作業の進め方を、30行に収めています。Claude Code は起動するたびにこれを読んでから作業を始めます。

```
# 書籍管理アプリ（my-books-app）

読んだ本・読みたい本を記録する個人用の Web アプリ。

## 技術スタック
- Next.js（App Router）+ TypeScript + Tailwind CSS
- データベースと認証: Supabase（books テーブル）

## よく使うコマンド
- 開発サーバー: `npm run dev`
- Lint: `npm run lint`
- 型チェック: `npx tsc --noEmit`
- テスト: `npx vitest run`

## フォルダの役割（`src/` フォルダは使わない）
- `app/`: ページ（フォルダ名がそのまま URL になる）
- `components/`: 画面の部品（BookCard、StatusBadge など）
- `lib/`: Supabase クライアントと共通の関数
- `types/`: 型定義（Book 型など）

## コードの決まり
- 書籍のステータスは `reading` / `completed` / `want_to_read` の3つだけ
- Supabase からのデータ取得は Server Component で行う
- `any` 型は使わない（分からないときは `unknown` にして絞り込む）

## 作業の進め方
- 変更したら `npm run lint` と `npx tsc --noEmit` を実行し、結果を報告する
- 頼まれた範囲のファイルだけを変更する。ほかに直したい所があれば提案だけする
- `.env.local` は読まない・書かない
- `git push` はしない（人間が行う）
```

▼ 実行結果（行数を数えて「200行以内」の目安を確かめる）

```
$ wc -l CLAUDE.md
30 CLAUDE.md
```

`wc -l` はファイルの行数を数えるコマンドです（Windows では Git Bash で使えます）。30行なので、目安の200行に十分収まっています。

▼ コードを1つずつ分解して解説

解説1: 冒頭の2行で「何のアプリか」を伝える

```
# 書籍管理アプリ（my-books-app）

読んだ本・読みたい本を記録する個人用の Web アプリ。
```

- 新人さんに最初に伝えるのと同じで、目的が分かっていると AI の判断（画面の文言、機能の優先度など）がぶれにくくなります。

解説2: 「よく使うコマンド」は確かめ方の辞書

```
## よく使うコマンド
- Lint: `npm run lint`
- 型チェック: `npx tsc --noEmit`
```

- ここに書いておくと、指示のたびに「確かめ方」を書かなくても、Claude Code がこのコマンドで検証してくれるようになります（0.1.2 の解説4）。
- 2-2 の `allow` に同じコマンドを入れておくと、確認画面も出ずに作業ループが回ります。

解説3: 決まりは「守れたかどうか確かめられる形」で書く

```
- 書籍のステータスは `reading` / `completed` / `want_to_read` の3つだけ
- `any` 型は使わない（分からないときは `unknown` にして絞り込む）
```

- 「きれいに書く」のような感想ではなく、「守ったか・守っていないか」がはっきり判定できる書き方にしています。
- 禁止するだけでなく「代わりにどうするか」（`unknown` にする）も書くと、AI が迷いません。

解説4: 「作業の進め方」で、人間とAIの役割分担を決める

```
- 頼まれた範囲のファイルだけを変更する。ほかに直したい所があれば提案だけする
- `git push` はしない（人間が行う）
```

- 0.4 の「安全に使うための基本ルール」を、AI 自身に毎回読ませる形にしたものです。
- ただし CLAUDE.md は「お願い」であり、絶対に守られる保証はありません。`.env` や `git push` のように**絶対に**させたくないことは、2-2 の `deny` やフックでも止めておきます（1-4 の使い分けの図も参照）。

CLAUDE.md は、置き場所によって「いつ読まれるか」が違います。

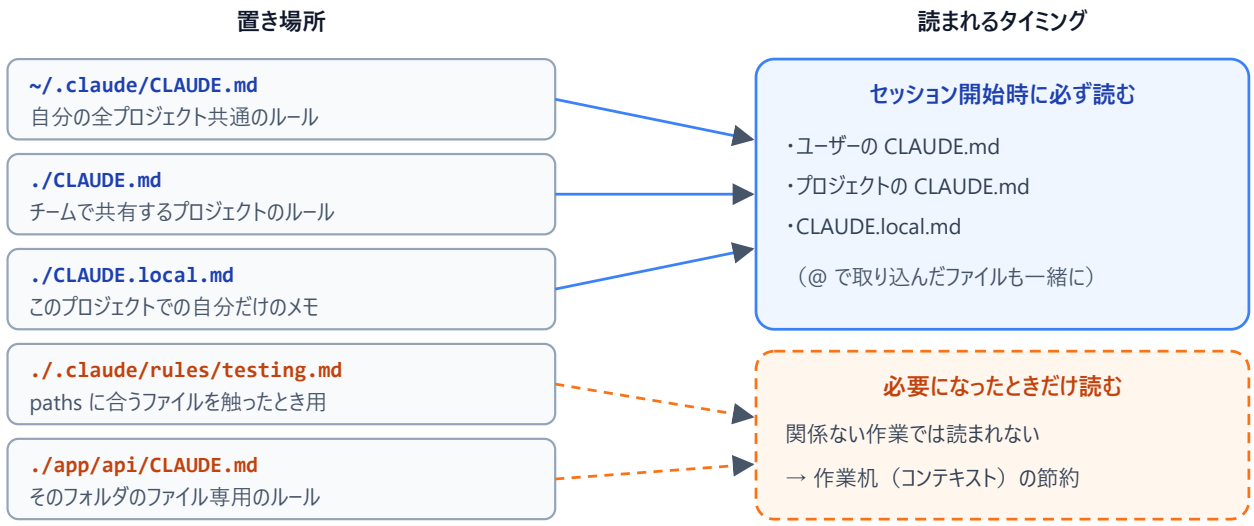


図11-12: CLAUDE.md とルールファイルが読まれるタイミング。実線はいつも、点線は必要なときだけ

> **つまずきポイント⑨: CLAUDE.md に「感想」や「何でも」を書きすぎる**
>
> 「きれいなコードを書くこと」「ベストプラクティスに従うこと」は、人によって意味が違うので、AI にとっては何も書いていないのとほぼ同じです。逆に、思いついたことを全部書いて数百行になると、大事な決まりが埋もれて守られにくくなります。**確かめられる形で、短く**書きましょう。

NG（あいまい・書きすぎ）：

```
## ルール
- きれいで読みやすいコードを書くこと
- ベストプラクティスに従うこと
- バグを出さないように気をつけること
- （以下、思いついた注意が300行続く…）
```

OK（具体的・短い）：

```
## コードの決まり
- 部品（コンポーネント）は components/ に1ファイル1つ置く
- any 型は使わない（分からないときは unknown にして絞り込む）
- 変更後は npm run lint と npx tsc --noEmit を実行して結果を報告する
```

5-5. CLAUDE.local.md（個人設定）

チームメンバーそれぞれのスタイルに合わせた個人設定をCLAUDE.local.mdに配置できます（gitignoreされるため共有されない）。

Claude Codeへのプロンプト：

```
CLAUDE.local.md を作成してください。以下を記載：

- 回答は全て日本語で行うこと
- コードの説明は詳細に行うこと（初学者向け）
- 変更を加える前に必ず変更内容を説明し承認を得ること
- エラーが発生したら原因の推測と複数の解決案を提示すること
```

5-6. サブディレクトリ CLAUDE.md

特定のディレクトリ以下でのみ有効なルールを設定できます。サブディレクトリの CLAUDE.md は「そのフォルダ配下のファイルを触るとき」だけ自動で読み込まれます。

`src/app/api/CLAUDE.md` ：

```
# API Route 固有ルール

- 全てのRoute Handlerでzodバリデーションを実施
- 認証チェックを最初に行う
- レスポンス形式を統一: { data: T } | { error: string }
- HTTPステータスコードを正しく使う
  - 200: 成功
  - 201: 作成成功
  - 400: バリデーションエラー
  - 401: 認証エラー
  - 403: 認可エラー
  - 404: リソース未存在
  - 500: サーバーエラー
```

5-7. `@` によるファイル参照

CLAUDE.md内で他のドキュメントを参照できます。

```
# 設計ドキュメント
@docs/requirements.md
@docs/database.md
@README.md
```

- `@<相対パス>`: そのファイルの中身を CLAUDE.md と一緒に読み込ませる。

> **注意**: 参照先のファイルもコンテキストに読み込まれるため、大きすぎるファイルを参照するとコンテキストを圧迫します。1ファイル数千行になるようなものは、要点だけ抜粋した別ドキュメントを作って参照しましょう。

5-8. CLAUDE.md の更新タイミングと方法

タイミング	更新内容	プロンプト例
新パッケージ追加時	技術スタック更新	「CLAUDE.mdの技術スタックに lucide-react を追加して」
ディレクトリ構成変更時	構成図更新	「CLAUDE.mdのディレクトリ構成を現状に合わせて更新して」
新しい規約決定時	規約追加	「CLAUDE.mdに'Server Actionsは使わない'を追加して」
バグの再発防止時	禁止事項追加	「CLAUDE.mdに'useEffectでデータフェッチしない'を追加して」
大きなリファクタ後	全体見直し	「CLAUDE.mdを現在のプロジェクト状態に合わせて全体更新して」

特定項目の更新プロンプト：

```
CLAUDE.mdを更新してください。以下の変更を反映：

1. 技術スタックに追加:
   - @tanstack/react-query v5（サーバー状態管理）
   - sonner（トースト通知）

2. コーディング規約に追加:
   - データフェッチは React Query の useQuery を使用
   - Server Component での直接クエリは初回ロードのみ
   - useEffect でのデータフェッチは禁止

3. ディレクトリ構成に追加:
   - src/lib/queries/ - React Queryのクエリ関数
```

5-9. 特定のCLAUDE.mdを除外する

大きなモノレポで不要なCLAUDE.mdを除外：

```
{
  "claudeMdExcludes": [
    "**/node_modules/**/CLAUDE.md",
    "**/other-project/CLAUDE.md"
  ]
}
```

- `claudeMdExcludes`: 読み込み対象から除外する glob パターンの配列。
- `**/node_modules/**`: あらゆる階層の node_modules を意味するパターン。

6. Phase 4：Rules の構成

6-1. Rules とは

`.claude/rules/` ディレクトリに配置するMarkdownファイルで、**特定のファイルパスにマッチした時に自動的に適用されるルール**です。CLAUDE.mdとは異なり、パスベースで条件分岐できます。

CLAUDE.md が「プロジェクト全体のルール」だとすれば、Rules は「このフォルダだけのルール」。たとえばコンポーネントを作るときと API Route を作るときでは気をつけるポイントが違うので、別々の Rules ファイルに分けます。

6-2. Rules の仕組み

```
---
paths:
  # globパターンでマッチ
  - "src/components/**/*.tsx"
---

# ここにルールを記述
コンポーネント作成時のルール...
```

- `---` で囲まれた部分は YAML フロントマター（メタ情報）。
- `paths`: このルールを発火させるファイルパターン。glob 記法。
- `**` はあらゆる深さのフォルダにマッチ、`*` は任意のファイル名にマッチ。

Claudeが該当パスのファイルを操作する時、自動的にこのルールがコンテキストに追加されます。

6-3. Rules 作成のプロンプト

Claude Codeへのプロンプト：

```
.claude/rules/ ディレクトリに以下の4つのルールファイルを作成してください。
各ファイルにはpathsフロントマターとルール本文を含め、
具体的なコード例も必ず記載してください。

---

## 1. components.md
対象パス: src/components/**/*.tsx

含める内容:
- コンポーネントの雛形（Server Component / Client Component）
- Props型の定義方法
- children の受け取り方
- 条件付きレンダリングのパターン
- forwardRef の使い方
- コンポーネント分割の基準（いつ分割するか）
- shadcn/ui コンポーネントのカスタマイズ方法
- 良い例と悪い例の比較コード

## 2. api-routes.md
対象パス: src/app/api/**/*.ts

含める内容:
- Route Handler の雛形（GET/POST/PATCH/DELETE）
- zodバリデーションの書き方
- 認証チェックのパターン
- エラーレスポンスの統一フォーマット
- Supabaseクエリの書き方
- ページネーションの実装パターン
- 良い例と悪い例の比較コード

## 3. database.md
対象パス: supabase/**/*.sql, src/lib/supabase/**/*.ts

含める内容:
- マイグレーションSQLの書き方
- RLSポリシーのテンプレート
- Supabaseクエリのベストプラクティス
- 型安全なクエリの書き方（generated types使用）
- N+1クエリの回避方法

## 4. testing.md
対象パス: **/*.test.ts, **/*.test.tsx, **/*.spec.ts

含める内容:
- Vitestのテスト構造（describe/it/expect）
- React Testing Libraryでのコンポーネントテスト
- モックの使い方（vi.mock, vi.fn）
- API Routeのテスト方法
```

```
- テストデータのファクトリパターン
- テストの命名規則（日本語OK）
- AAA（Arrange-Act-Assert）パターン
```

6-4. Rules ファイルの具体例

`.claude/rules/components.md` ：

````
---
paths:
  - "src/components/**/*.tsx"
---

# コンポーネント開発ルール

## Server Component（デフォルト）

```tsx
// Good: Server Component
// プロジェクト内の関数を @/ エイリアスで読み込む（相対パス禁止のため）
import { getProjects } from '@/lib/queries/projects'

// Props 型は interface で定義。命名は ComponentName + Props
interface ProjectListProps {
  // 必須の文字列プロパティ
  userId: string
}

// named export。default export は page.tsx/layout.tsx のみ許可
// async が付くと Server Component で await が使える
export function ProjectList({ userId }: ProjectListProps) {
  // サーバー側でデータ取得。クライアントには結果だけ届く
  const projects = await getProjects(userId)

  return (
    <ul>
      {/* map で配列を要素に変換。key は React の差分検出に必須 */}
      {projects.map((project) => (
        <li key={project.id}>{project.name}</li>
      ))}
    </ul>
  )
}
````
````

Client Component（インタラクティブな場合のみ）

```tsx
// Good: Client Component（'use client' を明示）
// ファイル先頭に書くと、このファイル全体がクライアント側でハイドレートされる
'use client'

// useState はクライアントでしか使えないので 'use client' が必要
import { useState } from 'react'
// shadcn/ui の Button コンポーネント
import { Button } from '@/components/ui/button'

interface CounterProps {
  // 初期カウント値
  initialCount: number
}

export function Counter({ initialCount }: CounterProps) {
  // useState: 値とその更新関数のペアを返す
  const [count, setCount] = useState(initialCount)

  return (
    <div>
      <p>Count: {count}</p>
      {/* onClick は React のイベントハンドラ。アロー関数で書く */}
      <Button onClick={() => setCount((c) => c + 1)}>
        Increment
      </Button>
    </div>
  )
}
```

禁止パターン

```
// Bad: any型（型安全が崩れる）
function Component(props: any) { ... }

// Bad: default export（page.tsx以外）
export default function Component() { ... }

// Bad: useEffectでデータフェッチ（Server Component で十分）
useEffect(() => {
  fetch('/api/data').then(...)
}, [])

// Bad: インラインスタイル（Tailwind を使うべき）
<div style={{ color: 'red' }}>...</div>
```

コンポーネント分割の基準

- 100行を超えたら分割を検討
- 200行を超えたら必ず分割
- 再利用される部分は即座に分割
- ロジック（hooks）とUI（component）を分離

`.claude/rules/api-routes.md`：

```markdown
---
paths:
  - "src/app/api/**/*.ts"
---

# API Route 開発ルール

## Route Handler の雛形

```ts
// Next.js のサーバー側型定義をインポート
import { NextRequest, NextResponse } from 'next/server'
// バリデーションライブラリ。スキーマで型と検証を同時に定義
import { z } from 'zod'
// Supabase の Server クライアント（cookie ベースの認証情報を持つ）
import { createClient } from '@/lib/supabase/server'

// 1. バリデーションスキーマ定義
// z.object() で各フィールドの型・制約を宣言的に書く
const createTaskSchema = z.object({
  // 1〜100文字の文字列
  title: z.string().min(1).max(100),
  // 任意、最大1000文字
  description: z.string().max(1000).optional(),
  // 列挙型
  status: z.enum(['todo', 'in_progress', 'review', 'done']).default('todo'),
  priority: z.enum(['low', 'medium', 'high']).default('medium'),
  // UUID 形式の文字列
  assigneeId: z.string().uuid().optional(),
  // ISO8601 形式
  dueDate: z.string().datetime().optional(),
})

// 2. Route Handler
// export async function POST で、このパスへの POST リクエストを処理
export async function POST(request: NextRequest) {
  try {
    // 3. 認証チェック
    // Supabase クライアント作成
    const supabase = await createClient()
    // 現在ログイン中のユーザーを取得
    const { data: { user }, error: authError } = await supabase.auth.getUser()

    // 未ログインなら 401 を返して終了
    if (authError || !user) {
      return NextResponse.json(
```
```

```
        // ユーザー向けメッセージ
        { error: '認証が必要です' },
        // HTTP ステータス
        { status: 401 }
      )
    }

    // 4. リクエストバリデーション
    // JSON ボディをオブジェクトに
    const body = await request.json()
    // safeParse は throw せず { success, data | error } を返す
    const parsed = createTaskSchema.safeParse(body)

    if (!parsed.success) {
      return NextResponse.json(
        { error: 'バリデーションエラー', details: parsed.error.flatten() },
        // クライアント側の入力ミスは 400
        { status: 400 }
      )
    }

    // 5. DB操作
    // from('tables').insert(...).select().single() の流れ
    const { data, error } = await supabase
      // 対象テーブル
      .from('tasks')
      // 検証済みデータ + 作成者
      .insert({ ...parsed.data, created_by: user.id })
      // 挿入結果を取得
      .select()
      // 1件だけ取り出す（複数返ったらエラー）
      .single()

    if (error) {
      return NextResponse.json(
        { error: 'タスクの作成に失敗しました' },
        // サーバー側エラーは 500
        { status: 500 }
      )
    }

    // 6. 成功レスポンス
    // 新規作成成功は 201 Created
    return NextResponse.json({ data }, { status: 201 })

  } catch {
    // 想定外の例外は 500 にまとめる
    return NextResponse.json(
      { error: '予期しないエラーが発生しました' },
      { status: 500 }
```

```
    )
  }
}
```

レスポンス形式（統一）

- 成功: `{ data: T }`
- エラー: `{ error: string, details?: unknown }`
- 一覧: `{ data: T[], count: number, page: number, limit: number }`

```
---

## 7. Phase 5：Skills の作成と活用

### 7-1. Skills とは

Skillsは**再利用可能なプロンプトテンプレート**です。`/skill-name` のスラッシュコマンドとして呼び出せます。繰り返し行う定型作業を自動化できます。

「同じような指示文を毎回書くのは面倒」「チームで指示の表現を揃えたい」というときに、Skill としてまとめておけば `/review` のような短いコマンドで呼び出せます。

### 7-2. Skill の配置場所
```

~/.claude/skills//SKILL.md ← 全プロジェクト共通 .claude/skills//SKILL.md ← プロジェクト固有

````
Skill 1つにつき1フォルダ作り、その中に `SKILL.md` を置きます。同じフォルダにヘルパースクリプトや参考資料を入れることもできます。

### 7-3. SKILL.md のフロントマター

```yaml
---
# 一意な識別子（小文字、ハイフン、最大64文字）
name: skill-id
# 呼び出し条件の判断に使われる（250文字以内）
description: "説明文"
# 引数のヒント（オートコンプリート用）
argument-hint: "[filepath]"
# true=手動呼び出しのみ、false=Claudeが自動呼び出し可
disable-model-invocation: false
# true=ユーザーが/で呼び出せる
user-invocable: true
# 確認なしで使ってよいツール（スペースかカンマ区切り、またはリスト）
allowed-tools: "Read Grep Edit"
# 使用モデル（opus / sonnet / haiku などの別名、またはモデルID）
model: opus
# 推論レベル（low/medium/high/xhigh/max）
effort: high
# forkでサブエージェントとして実行
context: fork
# サブエージェントタイプ
agent: Explore
# 自動発火のglobパターン
paths: ["src/**/*.tsx"]
---
````

各キーの意味は以下のとおりです。

- `name`：スラッシュコマンド名のもと（例 `review` → `/review`）。
- `description`：Claude が「いまこのスキル使うべき？」と判断する材料。
- `argument-hint`：補完時に表示する引数の説明（例 `[filepath]`）。
- `disable-model-invocation`：Claude が自動でこの Skill を呼ぶのを禁止するか。
- `user-invocable`：ユーザーがスラッシュコマンドとして呼べるか。
- `allowed-tools`：この Skill を実行している間、**確認なしで使ってよい**ツール。使えるツールを「制限」するものではなく、許可を「先渡し」するものです（以前の版の「ツールを限定」という説明を修正しました）。
- `model`：使うモデル。`opus`・`sonnet`・`haiku` のような別名で書くと、その時点の最新版が使われます（2026年10月時点では Opus 5.5 など Claude 5 系）。
- `effort`：推論の深さ。high のほうが時間とコストがかかるが品質が上がる。
- `context`：`fork` でサブエージェントとして独立コンテキストで実行。メインの会話を汚さない。
````

- `agent`: ビルトインのサブエージェントタイプ。
- `paths`: 該当パターンのファイルを編集するとき自動的に呼ばれる。

7-4. 変数の使い方

変数	説明
`$ARGUMENTS`	ユーザーが渡した引数全体
`$0`, `$1`, `$2`	位置引数
`${CLAUDE_SESSION_ID}`	セッションID
`${CLAUDE_PROJECT_DIR}`	プロジェクトルート
`${CLAUDE_SKILL_DIR}`	Skillファイルのディレクトリ
`!`command``	コマンド実行結果を注入（Skill読み込み時に実行）

`!`コマンド`` は「Skill が読み込まれた瞬間にそのコマンドを実行し、結果をその場に埋め込む」記法です。例: `!`git diff --name-only`` と書けば、Skill 起動時の変更ファイル一覧がそのままプロンプトに入ります。

7-5. Skills 作成のプロンプト

Claude Codeへのプロンプト：

```
以下の4つのSkillを .claude/skills/ に作成してください。
各SkillにはSKILL.mdを作成し、フロントマターと
詳細なプロンプトテンプレートを含めてください。

---

## Skill 1: /review（コードレビュー）
- 名前: review
- 引数: ファイルパス（省略可、省略時はgit diffの変更ファイル全て）
- 機能: コードをレビューし、セキュリティ・パフォーマンス・規約の
  3観点で問題を検出・修正提案
- ツール: Read, Grep, Glob
- モデル: opus
- context: fork（メインコンテキストを汚さない）

## Skill 2: /create-component（コンポーネント生成）
- 名前: create-component
- 引数: コンポーネント名
- 機能: CLAUDE.mdとRulesに準拠したコンポーネントの雛形を生成
- ツール: Read, Write, Edit
- 生成ファイル:
  - src/components/features/[name]/[Name].tsx
  - src/components/features/[name]/[Name].test.tsx

## Skill 3: /create-api（API Route生成）
- 名前: create-api
- 引数: リソース名（例: tasks）
- 機能: CRUD全操作のRoute Handlerを雛形生成
- ツール: Read, Write, Edit
- 生成ファイル:
  - src/app/api/[name]/route.ts（GET, POST）
  - src/app/api/[name]/[id]/route.ts（GET, PATCH, DELETE）
  - src/lib/validations/[name].ts（zodスキーマ）

## Skill 4: /simplify（コード改善）
- 名前: simplify
- 引数: フォーカス領域（省略可）
- 機能: 変更されたコードの重複排除・リファクタリング・効率改善
- ツール: Read, Grep, Edit
- context: fork
```

7-6. Skill の具体例

`.claude/skills/review/SKILL.md`：

```
---
name: review
description: コードレビューを実施。セキュリティ・パフォーマンス・規約の3観点で問題を検出し修正提案を行う。
argument-hint: "[filepath or empty for git diff]"
user-invocable: true
allowed-tools: Read Grep Glob Bash
model: opus
effort: high
context: fork
---

# コードレビュータスク

## レビュー対象
$ARGUMENTS が指定されている場合はそのファイルをレビュー。
指定がない場合は `git diff --name-only` の出力ファイルを全てレビュー。

## レビュー観点（全て確認すること）

### 1. セキュリティ（Critical）
- [ ] SQLインジェクションの可能性
- [ ] XSS（Cross-Site Scripting）の可能性
- [ ] 認証チェックの漏れ（全API Routeで実施されているか）
- [ ] 認可チェックの漏れ（RLSが適切か）
- [ ] 機密情報のハードコード（APIキー、パスワード等）
- [ ] CSRF対策

### 2. パフォーマンス（High）
- [ ] 不要な再レンダリング（memo, useMemo, useCallback の適切な使用）
- [ ] N+1クエリの有無
- [ ] 大きなバンドルサイズ（dynamic importの検討）
- [ ] 画像の最適化（next/image使用）
- [ ] 不要な'use client'（Server Componentで可能なものはないか）

### 3. コーディング規約（Medium）
- [ ] CLAUDE.mdの規約に準拠しているか
- [ ] 命名規約に従っているか
- [ ] any型が使われていないか
- [ ] console.logが残っていないか
- [ ] 未使用のインポートがないか
- [ ] ファイルサイズが300行以内か

### 4. エラーハンドリング（Medium）
- [ ] try-catchが適切に使われているか
- [ ] ユーザー向けエラーメッセージが適切か
- [ ] ローディング状態が処理されているか
- [ ] 空状態が処理されているか
```

```
### 5. アクセシビリティ（Low）
- [ ] 適切なaria属性
- [ ] キーボード操作可能
- [ ] 画像のalt属性
- [ ] フォームのlabel

## 出力形式
各問題を以下の形式で報告：

**[重要度: Critical/High/Medium/Low]** ファイル名:行番号
- 問題: 何が問題か
- 理由: なぜ問題か
- 修正案: 具体的なコード
```

`.claude/skills/create-component/SKILL.md`：

```
---
name: create-component
description: CLAUDE.mdとRulesに準拠したReactコンポーネントの雛形を生成する。ファイルとテストを同時に作成。
argument-hint: <ComponentName>
user-invocable: true
allowed-tools: Read Write Edit Bash
model: sonnet
effort: medium
---

# コンポーネント生成タスク

## 指示
「$ARGUMENTS」という名前のコンポーネントを生成してください。

## 生成前の確認
1. CLAUDE.md を読んでコーディング規約を確認
2. .claude/rules/components.md を読んでコンポーネントルールを確認
3. 既存の類似コンポーネントがあれば参考にする

## 生成ファイル

### 1. コンポーネント本体
パス: `src/components/features/$ARGUMENTS/$ARGUMENTS.tsx`

- Props interfaceを定義（${ARGUMENTS}Props）
- Server Component をデフォルト（インタラクティブなら'use client'追加）
- named export
- Tailwind CSSでスタイリング
- 適切なaria属性を含める

### 2. テストファイル
パス: `src/components/features/$ARGUMENTS/$ARGUMENTS.test.tsx`

- 基本的なレンダリングテスト
- Props変更時のテスト
- ユーザーインタラクションテスト（該当する場合）

## 生成後
- 生成したファイルの内容を簡潔に説明
- 使い方の例を1つ提示
```

7-6-1. 書籍アプリ用の小さなスキル `/check`（確かめ係）

上の2つは本格的な例でした。最初に作るなら、もっと小さく「毎回同じ確認作業」をスキルにするのがおすすめです。ここでは、変更のあとに lint・型チェック・テストをまとめて実行し、結果を表で報告してくれる `/check` を作ります。

▼ このコードがやること（先に日本語で）： `.claude/skills/check/SKILL.md` に置くスキルです。`/check` と入力すると、いま変わっているファイルの一覧を自動で読み込み、3つの確認コマンドを確認画面なしで実行し、結果を表にまとめます。このスキルはファイルを書き換えず、問題を見つけたら直し方を提案するだけにしています。

```
---
name: check
description: 書籍アプリの変更を lint・型チェック・テストで確かめて、結果を表にまとめる。
disable-model-invocation: true
allowed-tools:
  - Bash(npm run lint)
  - Bash(npx tsc --noEmit)
  - Bash(npx vitest run *)
---

# 変更の確認

## いま変わっているファイル
!`git status --short`

## やること
1. `npm run lint` を実行する
2. `npx tsc --noEmit` を実行する
3. `npx vitest run` を実行する
4. 結果を「チェック / 結果 / メモ」の3列の表にまとめる（失敗したものはエラーの要点も書く）

## 守ること
- このスキルではファイルを編集しない。失敗を見つけたら、直し方の提案だけをする
```

▼ 実行結果（`/check` と入力したときの報告の例。テストの数は 0.4.1 の sortBooks のテストを実行した場合）

```
いま変わっているファイル: lib/sortBooks.ts, lib/sortBooks.test.ts

| チェック          | 結果 | メモ                          |
|-------------------|------|-------------------------------|
| npm run lint      | 成功 | 警告なし                       |
| npx tsc --noEmit  | 成功 | 型エラーなし                   |
| npx vitest run    | 成功 | 1 ファイル・4 テストすべて通過 |
```

▼ コードを1つずつ分解して解説

解説1: `name` がコマンド名になる

```
name: check
description: 書籍アプリの変更を lint・型チェック・テストで確かめて、結果を表にまとめる。
```

- フォルダ名と `name` を `check` にすると、`/check` で呼び出せます。
- `description` は、スキルの一覧に表示される説明です。Claude が「このスキルを使うべきか」を判断する材料にもなります。

解説2: `disable-model-invocation: true` で「呼ばれたときだけ」動かす

```
disable-model-invocation: true
```

- `true` にすると、Claude が自分の判断でこのスキルを使うことはなくなり、あなたが `/check` と入力したときだけ動きます。
- 「いつ確認するか」を人間が決めたい作業に向いています。

解説3: `allowed-tools` で確認画面を省く

```
allowed-tools:
  - Bash(npm run lint)
  - Bash(npx tsc --noEmit)
  - Bash(npx vitest run *)
```

- ここに書いたコマンドは、このスキルを実行している間だけ確認なしで実行されます。
- コマンドにスペースが入るので、1行に並べず、`-` で始まるリスト（YAML のリスト）の形で書いています。

解説4: `!`git status --short`` で、その瞬間の状態を埋め込む

```
## いま変わっているファイル
!`git status --short`
```

- `!`コマンド`` は、スキルが呼ばれた瞬間にコマンドを実行し、その結果をこの場所に貼り込む書き方です（7-4）。
- Claude は「どのファイルが変わったか」を最初から知った状態で作業を始められます。

7-7. ビルトインSkills

Claude Codeには組み込みのSkillもあります。

Skill	用途	使い方
`/code-review`（別名 `/review`）	変更（差分）のバグを探すレビュー	`/code-review`
`/security-review`	変更のセキュリティ上の問題を探す	`/security-review`
`/simplify`	コード改善レビュー（重複や読みにくさの整理）	`/simplify focus on performance`
`/loop`	定期実行	`/loop 5m check if deploy finished`
`/batch`	大きな変更を小さく分けて並列に進める	`/batch 全コンポーネントの props 型を interface に統一`
`/claude-api`	Claude APIリファレンス読み込み	`/claude-api`

> **注意（以前の版からの修正）**：以前の版では `/commit` と `/frontend-design` を組み込みの Skill として紹介していましたが、2026年10月時点の公式ドキュメントの組み込みコマンド一覧にはありません。コミットは「この変更をコミットして」と言葉で頼むか、自分で `.claude/skills/commit/SKILL.md` を作れば `/commit` で呼べるようになります。本章で `/commit` と書いている箇所は、この自作スキル（または同じ内容の依頼）のことだと読み替えてください。
>
> また、7-5 で作る自作の `/review` と `/simplify` は、組み込みの同名コマンドと名前が重なります。混乱を避けたい場合は `/team-review` のように別の名前にしておくと安心です。

---

8. Phase 6：Subagents（カスタムエージェント）の構成

8-1. Subagents とは

Subagentsは**独立したAIエージェント**を定義し、特定のタスクを委任できる仕組みです。メインのコンテキストを汚さずに、専門的なタスクを並列実行できます。

たとえば「コードレビュー専門のエージェント」「テスト作成専門のエージェント」をそれぞれ用意し、必要に応じて使い分けると、メイン会話のコンテキストを節約しつつ専門タスクを任せられます。

8-2. 配置場所

```
~/.claude/agents/<name>.md      ← 全プロジェクト共通
.claude/agents/<name>.md        ← プロジェクト固有
```

8-3. Subagent のフロントマター

```yaml
---
# 一意な識別子
name: agent-id
# いつ委任するかの判断基準
description: "説明文"
# 使用モデル（sonnet / opus / haiku / inherit、またはモデルID）
model: opus
# 使ってよいツール（カンマ区切り）
tools: Read, Bash, Edit, Grep
# 使わせないツール（カンマ区切り）
disallowedTools: WebFetch
# 権限モード（default / acceptEdits / plan など）
permissionMode: default
# プリロードするSkill
skills: [review, create-component]
# 会話をまたいで覚えておく場所（user / project / local）
memory: project
# worktreeで並列実行
isolation: worktree
---
```

- `name`：エージェント識別子。
- `description`：メインのClaude がこのエージェントへ委任するか判断する材料。
- `model`：使用するモデル。`inherit` にするとメインの会話と同じモデルを使います。
- `tools`：このエージェントが使えるツール（カンマ区切り）。省略するとメインと同じツールをすべて使えます。
- `disallowedTools`：使わせないツール。
- `permissionMode`：このエージェントの権限モード（2-3 の表と同じ名前）。
- `skills`：起動時にあらかじめ読み込む Skill。
- `memory`：エージェント専用の記憶の保存場所。`project` なら `.claude/agent-memory/<名前>/` に保存されます。
- `isolation: worktree`：worktree で別作業ツリーを使って並列実行。

> **注意（以前の版からの修正）**：以前の版の `permissions:`（allow / deny）と `memory: enabled:` という書き方は、サブエージェントの設定には存在しません。ツールの制限は `tools` と `disallowedTools`、確認の出方は `permissionMode` で指定します。

8-4. ビルトインSubagent

Agent	用途	context指定
Explore	コードベース調査・検索	`context: fork` + `agent: Explore`
Plan	タスク計画の立案	Plan Mode使用時
general-purpose	汎用（デフォルト）	指定なし

8-5. Subagent 作成のプロンプト

Claude Codeへのプロンプト：

```
.claude/agents/ に以下の2つのカスタムSubagentを作成してください。

---

## Agent 1: code-reviewer.md
- 名前: code-reviewer
- 目的: コードレビューの専門エージェント
- モデル: opus
- 許可ツール: Read, Bash, Grep, Glob（Editは禁止）
- 禁止: WebFetch, Edit（読み取り専用）
- 動作:
  1. 指定されたファイルまたはgit diffを分析
  2. セキュリティ・パフォーマンス・規約の観点でレビュー
  3. 問題を重要度別に分類して報告
  4. 具体的な修正コードを提示（ただし自分では編集しない）

## Agent 2: test-writer.md
- 名前: test-writer
- 目的: テストコード作成の専門エージェント
- モデル: sonnet（コスト効率重視）
- 許可ツール: Read, Write, Edit, Bash, Grep
- 許可Bash: npm test*, npx vitest*
- 動作:
  1. 対象コンポーネント/関数のソースコードを読む
  2. 既存のテストパターンを調査
  3. Vitest + React Testing Library でテストを作成
  4. テストを実行して全パスすることを確認
```

8-6. Subagent の具体例

`.claude/agents/code-reviewer.md`：

```
---
name: code-reviewer
description: コードレビューの専門エージェント。セキュリティ、パフォーマンス、規約準拠を検査し、問題を報告する。自身ではコードを編集しない。
model: opus
tools: Read, Bash, Grep, Glob
disallowedTools: Edit, Write, WebFetch
---

あなたはコードレビューの専門家です。

## あなたの役割
- コードを分析して問題を発見する
- 具体的な修正案を提示する
- ただし、自分ではコードを編集しない（読み取り専用）

## レビュー手順
1. まず CLAUDE.md を読んでプロジェクトの規約を理解する
2. .claude/rules/ のルールファイルを確認する
3. 対象コードを精読する
4. 以下の観点でレビューする

## レビュー観点

### Critical（即座に修正必要）
- セキュリティ脆弱性（XSS, SQLi, 認証漏れ）
- データ損失の可能性
- 本番障害に直結する問題

### High（早急に修正推奨）
- パフォーマンス問題
- エラーハンドリングの欠如
- 型安全性の問題

### Medium（改善推奨）
- コーディング規約違反
- 可読性の問題
- テストの不足

### Low（余裕があれば改善）
- アクセシビリティ
- 命名の改善提案
- コメントの追加

## 出力形式
問題ごとに以下の形式で報告すること：
```

[Critical/High/Medium/Lov] 問題のタイトル

- ファイル: path/to/file.ts:行番号
- 問題: 具体的に何が問題か
- 理由: なぜこれが問題か
- 修正案: `typescript // 修正後のコード`



9. Phase 7：開発フェーズ — 機能実装の進め方

9-1. 実装順序（推奨）

```
Phase 7-1: 基盤構築
├── 共通レイアウト
├── DB接続
├── 共通UIコンポーネント
└── 認証基盤
        ↓
Phase 7-2: 認証機能
├── ログイン
├── 新規登録
├── ログアウト
└── セッション管理
        ↓
Phase 7-3: 主要CRUD（最もシンプルなリソース）
├── プロジェクトCRUD
└── タスクCRUD
        ↓
Phase 7-4: 高度な機能
├── カンバンボード（D&D）
├── フィルタリング・検索
└── ダッシュボード
        ↓
Phase 7-5: UI/UX改善
├── アニメーション
├── ローディング/エラー/空状態
├── レスポンシブ対応
└── ダークモード
        ↓
Phase 7-6: テスト
├── ユニットテスト
├── コンポーネントテスト
└── E2Eテスト
        ↓
Phase 7-7: デプロイ・最終調整
```

「土台 → 認証 → CRUD → 派生機能 → UI磨き → テスト → デプロイ」という順序が王道です。順序を飛ばすと後で痛い目を見ます（例: 認証より先にCRUDを書くと、後で RLS の入れ込みに苦労する）。

9-2. 1機能の実装フロー（黄金パターン）

Step 1: 仕様の説明（プロンプト）

「○○機能を実装します。仕様は以下…」

「まだコードは書かないで、実装方針を説明して」

▼

Step 2: Claude が実装方針を回答

「以下の手順で実装します: 1. … 2. … 3. …」

▼

Step 3: 方針を確認・修正指示

「方針 OK です。ただし○○は△△に変更して」

または「いいですね、進めてください」

▼

Step 4: 実装（Claude がコード生成）

※ 大きい機能は分割して段階的に実装

「まず○○だけ実装して。完了したら止まって」

▼

Step 5: 動作確認

`npm run dev` でブラウザ確認

エラーがあれば詳細を伝えて修正依頼

▼

Step 6: コードレビュー

`/review` を実行

または手動でレビュー依頼

▼

Step 7: テスト作成

`/create-component` で雛形があればテストも含まれる

追加テストが必要なら test-writer agent に委任

▼

Step 8: コミット

`/commit` を実行

日本語メッセージ + プレフィックス

※ コミット前に必ず `git diff` を自分の目で確認

▼

次の機能へ ↻

`/commit` を AI に任せきりにせず、必ず `git diff` で内容を確認してから走らせるのがおすすめです。push は人間が手動で行うのが基本。

9-3. 精度を最大化するプロンプトテンプレート

```
## 実装する機能
[機能名を1行で明確に]

## この機能の目的
[なぜこの機能が必要か。ユーザーストーリー形式で]
「〇〇として、△△したい。なぜなら□□だから。」

## 具体的な仕様
### 画面
- どの画面で使うか（URL含む）
- 画面のどの位置に配置するか

### データ
- 使用するテーブル/APIエンドポイント
- 必要なクエリ（SELECT/INSERT/UPDATE/DELETE）

### ユーザー操作フロー
1. ユーザーがooをクリック
2. △△が表示される
3. ユーザーが□□を入力
4. xxボタンを押す
5. 成功時：ooが表示される
6. 失敗時：エラートーストが表示される

### 状態管理
- ローディング中：スケルトンUI
- データなし：「まだooがありません」+ 作成ボタン
- エラー時：エラーメッセージ + リトライボタン
- 成功時：トースト通知「ooしました」

## エッジケース
- [入力が空の場合]
- [文字数上限の場合]
- [同時編集が発生した場合]
- [ネットワーク切断時]
- [認証切れの場合]

## 制約
- Server Component / Client Component どちらを使うか
- 使用するUI部品（shadcn/uiのどのコンポーネント）
- パフォーマンス要件（遅延読み込みの要否）

## 参考
- 類似の既存実装：[ファイルパス]
- docs/api.md の該当セクション

## 進め方
```

```
まずコードは書かずに実装方針を説明してください。
合意後に実装に移ります。
```

このテンプレを埋めていくだけで、AIの出力品質が劇的に上がります。

9-4. 実装例：基盤構築

Claude Codeへのプロンプト：

```
Phase 7-1: 基盤構築を開始します。以下を順番に実装してください。

### 1. 共通レイアウト（まずこれだけ）

src/app/layout.tsx を以下の仕様で実装：
- html lang="ja"
- フォント: Inter（next/font/google）
- ThemeProvider でダークモード対応（next-themes）
- Toaster（sonner）をグローバルに配置

src/components/layouts/AppLayout.tsx:
- ヘッダー: ロゴ、ナビゲーション、ユーザーメニュー
- サイドバー: デスクトップ時に表示、モバイルではハンバーガーメニュー
- メインコンテンツエリア
- レスポンシブ対応

shadcn/ui から必要なコンポーネントをインストールしてください。

完了したら一度止まってください。ブラウザで確認します。
```

> 「**完了したら一度止まってください」が重要。**これにより段階的にレビューでき、問題の早期発見が可能です。

9-5. 効果的な修正指示の出し方

```
[悪い例]
「動かないので直してください」
「エラーが出ます」
「なんか変です」

[良い例]
「TaskCardコンポーネントで以下のエラーが出ています。

エラーメッセージ:
TypeError: Cannot read properties of undefined (reading 'name')

発生場所:
src/components/features/task/TaskCard.tsx:25行目

再現手順:
1. /projects/1/board を開く
2. 担当者未設定のタスクが表示されるとクラッシュ

期待する動作:
担当者が未設定の場合は「未割り当て」と表示される

原因の推測:
task.assignee が null の場合の条件分岐が漏れている

このエラーを修正してください。」
```

エラーメッセージ全文・ファイル名・行番号・再現手順・期待動作の5点セットを揃えると、AI のデバッグ精度が一段上がります。

9-6. 大きすぎる変更を制御する方法

```
ストップ。変更範囲が大きすぎます。
以下のように分割して、1つずつ実装してください：

1. まずTaskCardコンポーネントのUI部分だけ
2. 確認後、TaskFormコンポーネント
3. 確認後、カスタムHook（useTaskList）
4. 確認後、API Route
5. 確認後、全体の結合

今は1だけ実装してください。2以降は指示があるまで待ってください。
```

「ストップ」と明確に伝えると、Claude は途中でも作業を中断して指示を仰ぎます。

9-7. 実践ウォークスルー：「確かめ方」つきで並べ替え関数を作る

ここまでの道具（4要素の指示・プランモード・Git・テスト）を全部使って、書籍管理アプリに並べ替え関数を足す流れを通しでやってみます。4-1 で計画した「手順1（sortBooks とテスト）」の部分です。

手順1: ブランチを切り、確かめ方つきで依頼する

```
# 作業用ブランチを作って移動する（0.4.1）
git switch -c feature/sort-books
```

```
lib/sortBooks.ts に、書籍の配列を並べ替える関数 sortBooks を作ってください。

目的:
トップページ（/）の書籍一覧を、利用者が好きな順番で見られるようにする準備です。

仕様:
- 並べ方は "title"（タイトルのあいうえお順）、"rating"（評価の高い順。評価なしは最
後）、"newest"（登録が新しい順）の3種類
- 引数で受け取った配列そのものは書き換えず、並べ替えた新しい配列を返す

確かめ方:
- 同じフォルダに Vitest のテスト sortBooks.test.ts を作り、3種類の並べ方と「元の配列を書き換えな
いこと」をテストする
- npx vitest run を実行し、全テストが通るまで直してから報告する
- テストの期待値は変えないこと。期待値が間違っていると思ったら、直す前に理由を説明して止まる

範囲:
- この2ファイル以外は変更しない
```

手順2: Claude Code が作ったコードとテスト

> **▼ このコードがやること（先に日本語で）：** `sortBooks` は、書籍の配列と並べ方（3種類のどれか）を受け取り、並べ替えた**新しい配列**を返す関数です。テストのほうは、3冊ぶんの見本データを使って「タイトル順・評価順・新しい順に正しく並ぶか」「元の配列が書き換わっていないか」を自動で確かめます。

`lib/sortBooks.ts`

```ts
// 並べ替えに使う項目だけを持つ「書籍の形」
export interface SortableBook {
  title: string;
  rating: number | null;
  created_at: string;
}

// 並べ替えの種類は3通りだけに限定する
export type SortKey = "title" | "rating" | "newest";

// 書籍の配列を受け取り、並べ替えた「新しい配列」を返す
export function sortBooks<T extends SortableBook>(books: T[], key: SortKey): T[] {
  // 元の配列を書き換えないよう、コピーしてから並べ替える
  const copied = [...books];

  if (key === "title") {
    // 日本語の読み順（あいうえお順）で比べる
    return copied.sort((a, b) => a.title.localeCompare(b.title, "ja"));
  }

  if (key === "rating") {
    // 評価なし（null）は 0 とみなし、評価の高い順に並べる
    return copied.sort((a, b) => (b.rating ?? 0) - (a.rating ?? 0));
  }

  // 新しい順: 登録日時を数値（ミリ秒）に直して大きい順に並べる
  return copied.sort(
    (a, b) => new Date(b.created_at).getTime() - new Date(a.created_at).getTime(),
  );
}
```

`lib/sortBooks.test.ts`

```
// Vitest のテスト用の関数を読み込む
import { describe, expect, it } from "vitest";
// テストしたい関数を読み込む
import { sortBooks } from "./sortBooks";

// テストで使う3冊ぶんのデータ
const books = [
  { title: "すいみん学", rating: 3, created_at: "2026-01-10T09:00:00Z" },
  { title: "あめの本", rating: null, created_at: "2026-03-05T09:00:00Z" },
  { title: "かぜの歌", rating: 5, created_at: "2026-02-20T09:00:00Z" },
];

describe("sortBooks", () => {
  it("タイトルのあいうえお順に並ぶ", () => {
    const result = sortBooks(books, "title");
    expect(result.map((b) => b.title)).toEqual(["あめの本", "かぜの歌", "すいみん学"]);
  });

  it("評価の高い順に並び、評価なしは最後になる", () => {
    const result = sortBooks(books, "rating");
    expect(result.map((b) => b.rating)).toEqual([5, 3, null]);
  });

  it("登録が新しい順に並ぶ", () => {
    const result = sortBooks(books, "newest");
    expect(result[0].title).toBe("あめの本");
  });

  it("元の配列は書き換えない", () => {
    sortBooks(books, "title");
    expect(books[0].title).toBe("すいみん学");
  });
});
```

▼ 実行結果（`npx vitest run`。Vitest 3.2.7、パスの一部は伏せています）

```
 RUN  v3.2.7 C:/…/my-books-app

 ✓ lib/sortBooks.test.ts (4 tests) 13ms

 Test Files  1 passed (1)
      Tests  4 passed (4)
   Start at  11:52:51
   Duration  1.98s (transform 323ms, setup 0ms, collect 293ms, tests 13ms, environment
0ms, prepare 932ms)
```

`✓` が付いて「4 passed（4つ成功）」と出れば、Claude Code の報告どおりに動いていることを**あなた自身が**確かめられたことになります。テストを動かす準備（Vitest のインストールと設定）は 10-1 を参照してください。

▼ コードを1つずつ分解して解説

解説1: 型で「受け取れる値」をしぼる

```
export type SortKey = "title" | "rating" | "newest";
```

- `SortKey` は「この3つの文字列のどれか」しか受け付けない型です。`sortBooks(books, "price")` のように存在しない並べ方を渡すと、型チェック（`npx tsc --noEmit`）でエラーになります。
- `<T extends SortableBook>` は「`title`・`rating`・`created_at` を持っていれば、ほかの項目（`author` や `memo` など）があってもよい」という意味です。書籍アプリの `Book` 型をそのまま渡せます。

解説2: `[...books]` で配列をコピーしてから並べ替える

```
const copied = [...books];
```

- `...`（スプレッド構文）で、配列の中身を新しい配列に並べ直してコピーしています。
- JavaScript の `sort` は**元の配列そのものを並べ替えてしまう**ので、コピーしないと、呼び出した側のデータまで順番が変わってしまいます（下の「よくある展開」を参照）。

解説3: 3種類の並べ方

```
return copied.sort((a, b) => a.title.localeCompare(b.title, "ja"));
```

- `localeCompare(…, "ja")` は、日本語の読み順で2つの文字列を比べる関数です。
- 評価順では `b.rating ?? 0` のように、評価なし（`null`）を 0 とみなして最後に回しています。`??` は「左が `null` か `undefined` なら右を使う」という演算子です。

解説4: テストは「見本データ」と「期待する結果」のペア

```
it("元の配列は書き換えない", () => {
  sortBooks(books, "title");
  expect(books[0].title).toBe("すいみん学");
});
```

- `it("説明", () => { … })` が1つのテストです。`expect(実際の値).toBe(期待する値)` で、結果が期待どおりかを確かめます。
- 「元の配列を書き換えない」のように、**仕様に書いた約束ごと**をテストにしておくと、AI が後でコードを書き換えても約束が守られているかを自動で確かめられます。

手順3（よくある展開）：最初の版でテストが落ちたら

実際の作業では、Claude Code の最初の版がテストに落ちることもよくあります。たとえば、コピーを忘れて次のように書いてしまった場合です。

NG（最初の版。`sortBooks.ts` の抜粋）：配列をコピーせずに並べ替えている

```
// （最初の版）配列をそのまま並べ替えている
const copied = books;
```

▼ 実行結果（`npx vitest run`。パスの一部は伏せています）

```
 RUN  v3.2.7 C:/…/my-books-app

 ❯ lib/sortBooks.test.ts (4 tests | 1 failed) 18ms
   ✓ sortBooks > タイトルのあいうえお順に並ぶ 10ms
   ✓ sortBooks > 評価の高い順に並び、評価なしは最後になる 0ms
   ✓ sortBooks > 登録が新しい順に並ぶ 1ms
   × sortBooks > 元の配列は書き換えない 6ms
     → expected 'あめの本' to be 'すいみん学' // Object.is equality

⎯⎯⎯⎯⎯⎯ Failed Tests 1 ⎯⎯⎯⎯⎯⎯

 FAIL  lib/sortBooks.test.ts > sortBooks > 元の配列は書き換えない
AssertionError: expected 'あめの本' to be 'すいみん学' // Object.is equality

Expected: "すいみん学"
Received: "あめの本"

 ❯ lib/sortBooks.test.ts:31:28
     29|   it("元の配列は書き換えない", () => {
     30|     sortBooks(books, "title");
     31|     expect(books[0].title).toBe("すいみん学");
       |                            ^
     32|   });
     33| });

⎯⎯⎯⎯⎯⎯⎯⎯⎯⎯⎯⎯⎯⎯⎯⎯⎯⎯⎯⎯[1/1]⎯


 Test Files  1 failed (1)
      Tests  1 failed | 3 passed (4)
   Start at  11:53:00
   Duration  621ms (transform 57ms, setup 0ms, collect 60ms, tests 18ms, environment
0ms, prepare 113ms)
```

OK（直した版）：コピーしてから並べ替える

```
// 元の配列を書き換えないよう、コピーしてから並べ替える
const copied = [...books];
```

- `Expected`（期待した値）が「すいみん学」、`Received`（実際の値）が「あめの本」です。タイトル順に並べ替えたせいで、元の `books` の先頭まで入れ替わってしまったことが分かります。
- 指示に「全テストが通るまで直してから報告する」と書いておいたので、Claude Code はこの失敗を自分で読み、`[...books]` に直して再実行し、成功してから報告してきます。**確かめ方を渡しておくと、AI が自分のミスを自分で見つけて直せる**、というのがこの例のポイントです。

手順4: 差分を読んでコミットする

最後に、0.4.1 の手順で `git status` と `git diff` を確認し、頼んだ2ファイルだけをコミットします。次の「画面につなぐ」作業（`SortSelect` と `page.tsx`）は、また別の指示・別のコミットにします。

9-8. AI の出力を鵜呑みにしない：検証チェックリスト

テストが通っても、それだけで安心はできません。空港の保安検査のように、AI の変更は**いくつもの関所**を通してから採用します。どの関所も、それぞれ別の種類の間違いを見つけるのが得意です。

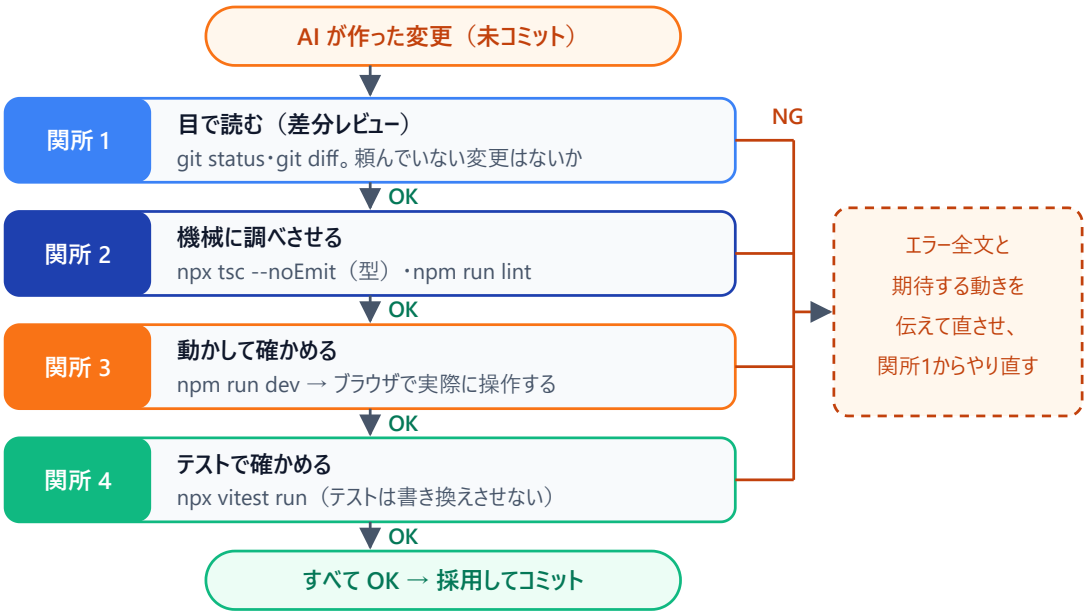


図11-13: AI の変更を採用するまでの4つの関所。どこかで NG なら、直してもらって最初からやり直します

▼ AI の出力を採用する前のチェックリスト

1. ☐ `git status` に、頼んでいないファイルが混ざっていない（関所1）
2. ☐ `git diff` の全行を読み、説明できない行がない。分からない行は「この行は何をしていますか？」と Claude に聞いて理解してから進む（関所1）
3. ☐ 実在しない関数・オプション・パッケージを使っていない。怪しいものは公式ドキュメントや型チェックで確かめる（関所1・2）

4. ☐ `npx tsc --noEmit` と `npm run lint` がエラーなく通る（関所2）
5. ☐ `npm run dev` で実際に画面を操作した。成功するとき・失敗するとき・データが0件のとき、の3パターンを試した（関所3）
6. ☐ `npx vitest run` が通る。**テストの期待値が勝手に書き換えられていない**（関所4）
7. ☐ API キーやパスワードがコードに直接書かれていない。`.env.local` に置くべき値が混ざっていない
8. ☐ Claude の「完了しました」という報告の内容と、実際の差分が一致している

> **ヒント:** チェックリストの2番は、とても大事です。AI が書いたコードを説明できないまま採用すると、次に不具合が起きたとき、あなたも AI も原因を推測するしかなくなります。「動いたから OK」ではなく「説明できるから OK」を合格ラインにしましょう。

型チェックで「存在しないメソッド」を見つける

> **▼ このコードがやること（先に日本語で）：** 書籍をタイトル順に取り出す関数です。NG 版は、AI が別のライブラリ（Prisma など）の書き方と混同して、Supabase には**存在しない** `findMany` というメソッドを使っています。OK 版は Supabase の正しい書き方（ `select` で取り出し、`order` で並べ替え）です。型チェックの結果を比べると、ハルシネーションが機械的に見つかることが分かります。

NG（`lib/getBooks.ts`。AI が提案した版）

```ts
// サーバー用の Supabase クライアントを作る関数を読み込む
import { createClient } from "@/lib/supabase/server";

// 書籍をタイトル順に取得する
export async function getBooksSortedByTitle() {
  const supabase = await createClient();
  // AI が提案したコード: findMany は supabase-js には存在しない
  const { data, error } = await supabase
    .from("books")
    .findMany({ orderBy: { title: "asc" } });
  if (error) throw error;
  return data;
}
```

▼ 実行結果（`npx tsc --noEmit`。supabase-js 2.117.3 / TypeScript 7.0.2）

```
lib/getBooks.ts(10,6): error TS2339: Property 'findMany' does not exist on type
'PostgrestQueryBuilder<any, any, any, "books", unknown>'.
```

OK（直した版）

```ts
// サーバー用の Supabase クライアントを作る関数を読み込む
import { createClient } from "@/lib/supabase/server";

// 書籍をタイトル順に取得する
export async function getBooksSortedByTitle() {
  const supabase = await createClient();
  // supabase-js の正しい書き方: select で取得し、order で並べ替える
  const { data, error } = await supabase
    .from("books")
    .select("*")
    .order("title", { ascending: true });
  if (error) throw error;
  return data;
}
```

▼ 実行結果（`npx tsc --noEmit`）

```
（何も表示されずに終了）
```

（型チェックのためだけに、`@/lib/supabase/server` を supabase-js で作った簡易版に置き換えた練習用フォルダで実行した結果です。第7章の本物の `createClient` でも、`findMany` については同じ TS2339 のエラーになります。）

▼ コードを1つずつ分解して解説

解説1: エラーメッセージの読み方

```
lib/getBooks.ts(10,6): error TS2339: Property 'findMany' does not exist on type
'PostgrestQueryBuilder<…>'.
```

- `(10,6)` は「10行目の6文字目」、`TS2339` は TypeScript のエラー番号です。
- 「`findMany` というプロパティ（メソッド）は、`PostgrestQueryBuilder`（Supabase のクエリを組み立てる部品）には存在しない」という意味です。つまり **AI が存在しない書き方をした**ことが、型チェックだけで分かります。

解説2: 正しい書き方は公式ドキュメントで確かめる

```ts
.select("*")
.order("title", { ascending: true });
```

- Supabase では、`from("テーブル名")` の後に `select`（取り出す）、`order`（並べ替え）、`eq`（条件）などをつなげて書きます（第7章で `order("created_at", { ascending: false })` を使いました）。
- AI に「findMany は存在しないとエラーが出ました。supabase-js の公式ドキュメントに沿った書き方で直してください」とエラー全文を貼って頼むのが近道です。

> **つまずきポイント⑩: AI が「それっぽい」存在しないメソッドを使う**
>
> AI は、似たライブラリ（Prisma、Drizzle など）の書き方を混ぜてしまうことがあります。見た目は自然なので、読んだだけでは気づきにくいのが厄介な点です。「型チェック（ `npx tsc --noEmit` ）を通す」を確かめ方に入れておけば、存在しないメソッドは上のように機械的に見つかります。

NG（存在しないメソッドを信じる）：

```
// Prisma 風の書き方が混ざっている
const { data } = await supabase
  .from("books")
  .findMany({ orderBy: { title: "asc" } });
```

OK（supabase-js の書き方）：

```
// select で取得し、order で並べ替える
const { data } = await supabase
  .from("books")
  .select("*")
  .order("title", { ascending: true });
```

> **つまずきポイント⑪: 「テストが通るようにして」と頼み、テストのほうを書き換えられる**
>
> テストが失敗したときに「テストが通るようにして」とだけ頼むと、AI がコードではなく**テストの期待値**を書き換えて、無理やり成功させることがあります。こうなるとテストは何も守ってくれません。9-7 の指示のように「テストの期待値は変えない」と書き、テストファイルの差分も必ず読みましょう。

NG（テストを書き換えて通した差分）：9-7 の「最初の版」のバグが残ったまま、テストが「成功」してしまいます。

```
diff --git a/lib/sortBooks.test.ts b/lib/sortBooks.test.ts
index ae01d85..6283a3c 100644
--- a/lib/sortBooks.test.ts
+++ b/lib/sortBooks.test.ts
@@ -26,8 +26,8 @@ describe("sortBooks", () => {
     expect(result[0].title).toBe("あめの本");
   });

-  it("元の配列は書き換えない", () => {
+  it("並べ替え後は先頭が変わる", () => {
     sortBooks(books, "title");
-    expect(books[0].title).toBe("すいみん学");
+    expect(books[0].title).toBe("あめの本");
   });
 });
```

OK（期待値を守らせる指示）：

```
npx vitest run で「元の配列は書き換えない」のテストが失敗しています。
テストファイル（sortBooks.test.ts）は変更せず、sortBooks.ts のほうを直してください。
テストの期待値が間違っていると思う場合は、直す前に理由を説明して止まってください。
```

---

10. Phase 8：テスト・品質管理

10-1. テスト環境のセットアップ

Claude Codeへのプロンプト：

```
テスト環境をセットアップしてください。

## パッケージ
- vitest
- @testing-library/react
- @testing-library/jest-dom
- @testing-library/user-event
- @vitejs/plugin-react
- jsdom
- playwright（E2E用）

## 設定ファイル
1. vitest.config.ts
   - React プラグイン
   - jsdom 環境
   - パスエイリアス（@/）対応
   - カバレッジ設定（v8 provider）

2. src/test/setup.ts
   - @testing-library/jest-dom のインポート
   - グローバルモック（next/navigation等）

3. playwright.config.ts
   - ベースURL: http://localhost:3000
   - ブラウザ: chromium
   - スクリーンショット: on failure

## npmスクリプト（package.jsonに追加）
- "test": "vitest run"
- "test:watch": "vitest"
- "test:coverage": "vitest run --coverage"
- "test:e2e": "playwright test"
- "test:e2e:ui": "playwright test --ui"

セットアップ後、サンプルテストを1つ作成して
npm test が通ることを確認してください。
```

10-2. テスト作成の依頼

Claude Codeへのプロンプト（または `/review` Skill を使用）：

```
タスクCRUD機能のテストを作成してください。

## テスト対象と種別

### ユニットテスト（src/lib/validations/task.test.ts）
- zodスキーマのバリデーション
  - 正常なデータ → パス
  - タイトル空 → エラー
  - タイトル101文字 → エラー
  - 不正なステータス値 → エラー
  - 不正なUUID → エラー

### コンポーネントテスト（src/components/features/task/）
- TaskCard.test.tsx
  - 正常にレンダリングされる
  - タイトル、担当者、期限が表示される
  - 担当者未設定時は「未割り当て」が表示される
  - クリックで onSelect が呼ばれる
  - 優先度バッジの色が正しい

- TaskForm.test.tsx
  - フォームが正常に表示される
  - 空のタイトルで送信するとバリデーションエラー
  - 正常な値で送信すると onSubmit が呼ばれる
  - キャンセルボタンで onCancel が呼ばれる

### API Routeテスト（src/app/api/tasks/route.test.ts）
- POST: 正常なリクエスト → 201
- POST: バリデーションエラー → 400
- POST: 認証なし → 401
- GET: タスク一覧取得 → 200
- PATCH: タスク更新 → 200
- DELETE: ソフトデリート → 200

テストはAAA（Arrange-Act-Assert）パターンで書いてください。
テスト名は日本語で記述してください。
作成後 npm test を実行して全テストがパスすることを確認してください。
```

AAA は「Arrange（準備）→ Act（実行）→ Assert（検証）」の3ステップでテストを書く方針です。1テスト1観点・前提と検証が一目で分かる構造を目指します。

11. Phase 9：デプロイと最終調整

11-1. デプロイ前チェック

Claude Codeへのプロンプト：

```
デプロイ前の最終チェックを実行してください。

1. ビルド確認
   npm run build を実行してエラーがないか確認

2. 型チェック
   npx tsc --noEmit を実行

3. Lint
   npm run lint を実行

4. テスト
   npm test を実行

5. コードスキャン
   以下をgrepで検索して該当があれば修正：
   - console.log（テストファイル以外）
   - any型（テストファイル以外）
   - TODO/FIXME コメント（一覧表示）
   - ハードコードされたURL/APIキー

6. 環境変数チェック
   - .env.example に全ての必要な変数が記載されているか
   - .env が .gitignore に含まれているか

7. セキュリティチェック
   - npm audit を実行
   - 脆弱性があれば対応

全ての結果をレポートしてください。問題があれば修正してください。
```

デプロイ前にこのチェックリストを実行する習慣を持つだけで、本番事故がぐっと減ります。

12. Context管理とMemoryシステム

12-1. Contextとは

Claude Codeのコンテキストウィンドウには上限があります。長い会話を続けると、古いメッセージが自動的に圧縮（compact）されます。

前述のとおりコンテキストウィンドウは AI が一度に扱えるトークン数の上限のことです。会話・読み込みファイル・指示すべてがこの「容器」に入り、満杯になると古いものから消えたり要約されたりします。

▼ たとえ話: コンテキストは Claude の「作業机」

机の上には、仕事の指示書（CLAUDE.md）、これまでのやり取りのメモ（会話の履歴）、読みかけの資料（読んだファイルやコマンドの出力）が積まれていきます。机が資料で埋まってくると、大事な指示書が資料の下に埋もれて見落とされやすくなります。人間と同じで、ときどき机を片付けるのが良い仕事のコツです。

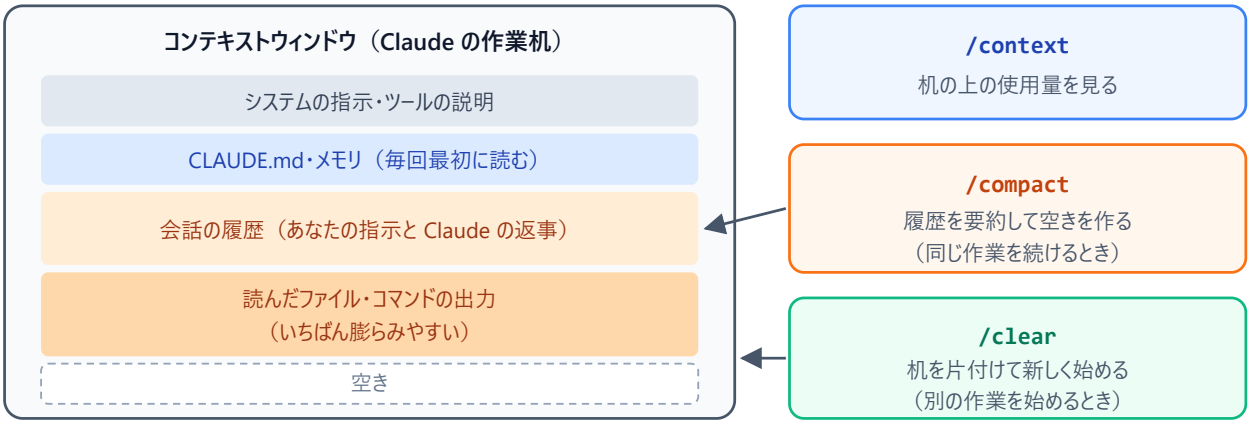


図11-14: コンテキストウィンドウは Claude の作業机。いっぱいになる前に /compact か /clear で片付けます

- CLAUDE.md とメモリは、`/compact` や `/clear` の後も毎回読み込まれます。だから「毎回守ってほしいこと」は会話ではなく CLAUDE.md に書くのです。
- 読んだファイルやコマンドの出力は、机をいちばん占領します。「`npm run build` の出力を全部見せて」のような依頼を繰り返すと、すぐに机が埋まります。

12-2. Context管理コマンド

コマンド	説明
`/compact`	手動でコンテキストを圧縮（会話を要約して容量削減）
`/clear`	コンテキストを完全にクリア（新しいセッション開始）
`/usage`	料金・プランの使用量の目安を確認（`/cost` は同じコマンドの別名）
`/context`	いまのコンテキストの使用量を色付きの図で表示

12-3. /compact の使いどころ

```
以下のタイミングで /compact を実行する：
- 1つの機能の実装が完了し、次の機能に移る時
- 長い議論が終わって結論が出た時
- エラーの修正が完了した時
- コンテキストウィンドウの使用率が高くなった時
```

> **注意**: `/compact` を実行するとCLAUDE.mdは再読み込みされますが、会話の細かいニュアンスは失われます。`SessionStart` Hook（マッチャー `compact`）で重要な情報を再注入すると良いです（2-5 の設定例を参照）。

公式のベストプラクティスでは、**関係のない次の作業に移るときは `/clear` で机をまっさらにすることが勧められています。**同じ機能の続きなら `/compact`、別の機能なら `/clear`、と使い分けると覚えやすいです。

> **つまずきポイント⑫: 同じ会話で何度も「違う、そうじゃない」と直し続ける**
>
> うまくいかない修正を同じ会話で何度も重ねると、机の上が「失敗した案」で埋まり、Claude がかえって混乱していきます。公式ドキュメントでも、同じ問題で2回以上修正してもうまくいかないときは、`/clear` して、学んだことを反映した良い指示で最初からやり直すよう勧めています。

NG（泥沼の追加修正）：

```
違う、そうじゃない
まだ動かない。もう一回直して
さっきの方法に戻して、でも別のやり方で
```

OK（`/clear` して、分かったことを入れて指示し直す）：

```
/clear
```

```
lib/sortBooks.ts の sortBooks で、評価順（"rating"）の並びが正しくありません。
分かっていること：
- 評価なし（null）の本が先頭に来てしまう。期待は「評価なしは最後」
- npx vitest run の「評価の高い順に並び、評価なしは最後になる」が失敗する
sortBooks.ts だけを直し、npx vitest run が通ったら報告してください。テストは変更しないでください。
```

12-4. Memory システム

Claude Codeは会話を跨いで記憶を保持するMemoryシステムを持っています。コンテキストとは別の場所に保管され、セッションをまたいで読み込まれます。

保存場所： `~/.claude/projects/<project>/memory/`

```
memory/
├── MEMORY.md          ← インデックス（セッション開始時に読み込み）
├── user_preferences.md ← ユーザーの好み
├── project_decisions.md ← 技術的な意思決定
└── feedback_testing.md  ← フィードバック記録
```

Memoryに保存すべき情報： - ユーザーの役割・スキルレベル - プロジェクト固有の意思決定とその理由 - フィードバック（「こうしてほしい/しないでほしい」） - 外部リソースへの参照

Memoryに保存すべきでない情報： - コードパターン（コードを読めばわかる） - Git履歴（git logで確認できる） - CLAUDE.mdに書いてあること

Memoryの操作プロンプト：

```
# 記憶させる
「覚えておいて：このプロジェクトではServer Actionsは使わず、
Route Handlersで統一する決定をした。理由はテストの書きやすさ。」

# 思い出させる
「前のセッションで決めたAPI設計方針を思い出して」

# 忘れさせる
「zustandを使う決定は撤回したので、その記憶を削除して」
```

Memory の有効/無効設定：

```
{
  "autoMemoryEnabled": true
}
```

- `autoMemoryEnabled`: Claude が自動的に Memory を読み書きするか。`false` にすると明示指示時のみ動く。

---

13. キーボードショートカットのカスタマイズ

13-1. 設定ファイル

場所： `~/.claude/keybindings.json`

13-2. デフォルトの主要ショートカット

キー	動作	コンテキスト
`Enter`	プロンプト送信	Chat
`Escape`	キャンセル/中断	Chat
`Escape` を2回	入力欄が空なら巻き戻しメニュー（`/rewind`）、文字があれば入力の消去	Chat
`Shift+Tab`	パーミッションモード切り替え	Chat
`Ctrl+C`	実行中断（何も動いていなければ入力の消去、もう1回で終了）	Global
`Ctrl+D`	終了（800ミリ秒以内に2回押す）	Global
`Ctrl+L`	画面の再描画（入力や会話は消えない）	Chat
`Ctrl+O`	詳細表示（thinking表示）切り替え	Global
`Ctrl+R`	履歴検索	Global
`Alt+T`（Win/Linux）	Thinking Mode 切り替え（Claude 5 系のモデルは常に考えるので効果なし）	Chat
`Ctrl+G`	外部エディタで編集	Chat

13-3. カスタマイズ例

```
{
  "$schema": "https://www.schemastore.org/claude-code-keybindings.json",
  "bindings": [
    {
      "context": "Chat",
      "bindings": {
        "ctrl+k": "chat:clearInput",
        "ctrl+e": "chat:externalEditor",
        "ctrl+u": null
      }
    },
    {
      "context": "Global",
      "bindings": {
        "ctrl+t": "app:toggleTodos",
        "ctrl+o": "app:toggleTranscript"
      }
    }
  ]
}
```

- `$schema`: エディタが入力補完するための JSON Schema 参照。
- `bindings`: コンテキストごとのバインディング配列。
- `context`: バインディングが効く場面（Chat / Global など）。
- `"ctrl+k": "chat:clearInput"`: Ctrl+K に `chat:clearInput` を割り当て。名前に反して入力は消えず、画面の再描画（表示が崩れたときの立て直し）をする操作です（既定では Ctrl+L）。
- `"ctrl+u": null`: 既存のバインディングを無効化。

13-4. 利用可能なコンテキスト

コンテキスト	説明
`Global`	どこでも有効
`Chat`	メイン入力エリア
`Autocomplete`	補完メニュー表示中
`Confirmation`	権限確認ダイアログ
`Transcript`	詳細表示モード
`HistorySearch`	Ctrl+Rの履歴検索中
`DiffDialog`	diff表示中

13-5. コード記法

```
ctrl+k          # Control + K
shift+tab       # Shift + Tab
ctrl+shift+c    # 複数修飾キー
ctrl+x ctrl+e   # コード入力（Ctrl+X → Ctrl+E）
shift+k         # Shift + K（K と k は同じ扱いなので、Shift は明示する）
escape, enter   # 特殊キー名
```

「コード入力」は2段押し（プレフィックスキーを押してから別のキーを押す）の記法です。Emacs 風の操作を好む人向け。

---

14. Worktrees（並列開発）

14-1. Worktrees とは

Git worktreeを使って、**メインの作業を中断せずに別ブランチで並列作業**できます。1つのリポジトリから複数の作業ディレクトリを派生させ、それぞれで別ブランチを進められる仕組みです。

14-2. 使い方

```
# 名前を指定してworktreeを作成
# --worktree オプションで feature-auth という新しい worktree を作る
claude --worktree feature-auth

# 自動で名前を生成
# 名前を省略するとランダムな名前で worktree を作成
claude --worktree

# tmux連携（macOS/Linux）
# -w は --worktree の短縮形。--tmux で新しい tmux ペインに展開
claude -w feature-auth --tmux
```

worktreeの保存場所： `.claude/worktrees/<name>/`

Gitブランチ： `worktree-<name>` が自動作成（`origin/HEAD` ベース）

14-3. .worktreeinclude ファイル

gitignoreされたファイルをworktreeにコピーするための設定：

プロジェクトルートに `.worktreeinclude` を作成：

```
.env
.env.local
.claude/settings.local.json
CLAUDE.local.md
```

- 各行に「gitignore されているがworktreeにはコピーしたい」ファイル名を書く。
- `.env` などはGit追跡から外れているが、worktreeで開発するときに必要なのでこの仕組みでコピーする。

> gitignoreされた **かつ** .worktreeincludeにマッチするファイルのみコピーされます。

14-4. 活用シーン

```
シーン1: メイン機能の開発中にバグ修正が必要
→ claude -w hotfix-auth で別worktreeを作成し、バグ修正

シーン2: 2つの機能を同時に進めたい
→ Terminal 1: claude -w feature-kanban
→ Terminal 2: claude -w feature-dashboard

シーン3: 破壊的な変更を試したい
→ claude -w experiment-new-ui で実験
→ 問題なければマージ、ダメなら破棄
```

---

15. CLIフラグとオプション一覧

15-1. セッション管理

```
# 新しいセッション開始（最も普通の起動方法）
claude
# 前回のセッションを再開（-c は continue の意味）
claude -c
# 名前指定でセッション再開
claude -r "session-name"
# セッションに名前をつけて開始（後で -r で呼び戻せる）
claude -n "my-feature"
# Pull Request #123 にひも付いたセッションの一覧から選んで再開
claude --from-pr 123
```

15-2. モードとモデル

```
# Plan Mode で起動（読み取り専用）
claude --permission-mode plan
# 編集は確認なしで自動承認（信頼できるタスク向け）
claude --permission-mode acceptEdits
# Auto Mode（バックグラウンドで安全性チェック）
claude --permission-mode auto
# Opus モデルを使用（高品質・高コスト）
claude --model opus
# Sonnet モデルを使用（安価・高速）
claude --model sonnet
# 推論レベル（low / medium / high / xhigh / max）を指定する。最上位は max
claude --effort high
```

15-3. 非対話モード（スクリプト連携）

```
# -p は print mode。質問を投げて回答だけ受け取る（対話なし）
claude -p "このプロジェクトの構成を教えて"

# JSON 出力に切り替え。スクリプトでパースしやすい
claude -p "package.jsonの依存関係を一覧して" --output-format json

# 最大ターン数を制限（無限ループ防止）
claude -p "テストを全部実行して" --max-turns 5

# 予算上限を USD で指定。超えそうになると停止
claude -p "全機能を実装して" --max-budget-usd 5.00
```

15-4. コンテキストとツール

```
# 別ディレクトリ（共有ライブラリ等）もコンテキストに含める
claude --add-dir ../shared-lib

# 使用ツール制限（カンマ区切り）
claude --tools "Bash,Edit,Read"
# 特定ツールだけ禁止
claude --disallowedTools "WebFetch"

# システムプロンプトを上書き（Claude の根本的な指示を変える）
claude --system-prompt "You are a TypeScript expert"
# システムプロンプトをファイルから読み込み
claude --system-prompt-file ./custom-prompt.txt

# MCP 設定ファイルを個別に指定
claude --mcp-config ./custom-mcp.json
```

15-5. デバッグ

```
# デバッグモード（内部の挙動を詳細出力）
claude --debug
# デバッグログをファイルに記録
claude --debug-file /tmp/claude.log
# 通常より詳細な出力
claude --verbose
```

16. トラブルシューティング

16-1. よくある問題と解決法

問題	原因	解決法
Claudeが同じ変更を繰り返す	コンテキスト不足	`/compact` して明確に再指示
生成コードの品質が低い	指示が曖昧	具体的な仕様とコード例を提示
意図と異なるファイルを編集	パスの認識ずれ	絶対パスを明示して指示
コンテキストが足りない	会話が長すぎる	`/compact` or `/clear` で新セッション
Hookが動かない	matcherの不一致	`--debug` でHookの発火を確認
MCPサーバーに接続できない	設定ミス	`claude mcp list` で状態確認
Permission denied	deny設定	`settings.json` のdenyリストを確認

16-2. コード品質が低い場合の改善指示

```
生成されたコードに以下の問題があります。修正してください：

1. コンポーネントが大きすぎる（300行超）
   → 責務ごとに分割してください
   → UIとロジック（カスタムHook）を分離してください

2. ロジックがコンポーネントに直書き
   → カスタムHookに抽出してください
   → ファイル: src/hooks/use[Feature].ts

3. マジックナンバーがある
   → 定数ファイルに定義してください
   → ファイル: src/lib/constants.ts

4. 型定義が甘い
   → any/unknown を具体的な型に変更
   → 型ファイル: src/types/[feature].ts

CLAUDE.mdの規約に準拠しているか再確認してから修正してください。
```

16-3. Claude が暴走した場合

```
ストップ。

今行った変更を全て説明してください。
どのファイルを、どのように変更しましたか？
一覧にして報告してください。

報告後、必要な修正があれば指示します。
勝手に追加の変更はしないでください。
```

それでも止まらない場合は、`Ctrl+C` で実行を強制中断し、`git status` と `git diff` で被害状況を確認してから、必要に応じて `git restore` や `git checkout` で巻き戻します（破壊的コマンドは事前にコミットしてあれば安全に戻せます）。

---

17. ベストプラクティス総まとめ

17-1. プロンプトの黄金ルール 10箇条

#	ルール	具体例
1	**具体的に指示する**	「いい感じに」→「タイトルは1-100文字、zodバリデーション付き」
2	**段階的に進める**	「全部作って」→「まずレイアウトだけ。確認後に次へ」
3	**コンテキストを与える**	「ボタンを作って」→「カンバンボードのタスク作成ボタン。クリックでモーダルが開く」
4	**制約を明示する**	「フォーム作って」→「react-hook-form + zod。Server Actionsは使わない」
5	**レビューを挟む**	各機能実装後に `/review` を実行
6	**エッジケースを示す**	「データなし・ローディング・エラー時の表示も実装して」
7	**参照を示す**	「src/components/features/project/ProjectCard.tsxを参考にして」
8	**まず方針を確認**	「まだコードは書かないで。実装方針を説明して」
9	**停止ポイントを設ける**	「完了したら一度止まって。ブラウザで確認します」
10	**エラーは詳細に伝える**	エラーメッセージ + ファイル + 行番号 + 再現手順

17-2. 機能別の推奨設定

機能	フェーズ	設定場所
危険コマンドのブロック	Phase 0（最初）	`~/.claude/settings.json`
MCP連携	Phase 0	`.mcp.json`（共有）/ `~/.claude.json`（個人）
Hooks（自動フォーマット等）	Phase 0	`settings.json`
Keybindings	Phase 0	`~/.claude/keybindings.json`
CLAUDE.md	Phase 3（要件確定後）	プロジェクトルート
Rules	Phase 4（実装開始前）	`.claude/rules/`
Skills	Phase 5（開発中）	`.claude/skills/`
Subagents	Phase 6（開発中）	`.claude/agents/`
Memory	随時（自動）	`~/.claude/projects/*/memory/`
Worktrees	必要時（並列開発）	CLIフラグ

17-3. やりがちな失敗と対策

失敗パターン	対策
要件を曖昧にしたまま実装開始	Plan Modeで要件を先に固める
CLAUDE.mdを作らない/更新しない	Phase 3で作成し、変更の度に更新
一度に大量の機能を依頼	1機能ずつ実装→確認→コミット
エラーの詳細を伝えない	エラーメッセージ、ファイル、行番号を正確に
レビューを省略	`/review` Skillで毎回レビュー
Gitコミットを溜める	`/commit` で機能単位にこまめに
コンテキストを管理しない	機能完了ごとに `/compact`
Rulesを活用しない	パス別ルールで規約自動適用
Skillsを作らない	繰り返す操作はSkill化して効率化
全てデフォルトモードで実行	設計=Plan、実装=acceptEdits、レビュー=Plan
AIにそのまま git push させる	コミット内容を確認し、push は人間が手動で行う
`git status` を確認せず作業開始	毎セッション最初に `git status` / `git diff` で現状把握

17-4. 推奨ワークフロー全体図

Phase 0: 環境構築

- Claude Code インストール
- `settings.json`（Permissions + Hooks）
- MCP 設定
- Keybindings 設定

Phase 1: プロジェクト初期化

- Next.js + TypeScript + Tailwind
- Git 初期化
- `.claude/` ディレクトリ構造作成

Phase 2: 要件定義 Plan Mode

- アプリ全体像の定義
- 画面設計
- データベース設計
- API 設計
- 設計ドキュメント保存

Phase 3-6: 開発基盤構築

- `CLAUDE.md` 作成
- Rules 配置
- Skills 作成
- Subagents 定義
- ESLint / Prettier / TypeScript 設定

Phase 7: 機能開発（繰り返し）

仕様説明 → 方針合意 → 実装 → 確認 → `/review` → 修正 → テスト → `/commit`

※ 機能完了ごとに `/compact` ／ ※ 並列作業は Worktree を活用 ／ ※ コミット前に `git diff`、push は人間が手動

Phase 8-9: テスト・デプロイ

- テスト環境セットアップ
- ユニット / コンポーネント / E2E テスト
- デプロイ前チェック
- Vercel デプロイ

18. 付録：設定ファイルパス早見表

機能	ファイルパス	Git共有
ユーザー設定	`~/.claude/settings.json`	No
プロジェクト設定	`.claude/settings.json`	Yes
ローカル設定	`.claude/settings.local.json`	No
CLAUDE.md（共有）	`./CLAUDE.md` or `./.claude/CLAUDE.md`	Yes
CLAUDE.md（個人）	`./CLAUDE.local.md`	No
CLAUDE.md（全体）	`~/.claude/CLAUDE.md`	No
AGENTS.md（CLAUDE.md が無いとき）	`./AGENTS.md`	Yes
Rules	`.claude/rules/*.md`	Yes
Skills	`.claude/skills/<name>/SKILL.md`	Yes
Skills（個人）	`~/.claude/skills/<name>/SKILL.md`	No
Subagents	`.claude/agents/<name>.md`	Yes
Subagents（個人）	`~/.claude/agents/<name>.md`	No
Hooks	`settings.json` 内の `hooks`	設定場所による
MCP（共有・project スコープ）	`.mcp.json`（プロジェクト直下）	Yes
MCP（個人・local スコープ）	`~/.claude.json` の中のプロジェクト別の欄	No
MCP（全体・user スコープ）	`~/.claude.json`	No
Keybindings	`~/.claude/keybindings.json`	No
Memory	`~/.claude/projects/<project>/memory/`	No
Worktree	`.claude/worktrees/<name>/`	No
Worktreeインクルード	`.worktreeinclude`	Yes

付録：開発開始から完了までのチェックリスト

```
=== Phase 0: 環境構築 ===
□ Claude Code インストール済み
□ settings.json で危険コマンドをブロック済み
□ Permissions の allow/deny リスト設定済み
□ MCP サーバー設定済み（GitHub, Supabase等）
□ Hooks 設定済み（自動フォーマット、保護ファイル、通知）
□ Keybindings カスタマイズ済み（必要なら）

=== Phase 1: プロジェクト初期化 ===
□ Next.js プロジェクト作成済み
□ 追加パッケージインストール済み
□ Git リポジトリ初期化済み
□ .claude/ ディレクトリ構造作成済み
□ .gitignore 設定済み（.env, settings.local.json等）

=== Phase 2: 要件定義（Plan Mode） ===
□ アプリ全体像の定義完了
□ 画面一覧と画面設計完了
□ データベース設計完了
□ API一覧と設計完了
□ 設計ドキュメント保存済み（docs/）

=== Phase 3-6: 開発基盤 ===
□ CLAUDE.md 作成済み
□ CLAUDE.local.md 作成済み（必要なら）
□ .claude/rules/ にルールファイル配置済み
□ .claude/skills/ にSkill作成済み
□ .claude/agents/ にSubagent定義済み
□ ESLint 設定済み
□ Prettier 設定済み
□ TypeScript strict設定済み

=== Phase 7: 機能開発 ===
□ 基盤（レイアウト、DB接続、共通コンポーネント）構築済み
□ 認証機能 実装・レビュー・テスト・コミット済み
□ 主要CRUD 実装・レビュー・テスト・コミット済み
□ 高度な機能 実装・レビュー・テスト・コミット済み
□ UI/UX改善済み

=== Phase 8-9: テスト・デプロイ ===
□ テスト環境セットアップ済み
□ ユニットテスト作成・パス済み
□ コンポーネントテスト作成・パス済み
□ E2Eテスト作成・パス済み
□ ビルド成功確認済み（npm run build）
□ 型チェック通過済み（npx tsc --noEmit）
```

```
□ Lint通過済み（npm run lint）
□ console.log / any型 / TODO 残りなし
□ 環境変数整理済み（.env.example）
□ npm audit 脆弱性なし
□ デプロイ完了
```

練習問題（解答・解説つき）

この章の内容が身に付いたかを確かめましょう。まず自分で考えてから、後ろの「解答と解説」を読んでください。Claude Code が手元にある人は、実際に書籍管理アプリで試すとより理解が深まります。

問題1: あいまいな指示を書き直す（0.1.2）

次の指示は、Claude Code に意図が伝わりにくい例です。「トップページ（`/`）の書籍一覧の上に、登録されている本の冊数を『全 3 冊』のように表示したい」という意図が伝わるよう、図11-3 の4要素（目的・場所と範囲・完成条件・確かめ方）を入れて書き直してください。

```
一覧ページをいい感じにして
```

問題2: 権限ルールの判定を予想する（2-2・2-3）

`.claude/settings.json` が次のとき、Manual（`default`）モードで Claude Code が (a)〜(d) のコマンドを実行しようとしたら、それぞれどうなりますか？「確認なしで実行」「確認が出る」「ブロック」のどれかで答えてください。

```
{
  "permissions": {
    "allow": ["Bash(npm run *)", "Bash(git commit *)"],
    "ask": ["Bash(git commit *)"],
    "deny": ["Bash(npm run deploy)"]
  }
}
```

- (a) `npm run lint`
- (b) `npm run deploy`
- (c) `git commit -m "feat: 並べ替えを追加"`
- (d) `npm install zod`

問題3: 報告と差分が合わない（0.4.1・9-8）

`sortBooks.ts` とそのテストだけを作るよう頼んだところ、Claude Code から「並べ替え機能を実装し、テストも通りました」と報告がありました。`git status --short` の結果は次のとおりです。あなたは次に何を確かめ、どう対処しますか？

```
 M lib/sortBooks.test.ts
 M lib/supabase/server.ts
?? components/SortSelect.tsx
?? lib/sortBooks.ts
```

（`sortBooks.test.ts` は、前回のコミットで一度作ってあったものとします。）

問題4: どの機能に書く？（1-4）

次の (a)〜(e) をかなえるには、CLAUDE.md・Rules・Skills・Hooks・Permissions のどれを使うのが最も適切ですか？

- (a)「部品（コンポーネント）は `components/` に1ファイル1つ置く」という決まりを、どの作業でも守ってほしい
- (b) Claude がファイルを編集するたびに、必ず Prettier で整形したい
- (c) `.env.local` を、どんな場合でも Claude に読ませたくない
- (d)「lint・型チェック・テストを実行して結果を表にまとめる」という作業を、`/` で始まる短い命令で何度も頼みたい
- (e) `app/api/` の中のファイルを触るときだけ守ってほしい決まりがある

問題5: 計画に頼んでいない変更が入っている（4-1）

プランモードで「`sortBooks` 関数とテストを作る」計画を頼んだところ、計画の「変更するファイル」に次の行が含まれていました。承認画面ではどの選択肢を選び、何と伝えますか？

```
3. app/layout.tsx（変更）: ヘッダーのデザインを新しくする
```

問題6: どこまで戻す？（0.4.2）

次の (a)〜(c) の場面で、最も適切な「戻し方」を答えてください。

- (a) Claude Code が作業中だが、明らかに頼んだのと違うファイルを編集し始めた
- (b) コミットしていない状態で、Claude が `Edit` ツールで書き換えた3つのファイルを、2つ前の指示を送った時点の状態に戻したい
- (c) 昨日コミットして push 済みの変更に不具合があったので、その変更を取り消したい

問題7: 同じバグで3回つまずいた（12-3）

`sortBooks` の評価順のバグを直そうと、同じ会話で3回修正を頼みましたが、まだ直りません。このあと別の機能（書籍の削除）にも取りかかる予定です。どうするのがよいですか？

問題8: AI のコードの問題を見つける（9-8）

Claude Code が次のコードを提案しました。問題点と、それを機械的に見つける方法、正しい書き方を答えてください。

```
// サーバー用の Supabase クライアントを作る関数を読み込む
import { createClient } from "@/lib/supabase/server";

// 書籍をタイトル順に取得する
export async function getBooks() {
  const supabase = await createClient();
  const { data } = await supabase.from("books").select("*").sortBy("title");
  return data;
}
```

解答と解説

問題1の解答

```
app/page.tsx だけを変更してください。

目的:
登録した本が何冊あるかを、一覧を見た瞬間に分かるようにしたい。

変更内容:
- 書籍一覧の上に「全 N 冊」（N は取得した書籍の件数）と表示する
- 0 冊のときは表示しない（今ある「書籍が登録されていません」の案内だけにする）

守ること:
- BookList などほかのファイルは変更しない

確かめ方:
- npm run lint と npx tsc --noEmit を実行し、結果を報告してください
- 私は npm run dev で、本が 0 冊・3 冊のときの表示を確認します
```

解説:「いい感じに」を、①目的（冊数がすぐ分かる）、②場所と範囲（`app/page.tsx` だけ）、③完成条件（「全 N 冊」と表示、0冊のときは出さない）、④確かめ方（lint と型チェック、あなたのブラウザ確認）に分解しました。特に「0冊のとき」のような**境目のケース**を書いておくと、AI の推測に任せる部分が減ります。文面は人によって違ってかまいません。4つの要素がそろっていれば正解です。

問題2の解答

コマンド	結果	理由
(a) `npm run lint`	確認なしで実行	deny・ask には一致せず、allow の `Bash(npm run *)` に一致
(b) `npm run deploy`	ブロック	allow にも一致するが、先に調べる deny の `Bash(npm run deploy)` に一致
(c) `git commit -m "…"`	確認が出る	allow にも書いてあるが、先に調べる ask に一致
(d) `npm install zod`	確認が出る	どのルールにも一致せず、Manual モードなので確認

解説: ルールは **deny → ask → allow** の順に調べられ、最初に一致したもので決まります（図11-9）。(b) と (c) のように、allow に同じものが入っていても、deny や ask が先に勝ちます。(d) のように「どれにも一致しない」ときの動きはモードしだいで、Manual なら確認が出ます。

問題3の解答

1. **`sortBooks.test.ts` の差分を読む**: `git diff lib/sortBooks.test.ts` で、テストの期待値が書き換えられていないかを確認します（つまずきポイント⑪）。
2. **頼んでいない `server.ts` を戻す**: `git diff lib/supabase/server.ts` で中身を確認し、不要なら `git restore lib/supabase/server.ts` で元に戻します。
3. **頼んでいない `SortSelect.tsx` を消す**: 新しいファイル（`??`）は `git restore` では消えないので、今回使わないならファイルを削除します（次の作業で使うなら、別のコミットに回します）。
4. **自分でテストを実行**: `npx vitest run` を自分で実行して、報告どおり通ることを確かめます。
5. **頼んだ2ファイルだけコミット**: `git add lib/sortBooks.ts lib/sortBooks.test.ts` の後に `git commit` します。

解説: 「テストも通りました」という報告は、差分を読むまでは「自己申告」にすぎません。`git status` で頼んでいないファイル（`server.ts`・`SortSelect.tsx`）を見つけたら、中身を確かめてから戻すか、別の作業として切り分けます。テストファイルが変更されている場合は、期待値が都合よく変えられていないかを必ず確認しましょう。

問題4の解答

- (a) **CLAUDE.md**（どの作業でも守ってほしい決まり）
- (b) **Hooks**（`PostToolUse` で Edit/Write の後に必ず実行。2-5）
- (c) **Permissions**（`deny` に `Read(.env.local)` を入れる。2-2）
- (d) **Skills**（`/check` のようなスキルを作る。7-6-1）
- (e) **Rules**（`.claude/rules/` に `paths` 付きで置く。`app/api/CLAUDE.md` でも可）

解説: (b) と (c) は「AI の判断に関係なく必ず効いてほしい」ことなので、お願いである CLAUDE.md ではなく、仕組みで強制する Hooks・Permissions を使います（1-4 の図11-8）。(a) と (e) の違いは、全体に効かせるか、特定の場所だけに効かせるかです。

問題5の解答

`No, keep planning` を選び、次のように伝えます。

```
計画から「3. app/layout.tsx の変更」を外してください。
今回は sortBooks.ts と sortBooks.test.ts の2ファイルだけを作ります。
ヘッダーのデザインは変更しないでください。
```

解説: 承認（`Yes, …`）を選ぶと、その計画のまま実装が始まってしまいます。計画の段階で範囲外の変更を見つけたら、`No, keep planning` で計画を直させるのがいちばん手間がかかりません。計画が直ったことを確認してから、`Yes, manually approve edits` で承認しましょう。

問題6の解答

- (a) **`Esc` で止める。**そのうえで「〇〇は編集しないでください」と指示し直します。
- (b) **`Esc` を2回（または `/rewind`）** で巻き戻しメニューを開き、2つ前の指示の時点を選んで「コードを戻す（Restore code）」を選びます。
- (c) **`git revert コミットID`** で、その変更を打ち消すコミットを新しく作ります。

解説: (b) のチェックポイントは、Claude が `Edit`・`Write` で行った変更なら戻せますが、`rm` などの Bash コマンドで行った変更は戻せません。(c) のように push 済みのコミットは、履歴を書き換える `git reset --hard` ではなく、打ち消しのコミットを追加する `git revert` を使うのが安全です（チームの履歴を壊しません）。

問題7の解答

`/clear` で会話をリセットし、ここまでに分かったこと（どの入力で、何が期待と違うか、どのテストが落ちるか）を書いた**新しい良い指示**で、もう一度頼みます。削除機能に移るときも、その前に `/clear` します。

解説: 同じ会話で失敗した修正を重ねると、コンテキスト（作業机）が失敗した案で埋まり、Claude がかえって混乱します。公式ドキュメントでも、2回以上修正してもうまくいかないときは `/clear` してやり直すことが勧められています。関係のない次の作業に移るときも `/clear` が基本です。CLAUDE.md は `/clear` の後も読み込まれるので、決まりごとが消える心配はありません。

問題8の解答

問題点: `sortBy` というメソッドは supabase-js に存在しません（AI のハルシネーション）。

見つけ方: `npx tsc --noEmit` で型チェックをすると、次のエラーが出ます（supabase-js 2.117.3 / TypeScript 7.0.2 で確認）。

```
lib/getBooks.ts(7,61): error TS2339: Property 'sortBy' does not exist on type
'PostgrestFilterBuilder<any, any, any, any[], "books", unknown, "GET", false>'.
```

正しい書き方:

```ts
// サーバー用の Supabase クライアントを作る関数を読み込む
import { createClient } from "@/lib/supabase/server";

// 書籍をタイトル順に取得する
export async function getBooks() {
  const supabase = await createClient();
  // select で取得し、order で並べ替える
  const { data } = await supabase
    .from("books")
    .select("*")
    .order("title", { ascending: true });
  return data;
}
```

解説: 見た目が自然なので読むだけでは見落としがちですが、型チェックなら確実に見つかります（9-8 の関所2）。あわせて、元のコードは `error` を受け取っていないので、取得に失敗しても気づけません。第7章のように `error` も受け取って確認するのが安全です。

> **最後に**: バイブコーディングの核心は「AIに丸投げ」ではなく「AIと対話しながら品質の高いコードを段階的に構築する」ことです。
>
> Claude Codeの全機能（Settings、Hooks、Skills、Rules、MCP、Plan Mode、Worktrees、Subagents、Memory）を適切なフェーズで活用し、要件を明確にし、段階的に進め、各段階でレビューすることで、プロダクション品質のアプリケーションを効率的に開発できます。
>
> 特に重要な3つの習慣：
>
> 1. 「**まだコードは書かないで**」— 方針合意を先に取る
> 2. 「**完了したら一度止まって**」— 段階的レビューで品質を維持
> 3. 「**/review → /commit**」— レビューしてからコミットする
>
> そして大前提として、`git status` で現状を確認してから依頼し、`git diff` で結果を確認してから commit、`git push` は人間が手動で行う、というGit運用を徹底することで、AIと安全に協調できます。